

Imitation Learning-Based Method for Automated Grasping of Wire Harness

Master's Thesis in Mechanical Engineering

Friedrich-Alexander-Universität Erlangen-Nürnberg
Institute for Factory Automation and Production Systems
Prof. Dr.-Ing. Jörg Franke



Author: Yue Xing, (M. Sc.) 23396082

Supervisors: Prof. Dr.-Ing. Jörg Franke
M. Sc. Simon Lamprecht

Submission date: 17.08.2026

Processing time: 6 Months

Declaration of Independent Work

I hereby declare that I, Yue Xing (23396082), have completed the submitted work independently and without unauthorized assistance from third parties, as well as without the use of undisclosed and, in particular, unauthorized aids. The work has not been submitted in the same or a similar form to any other examination authority and has not been accepted by any other examination authority as part of an examination.

The parts of the work that have been taken verbatim or in terms of meaning from other sources are clearly marked with citations of their origins (see citation guidelines). This also applies to drawings, sketches, visual representations, as well as sources from the internet. When selecting artificial intelligence (AI) tools, compliance with data protection regulations (e.g., GDPR, HAWKI) must be ensured; tools without sufficient data protection guarantees should be avoided. No personal or confidential data may be entered into AI systems.

The author is responsible for independent execution of the work, for the accuracy of the information presented, as well as for the reflective, professionally appropriate, and ethically justifiable use of AI.

Violations of the above rules will be classified as deception or attempted deception and will result in the examination being graded as "failed".

I, Yue Xing, hereby declare the extent to which I have used AI, including tools such as ChatGPT, Co-Pilot, HAWKI, DeepL Write, Grammarly or similar tools, in the preparation of this examination. I have used AI-supported tools as follows (please tick all applicable options):

- ☐ Not at all – The work was created entirely without the use of AI tools.
- ☐ For research and idea generation – AI was used to develop initial ideas, concepts or questions (must be documented in the work).
- ☐ For creating outlines/structuring texts – AI was used as an aid to structure the work, and own adjustments were made.
- ☐ For help with wording (e.g., improving sentence structure, style) – AI was used to improve style and readability, but not to generate the entire text.
- ☐ For correcting grammar and spelling – AI was used for linguistic optimization, but not to change the content or generate text.
- ☐ For translating texts – AI was used for translations, whereby all translated passages were revised and checked for accuracy.

- ☐ For creating or optimizing graphics, diagrams, or visual representations – AI was used for visual representations that were interpreted and classified in the work.
- ☐ For programming/code creation – AI was used to generate code, with all generated content being reviewed and commented on in a comprehensible manner.
- ☐ For the creation of AI-supported summaries of scientific texts – AI tools were used to create summaries, the content of which was commented on in a comprehensible manner.
- ☐ For other purposes, namely: _____

I confirm that I have provided complete and truthful information about all uses. I am aware that concealing or providing false information about the use of AI tools may be considered an attempt at deception and may have consequences under examination law.

Location, Date

Handwritten Signature

Abstract

The manipulation of flexible wire harnesses remains a challenging task in industrial automation because their shape changes continuously during handling and cannot be described by a single repeatable pose. This thesis investigates whether an end-to-end imitation-learning approach can learn automated wire-harness grasping directly from real-robot demonstrations without explicitly modelling the deformation of the harness or manually defining task-specific trajectories.

A leader–follower system based on SO-101 robot arms was established for teleoperation, demonstration collection, policy deployment, and real-world evaluation. The robot observes the task through a fixed top-view RGB camera, a front-view RGB camera mounted near the gripper, and its six-dimensional proprioceptive state. A dataset of 195 real-robot demonstration episodes was collected and organised using the LeRobot framework. Action Chunking with Transformers (ACT) was then trained to predict sequences of continuous arm and gripper actions. Ten policy configurations were investigated by varying the action chunk size, the number of executed action steps, the Kullback–Leibler regularisation weight, and the learning rate.

Policy performance was evaluated using both offline action-prediction metrics and closed-loop physical experiments. The real-robot evaluation consisted of 36 trials distributed across 18 cells of a 6×3 workspace grid, allowing both overall task success and spatial robustness to be assessed. Under the defined experimental conditions, the best tested configuration, with a chunk size of 150, 100 executed action steps, and a KL weight of 15, achieved 21 successful trials out of 36 (58.3%), compared with 8 out of 36 (22.2%) for the baseline configuration. Successful executions were not uniformly distributed across the workspace, with higher success observed in the right-hand region.

The experiments demonstrate that ACT can learn coordinated approach, grasp, transport, and lift behaviour for a flexible wire harness from a comparatively small real-world demonstration dataset. However, the configuration with the lowest offline action-prediction error was not the configuration with the highest physical task success. This shows that offline reconstruction metrics alone are insufficient for evaluating closed-loop manipulation policies. Real-robot task success, spatial coverage, prediction error, and motion characteristics should therefore be considered together when assessing learned robotic manipulation policies.

Keywords: wire harness manipulation, imitation learning, Action Chunking with Transformers, robot learning, deformable object manipulation, visuomotor policy

Contents

1. Introduction	1
1.1. Motivation and Background	2
1.2. Contribution	3
1.3. Structure of the Thesis	4
2. Fundamentals	5
2.1. Robot Manipulation	5
2.2. Imitation Learning	6
2.3. Visual Robot Learning	8
2.3.1. Visual Perception in Robot Learning	8
2.3.2. Visual Information for Wire Harness Manipulation	10
2.3.3. Image Representation and Observation	13
2.4. Transformer in Robot Learning	16
2.5. Conditional Variational Autoencoder	20
3. State of the Art	22
3.1. Overview of Robot Learning Policies	22
3.2. Action Chunking with Transformers	25
3.3. Robotic Foundation Models	29
3.4. Benchmarks for Robot Policies	32
3.4.1. Benchmark Task	33
3.4.2. Benchmark Dimensions Used in This Study	33
4. Method	35
4.1. Task Definition	35
4.1.1. Task Scope and Assumptions	37
4.1.2. Observation and Action Spaces	37
4.1.3. Task Stages and Termination Conditions	38
4.2. Dataset Collection Method	38
4.2.1. Hardware and Camera Configuration	38
4.2.2. Workspace Partitioning and Harness Placement	39
4.2.3. Leader–Follower Demonstration Procedure	39
4.2.4. Robot State and Action Acquisition	40
4.2.5. Sampling Frequency and Temporal Synchronisation	40
4.2.6. Dataset Composition	41
4.2.7. Dataset Storage Format	42

4.3. Policy Training	42
4.3.1. Image Preprocessing	43
4.3.2. State and Action Standardisation	44
4.3.3. Action-Chunk Construction	44
4.3.4. Training Objective and Baseline Configuration	45
4.3.5. Training Duration and Model Output	45
4.4. Testing Protocol	46
4.4.1. Spatial Trial Allocation	46
4.4.2. Controlled Experimental Conditions	46
4.4.3. Trial Initialisation	46
4.4.4. Policy Execution	47
4.4.5. Trial Outcome Assessment	47
4.4.6. Qualitative Failure Analysis Framework	48
4.4.7. Termination and Safety Conditions	49
4.4.8. Invalid Trials and Repetition Procedure	50
4.4.9. Evaluation Metrics	51
4.4.10. Motion Smoothness	52
4.5. ACT Parameter Tuning Strategy	52
4.5.1. chunk_size / n_action_steps	52
4.5.2. KL Weight	54
4.5.3. Learning Rate	54
5. Implementation	56
5.1. Hardware Setup	56
5.1.1. Robot Manipulation Platform	56
5.1.2. Camera System	56
5.1.3. Experimental Workbench	57
5.1.4. Deployment Computer	57
5.1.5. Training Workstation	57
5.2. Software Setup	59
5.2.1. LeRobot	59
5.2.2. Deployment Software Environment	60
5.2.3. Training Software Environment	61
5.2.4. PyTorch and CUDA Acceleration	61
5.2.5. Supporting Libraries	62
5.2.6. Transfer from Training to Deployment	62
5.3. Data Structure	62
5.3.1. Dataset Overview	62
5.3.2. Episode Structure	63
5.3.3. Image Data Structure	64

5.3.4. Robot State and Action Structure	65
5.3.5. Dataset File Structure	66
5.3.6. Data Preprocessing Structure	68
5.3.7. Action Chunk Data Structure	69
5.4. ACT Implementation	70
5.4.1. ACT Input Interface	70
5.4.2. Visual Feature Extraction	70
5.4.3. Transformer and CVAE Implementation	71
5.4.4. Action Chunk Prediction and Execution	72
5.4.5. ACT Loss Implementation	73
5.4.6. Pre-processing and Policy Output	73
5.4.7. Online Inference and Runtime Handling	74
5.5. Training Configuration	74
5.5.1. Training Dataset Configuration	75
5.5.2. Baseline Training Configuration	75
5.5.3. Training Procedure	75
5.5.4. Hyperparameter Experiment Configuration	77
5.5.5. Training Runtime and Model Storage	79
6. Results	80
6.1. Evaluation Overview	80
6.2. Offline Policy Evaluation Results	80
6.3. Real-Robot Task Success	82
6.4. Spatial Distribution of Successful Grasps	83
6.5. Failure Case Analysis	84
7. Discussion	85
7.1. Interpretation of Results	85
7.2. Strengths of the Proposed System	86
7.3. Limitations	87
8. Summary and Outlook	90
8.1. Summary	90
8.2. Main Findings	91
8.3. Future Work	91
Bibliography	95
A. Supplementary Materials	101
A.1. Complete Spatial Evaluation Results	101
A.2. Complete Offline Evaluation Results	101

B. Reproducibility and Code Availability	105
B.1. Offline Evaluation Procedure	105
B.2. Spatial Heat-Map Generation	105

List of Figures

4.1. Overview of the experimental method from task definition and demonstration collection to policy comparison.	35
4.2. Wire harness used as the manipulation object. The harness contains a multi-pin connector, multiple flexible conductors, a fabric-wrapped central section, and separated free wire ends.	36
4.3. Workspace partition used for balanced data collection and spatial evaluation. Columns A–B, C–D, and E–F form the left, centre, and right collection regions, respectively; the 6×3 grid defines the 18 evaluation cells.	39
5.1. Overview of the experimental workbench, including the SO101 leader–follower system, aluminium mounting frame, fixed camera and illumination arrangement, and the spatially divided manipulation surface.	58
6.1. Representative offline Action L1 error curves averaged over consecutive 1000-frame windows.	81
6.2. Spatial distribution of successful grasps for representative configurations. Each region was tested twice.	83
A.1. <i>C100–A80</i> policy successful-grasp heat map.	101
A.2. <i>C150–A100–KL15</i> policy successful-grasp heat map.	101
A.3. <i>C150–A100</i> policy successful-grasp heat map.	102
A.4. Baseline policy successful-grasp heat map.	102
A.5. <i>KL15</i> policy successful-grasp heat map.	102
A.6. <i>KL5</i> policy successful-grasp heat map.	102
A.7. $LR3 \times 10^{-5}$ policy successful-grasp heat map.	103
A.8. <i>KL18</i> policy successful-grasp heat map.	103
A.9. <i>KL18</i> policy offline evaluation.	103
A.10. <i>C100–A80</i> policy offline evaluation.	103
A.11. <i>C150–A100</i> policy offline evaluation.	104
A.12. <i>C100–A100–KL15</i> policy offline evaluation.	104
A.13. <i>KL5</i> policy offline evaluation.	104

List of Tables

3.1. Simplified comparison of ACT, SmoVLA, and π_0	31
5.1. Hardware configuration of the data-collection and deployment platform.	57
5.2. Hardware configuration of the ACT training workstation.	59
5.3. Software environment of the Jetson deployment platform.	60
5.4. Software environment used for ACT training.	61
5.5. Baseline ACT configuration used in this study.	76
5.6. ACT configurations used in the hyperparameter experiments.	78
6.1. Offline policy replay metrics.	81
6.2. Real-robot success rates for the evaluated ACT configurations.	82
6.3. Spatial coverage of the available real-robot evaluations.	83

List of Equations

2.1. SO-101 proprioceptive state	6
2.2. Policy observation	6
2.3. SO-101 target action	6
2.4. General imitation-learning policy	7
2.5. Visuomotor behaviour-cloning policy	7
2.6. Behaviour-cloning reconstruction loss	7
2.7. Vision-based robot policy	8
2.8. Visual-proprioceptive observation	9
2.9. Multi-view policy observation	9
2.10. Visual and proprioceptive state	12
2.11. Image-based DLO forward model	13
2.12. RGB image representation	13
2.13. Image preprocessing operator	13
2.14. Encoded visual feature	14
2.15. Single-view visual observation	14
2.16. Multi-scale visual feature hierarchy	14
2.17. Multi-camera image set	15
2.18. View-specific feature encoding	15
2.19. Multi-view feature fusion	15
2.20. Wire-harness policy observation	16
2.21. Front and top camera feature encoding	16
2.22. Multi-view ACT action prediction	16
2.23. Scaled dot-product attention	16
2.24. Behaviour-cloning demonstration trajectory	17
2.25. Context-conditioned future-action policy	17
2.26. Any2Policy cross-attention	19
2.27. DreamVLA future world embedding	19
2.28. DreamVLA future prediction heads	19
2.29. DreamVLA action-sequence generation	19
2.30. CVAE posterior distribution	20
2.31. CVAE reparameterisation	20
2.32. General CVAE objective	21
3.1. General robot policy mapping	22
3.2. Thesis policy observation	22
3.3. Learning-from-demonstration dataset	22
3.4. Direct policy approximation	23

3.5. State-transition model	23
3.6. Single-step action policy	23
3.7. Multi-step sequence policy	24
3.8. Conventional behaviour-cloning policy	25
3.9. ACT future-action policy	25
3.10. Generic ACT observation	26
3.11. Generic robot action space	26
3.12. ACT action chunk	26
3.13. ACT latent posterior distribution	26
3.14. ACT decoder output	26
3.15. ACT training objective	27
3.16. ACT action reconstruction loss	27
3.17. ACT KL regularisation	27
3.18. Temporal-ensemble action	27
3.19. Temporal-ensemble decay weights	27
4.1. Six-dimensional robot state	38
4.2. Six-dimensional robot action	38
4.3. Recorded frame tuple	41
4.4. Demonstration episode	41
4.5. Demonstration dataset	42
4.6. Total recorded duration	42
4.7. ACT training sample	43
4.8. Raw RGB image representation	43
4.9. Image tensor conversion	43
4.10. ImageNet image normalisation	43
4.11. Robot-state standardisation	44
4.12. Robot-action standardisation	44
4.13. Action inverse standardisation	44
4.14. Target action chunk	44
4.15. Predicted action chunk	45
4.16. Target control period	47
4.17. Binary trial outcome	48
4.18. Timeout indicator	49
4.19. Valid-trial decomposition	50
4.20. Real-robot task-success rate	51
4.21. Successful-region coverage	51
4.22. Offline Action L1 error	51
4.23. First-order action smoothness	52
5.1. Camera tensor dimensions	60

5.2. Total dataset frame count	63
5.3. HWC-to-CHW tensor conversion	65
5.4. Scaled RGB tensor domain	65
5.5. Batched camera tensor	65
5.6. Two-camera tensor list	65
5.7. Ordered state and action channels	65
5.8. Image-processing pipeline	68
5.9. Pixel-value scaling	68
5.10. Generic feature normalisation	68
5.11. Action denormalisation	69
5.12. Action-chunk temporal offsets	69
5.13. Action padding mask	69
5.14. Masked action L1 loss	69
5.15. ResNet camera feature map	71
5.16. Zero latent at inference	72
5.17. Batched predicted action chunks	72
5.18. Baseline action-queue duration	73
5.19. Latent KL-divergence loss	73
5.20. Total ACT loss	73
5.21. Executed action subsequence	78

List of Abbreviations

ACT	Action Chunking with Transformers
AI	Artificial Intelligence
API	Application Programming Interface
CHW	Channel–Height–Width
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
CVAE	Conditional Variational Autoencoder
DLO	Deformable Linear Object
FPS	Frames Per Second
GPU	Graphics Processing Unit
HWC	Height–Width–Channel
IL	Imitation Learning
KL	Kullback–Leibler
L1	Least Absolute Error
LfD	Learning from Demonstration
LR	Learning Rate
MPC	Model Predictive Control
RGB	Red–Green–Blue
RGB-D	Red–Green–Blue and Depth
SO101	SO-101 Robot Arm
VLA	Vision–Language–Action
VLM	Vision–Language Model
YOLO	You Only Look Once

1. Introduction

Automotive wire harnesses connect electrical and electronic components, yet their production and handling still require substantial manual work. Routing, bundling, grasping, connector handling, and insertion are difficult to automate because a harness is not a rigid part with a single repeatable pose. It bends, twists, slips, becomes occluded, and changes configuration as soon as it is grasped. These characteristics limit the direct transfer of conventional position-based automation methods from rigid-component assembly to wire-harness production [1], [2].

The practical problem addressed by this thesis is therefore the reliable robotic acquisition of a flexible wire harness whose initial position and local shape vary within the workspace. A robot must locate an accessible section, approach it without displacing it, close the gripper at the correct time, retain it during motion, and lift it despite uncertain deformation and contact. Conventional systems can solve individual wire-handling tasks using calibrated multi-view geometry, markers, force sensing, specialised end effectors, and task-specific motion primitives. However, extending such systems to additional configurations often requires renewed modelling, calibration, and rule design [3], [4], [5].

This thesis investigates whether a demonstration-based visuomotor policy can learn the required coordination directly from real robot data. The policy receives two RGB views and the current robot state and predicts continuous arm and gripper actions. Action Chunking with Transformers (ACT) is used because it predicts temporally coherent action sequences rather than isolated commands [6]. The central research question is not merely whether a successful video can be produced, but how reliably the learned behaviour works across the defined workspace and how the ACT chunk length, execution horizon, KL regularisation, and learning rate affect physical task success.

To answer this question, teleoperated demonstrations are collected with a leader–follower SO-101 system, organised in the LeRobot data format, and used to train ten ACT configurations. The policies are compared through offline action reconstruction and a spatially stratified real-robot protocol covering 18 workspace cells. This combination exposes two issues that selected video examples cannot show: success may be concentrated in favourable regions, and a low offline prediction error may not correspond to reliable closed-loop robot behaviour. The study consequently treats physical success and spatial coverage as the primary evidence, while offline error and motion variation are used as diagnostic measures.

1.1. Motivation and Background

Industrial robots achieve high accuracy and repeatability when objects, fixtures, trajectories, and process conditions are known in advance. This operating principle supports highly structured processes such as welding, painting, material handling, and machine loading. Wire-harness production differs because many operations involve compliant parts whose geometry is not preserved during handling. Automotive harnesses also contain branches, connectors, tapes, clips, and wires with different mechanical properties. Although individual production stages have been automated, routing, insertion, and bundling remain important research areas [1], [2], [7].

A wire harness can be described as a deformable linear object rather than a rigid body. Its observable configuration depends on material properties, gravity, friction, contact, gripping force, and the previous robot motion. Moving one segment can change the position of another segment or connector, and parts of the harness may disappear behind the robot, the gripper, or the harness itself. Perception and control are therefore coupled: a selected grasp changes the object configuration, and that change can invalidate the state estimate or trajectory on which the grasp was based. Exact reconstruction of the complete three-dimensional configuration is possible only under additional sensing and modelling assumptions, and even then uncertainty remains [4], [8].

Existing systems illustrate both the feasibility and the engineering cost of automated harness handling. Jiang et al. combined three robot arms, several cameras, laser sensing, artificial markers, specialised end effectors, and predefined sequences for automotive harness assembly [3]. Zürn et al. used multi-stereo observations and a learned spline representation to estimate grasp poses for harnesses in a bin [9]. España et al. generated candidate grasp points from image segmentation and clustering, but relied on strong foreground contrast and a fixed camera view [10]. Zhang et al. combined a learned wire-dynamics model with model-predictive control, tension regulation, and predefined insertion primitives [11]. These methods solve valuable parts of the problem, but their perception, planning, and control components remain tailored to particular harnesses, sensors, fixtures, or operation sequences.

Imitation learning offers a complementary route. Instead of programming a separate decision rule for every visible shape and contact condition, a human operator demonstrates the desired motion and a policy learns the relationship between observations and actions. For the present task, the demonstrations contain global and local camera views, the robot configuration, and continuous arm and gripper commands. This allows the model to associate the appearance of the harness and gripper with the actions used to approach, grasp, transfer, and lift it. Human demonstrations have also been used to transfer grasp preferences to new wire-harness configurations, supporting the relevance of learning from demonstration for this object class [12].

The learning-based formulation does not remove the physical difficulty of the task. Be-

haviour cloning is sensitive to demonstration coverage and to the distribution shift created by its own execution errors. If the robot deviates from the demonstrated states, later observations may be unfamiliar and errors may accumulate. Demonstrations collected from only one operator can also encode spatial or stylistic bias. A policy must therefore be assessed through repeated closed-loop trials rather than only through training loss, offline reconstruction error, or selected successful videos [13], [14].

ACT is selected because the task consists of temporally related phases rather than independent single-step movements. Its action-chunk representation can encode a coordinated approach, gripper closure, transfer, and lift within a future action sequence [6]. The predicted chunk length and the number of actions executed before the next observation determine a trade-off between temporal coherence and feedback frequency. KL regularisation affects the latent representation of demonstration variability, while the learning rate influences optimisation. These parameters are consequently treated as experimental variables rather than implementation details.

LeRobot provides the common interface used to record demonstrations, organise multi-modal episodes, train ACT, and deploy the resulting policy [15]. It is an enabling framework rather than the research contribution itself. The investigated task is deliberately constrained to one harness, one fixed camera, the camera move with the wrist, a fixed background, and a known reachable table area. Within this scope, the research problem is formulated as follows: determine whether ACT can learn a valid approach–grasp–transfer–lift behaviour from a small real-world demonstration dataset, quantify how its selected parameters affect task success, and identify where in the workspace the learned behaviour is and is not reliable. Generalisation to unseen harness types, cluttered industrial scenes, connector insertion, and production-level reliability is outside the demonstrated scope.

1.2. Contribution

This thesis makes the following five contributions:

1. It formulates flexible wire-harness acquisition as a reproducible visuomotor imitation-learning task with explicit success, failure, timeout, and safety conditions. This converts an informal grasping demonstration into a testable physical task.
2. It provides a real-robot demonstration dataset of 195 episodes with two complementary RGB views, robot states, and continuous actions. The dataset is spatially balanced across three coarse collection regions and is published in a standard LeRobot-compatible format for reuse.
3. It establishes a spatially stratified evaluation protocol using a 6×3 workspace grid and two physical trials per cell. The resulting heat maps distinguish broad but unreliable behaviour from repeatable success confined to favourable positions.

4. It presents a controlled comparison of ten ACT configurations and provides empirical evidence that the action-chunk length, executed action horizon, and KL weight materially affect closed-loop performance. Under the defined conditions, the best tested configuration achieved 21/36 successes compared with 8/36 for the baseline.
5. It demonstrates that offline action reconstruction does not reproduce the ranking obtained in physical trials. This result supports a combined evaluation in which real-robot task success and spatial coverage are primary, while offline error and action variation remain diagnostic metrics.

1.3. Structure of the Thesis

Chapter 2 introduces the fundamentals of robot manipulation, imitation learning, visual robot learning, Transformers, and conditional variational autoencoders. Chapter 3 reviews ACT, alternative robot policies, robotic foundation models, and relevant benchmarks. Chapter 4 defines the task, data collection, policy-training method, and physical evaluation protocol. Chapter 5 documents the hardware, software, dataset, ACT implementation, and training configurations. Chapter 6 reports the offline and real-robot results, including spatial success distributions and observed failure cases. Chapter 7 interprets these findings and discusses strengths and limitations. Chapter 8 summarises the conclusions and outlines future extensions toward more diverse harnesses, connector recognition, motion planning, and broader policy comparisons.

2. Fundamentals

This chapter presents the theoretical foundations of the vision-based wire harness manipulation system developed in this thesis. Robotic manipulation is described through perception, state representation, action spaces, end effectors, joint-position control, and trajectory execution. These concepts define how information about the robot and its environment is represented and how policy outputs are converted into physical robot motion.

The learning process is based on imitation learning, in which an observation-to-action mapping is learned from teleoperated demonstrations. Visual robot learning extends this formulation by combining camera observations with proprioceptive robot states. Multi-view images provide information about the workspace, wire harness configuration, gripper position, and changes caused by robot actions, while joint states describe the internal configuration of the robot.

Transformer architectures provide a mechanism for processing visual features, robot states, and action sequences within a unified model. Their attention mechanism supports the representation of relationships between different input elements and the prediction of multiple future actions. The Conditional Variational Autoencoder represents variations within human demonstrations through a latent variable and forms an important component of the Action Chunking with Transformers policy. These concepts establish the theoretical basis for the policy design, experimental method, and system implementation presented in the following chapters.

2.1. Robot Manipulation

Robot manipulation concerns the controlled physical interaction between a robot and objects in its environment. A manipulation system combines perception, state estimation, action generation, and low-level control to change an object state while respecting the kinematic and physical constraints of the robot. Modern learning-based manipulation commonly represents this interaction as a sequence of observations and actions rather than as one isolated command [16], [17].

At discrete time step t , the observation o_t contains the information available to the policy, while the action a_t denotes the command selected for execution. Visual observations describe task-relevant properties of the scene, and proprioception describes the internal robot configuration. The policy is a mapping from the current observation, or a history of observations, to one or more actions. The following notation is used consistently throughout this thesis: t is a discrete time-step index and j is a component index.

For the robot used in this study, the proprioceptive state at time step t is represented by

$$s_t = \begin{bmatrix} s_{1,t} & s_{2,t} & s_{3,t} & s_{4,t} & s_{5,t} & s_{6,t} \end{bmatrix}^\top \in \mathbb{R}^6, \quad (2.1)$$

Here, s_t denotes the complete proprioceptive state vector, $s_{j,t}$ denotes its j th component at time step t , and $(\cdot)^\top$ denotes transposition. The set $\mathbb{R}^6$ indicates that s_t contains six real-valued components. Components $s_{1,t}$ to $s_{5,t}$ are the measured arm-joint positions, and $s_{6,t}$ is the measured gripper position.

The policy observation is defined as

$$o_t = (I_t^{\text{top}}, I_t^{\text{front}}, s_t), \quad (2.2)$$

In this expression, o_t is the complete observation at time step t , I_t^{top} is the RGB image from the fixed top-view camera, I_t^{front} is the RGB image from the gripper-mounted front-view camera, and s_t is the proprioceptive state defined above. The superscripts identify the camera view; they are labels rather than mathematical powers.

The corresponding target-position action is represented as

$$a_t = \begin{bmatrix} a_{1,t} & a_{2,t} & a_{3,t} & a_{4,t} & a_{5,t} & a_{6,t} \end{bmatrix}^\top \in \mathbb{R}^6, \quad (2.3)$$

Here, a_t is the complete action vector and $a_{j,t}$ is its j th component. Components $a_{1,t}$ to $a_{5,t}$ specify target arm-joint positions, whereas $a_{6,t}$ specifies the target gripper position. Joint-position control converts these targets into physical motion through the low-level actuator interface; the policy does not directly command motor torque. Consecutive actions form a trajectory, and an action-chunking policy may predict a finite future sequence that is executed in temporal order. Hardware-specific ranges, command names, safety limits, and the exact SO-101 interface are implementation details and are therefore documented in Chapters 4 and 5 rather than in this general fundamentals section.

2.2. Imitation Learning

Imitation Learning (IL), also referred to as Learning from Demonstration (LfD), Programming by Demonstration, or Teaching by Showing, is an important paradigm in robot learning. In the context of robotic learning, one of the early representative discussions can be traced back to [18]. This work describes robot learning from demonstration as a process in which a robot learns a task by observing a human or another robot performing it. The robot may either directly mimic the motion of the demonstrator or learn how the demonstrator

acts, reacts, and handles errors in different situations, namely learning a policy. The study also emphasizes that simply reproducing demonstrated motions may be insufficient, since the robot and the human demonstrator may differ in dynamics, morphology, and execution accuracy. Therefore, a more effective approach is to learn a task model and a reward function from demonstrations and then compute a policy that is suitable for the robot itself.

In a more general formulation, [17] defines imitation learning as the process by which an agent acquires skills or behaviors by observing a teacher demonstrating a given task. The central idea is to learn a mapping from states or observations to actions from demonstration data. A demonstration can be represented as an input-output pair (x, y) , where x denotes a feature vector describing the current state and y denotes the action performed by the demonstrator. The learning objective is to obtain a policy π that predicts the action from the current state:

$$u(t) = \pi(x(t), t, \alpha), \quad (2.4)$$

where $u(t)$ is the predicted action, $x(t)$ is the state or feature representation at time t , and α denotes the learnable policy parameters. In the case of behavior cloning for visuomotor robot control, this idea can be written as learning a parameterized policy:

$$\pi_{\theta}(a_t \mid o_t), \quad (2.5)$$

where o_t denotes the current observation, a_t denotes the action, and π_{θ} is the policy learned from demonstrations. The model is commonly trained by minimizing the deviation between the predicted action $\hat{a}_t$ and the expert action a_t :

$$L_{BC} = \sum_t \|a_t - \hat{a}_t\|. \quad (2.6)$$

In this way, robot control is transformed into a supervised learning problem, allowing the robot to learn an observation-to-action mapping from expert demonstrations without explicitly programming the entire control logic or manually designing task-specific reward functions. For robotic manipulation, [16] provides a survey of imitation-learning methods. The study points out that traditional robotic manipulation relies heavily on manual coding, which is difficult to adapt to complex and uncertain environments. Imitation learning avoids learning skills from scratch by using expert demonstrations and improves adaptability, learning efficiency, and compatibility with other learning mechanisms. The survey summarizes the imitation learning process for robotic systems into three main components: demonstration, representation, and learning algorithm. Demonstrations can be categorized into indirect and direct demonstrations, while direct demonstrations include kinesthetic teaching and teleoperation. Teleoperation is particularly attractive because it provides high safety, wide applicability, and high-quality demonstration samples, although fine manipulation may still suffer from missing contact or force information. For wire harness manipulation, [12]

demonstrates how imitation learning can be applied to deformable object grasping. Since wire harnesses have uncertain poses and are difficult to model analytically, the proposed system allows human operators to teach the robot grasping actions through hand demonstrations. Furthermore, [19] combines deep imitation learning with human teleoperation for humanoid loco-manipulation. This shows that teleoperation demonstrations can reduce the complexity of manual programming and provide a scalable way to collect data for complex manipulation tasks. By combining high-level action abstractions with a low-level controller, the learned policy can handle complex sensorimotor behaviors in high-degree-of-freedom robotic systems more efficiently. Inspired by these studies, this thesis adopts imitation learning as the main learning paradigm for wire harness grasping experiments. Instead of manually programming fixed grasping trajectories or explicitly designing reward functions, expert demonstrations are collected through teleoperation, and the robot learns a mapping from multimodal observations to actions. Specifically, the policy takes camera images and robot states as observations and outputs arm and gripper actions, enabling the robot to learn closed-loop behaviors required for grasping, lifting, and subsequent manipulation of wire harnesses. This approach follows the fundamental principle of imitation learning, namely learning an observation-to-action mapping from expert demonstrations, and is well suited for wire harness manipulation where deformation, partial observability, and task variability make rule-based control difficult.

2.3. Visual Robot Learning

2.3.1. Visual Perception in Robot Learning

Conventional robotic systems often rely on explicitly specified task goals, object or environment models, coordinate transformations, motion plans, and task-specific reward functions. Such model-based pipelines can achieve high accuracy in structured environments, but adapting them to new objects, tasks, or environmental configurations usually requires additional engineering. Visual robot learning instead treats camera observations as part of the robot state and learns task-relevant relationships between visual observations and robot actions. Previous studies have shown that control policies can be learned directly from visual inputs. However, they have also demonstrated that end-to-end learning from pixels can be data-intensive and strongly depends on whether the learned visual representation preserves the information required for control [20], [21], [22].

A general vision-based robot policy can be formulated as

$$\mathbf{a}_t = \pi_{\theta}(\mathbf{o}_t), \quad (2.7)$$

where π_{θ} denotes a policy parameterized by θ , $\mathbf{o}_t$ is the observation available at time step t ,

and $\mathbf{a}_t$ is the robot action. In an image-only formulation, $\mathbf{o}_t$ may consist of a single RGB image. In practical manipulation systems, however, visual information is commonly combined with proprioceptive information because an image describes the external scene but does not necessarily provide an accurate representation of the robot's internal configuration. The observation can therefore be written as

$$\mathbf{o}_t = (\mathbf{I}_t, \mathbf{q}_t), \quad (2.8)$$

where $\mathbf{I}_t$ represents the current camera image and $\mathbf{q}_t$ denotes the current joint configuration or another form of proprioceptive state. The corresponding policy is therefore not based only on visual appearance. Instead, it combines information about the environment with information about the robot itself in order to generate an appropriate action.

For a multi-camera system, the observation can be extended to

$$\mathbf{o}_t = (\mathbf{I}_t^{\text{front}}, \mathbf{I}_t^{\text{top}}, \mathbf{q}_t). \quad (2.9)$$

Different camera viewpoints can provide complementary information about the same manipulation process. A front-facing camera can provide detailed information about the local interaction between the gripper, wire harness, and plug, whereas a top-facing camera can provide a more global representation of the workspace and the spatial arrangement of the objects. Multi-view visual learning can additionally encourage the representation to preserve the underlying physical state while becoming less sensitive to viewpoint-specific appearance. For example, multi-frame Time-Contrastive Networks learn similar embeddings for synchronized observations of the same event from different viewpoints and use temporal changes to encode both static and motion-related properties [20].

It is important to distinguish *visual perception* from *visual robot learning*. Visual perception mainly addresses questions such as which objects are present, where they are located, or to which semantic category they belong. Visual robot learning addresses the additional question of what the robot should do on the basis of the perceived scene. Semantic recognition alone is therefore not sufficient for manipulation. A representation that is invariant to object pose or spatial position may be useful for image classification, but these variables are often essential for robot control [20]. Similarly, detecting and localizing an object does not directly specify where the robot should establish contact or how it should move the object after contact. Bahl et al. therefore represent visual affordances using contact points and post-contact trajectories, providing actionable information about both *where* and *how* an object may be manipulated [21].

Vision can consequently contribute to robot learning at several levels. It may serve as the direct input to a visuomotor policy, as a source of latent state representations, as an affor-

dance representation, as a learned reward signal, or as a basis for discovering higher-level skills. Dwibedi et al. use multi-frame visual observations to learn representations that encode position and velocity and can replace parts of the true state used by a continuous control policy [20]. Mohtasib et al. use image-based task-success classifiers to estimate dense or sparse visual rewards for reinforcement learning [22]. Bahl et al. predict contact locations and post-contact motion trajectories from images and apply them to imitation learning, exploration, goal-conditioned learning, and reinforcement-learning action spaces [21]. At a higher level of abstraction, Extract uses differences between vision-language-model embeddings to group image–action trajectories into discrete, semantically aligned skills that can subsequently be selected and parameterized by a reinforcement-learning policy [23].

Despite these advantages, the effectiveness of visual robot learning depends strongly on the quality and coverage of the training data. The image must contain sufficient information to distinguish task-relevant states, including object position, contact state, motion direction, and task progress. Occlusion, camera motion, viewpoint changes, domain shift, and states that are absent from the training data can lead to ambiguous visual representations and incorrect actions. In particular, visual reward models may generate inaccurate rewards when the goal is not clearly visible, while end-to-end visual policies may fail when the deployment observations differ substantially from the training data [20], [21], [22].

In the system considered in this thesis, the observation in Equation 2.9 consists of a front-view image, a top-view image, and the current joint state of the robot. These inputs are provided to the Action Chunking with Transformers policy. The front view captures local grasping and contact information, the top view describes the global arrangement of the wire harness and the workspace, and the joint state provides the robot configuration that cannot be inferred reliably from the images alone. The learned policy therefore establishes a direct relationship between multi-view visual observations, proprioceptive information, and robot actions.

2.3.2. Visual Information for Wire Harness Manipulation

General visual robot learning considers camera images as observations from which a robot can infer task-relevant information and generate corresponding actions. When this principle is applied to wire harness manipulation, however, the visual observation must represent an object whose configuration changes continuously during interaction. Wires and cables are commonly categorized as deformable linear objects (DLOs). In contrast to rigid objects, which can often be represented by a six-degree-of-freedom pose, DLOs possess a high-dimensional configuration space and can change their shape under grasping, pulling, gravity, contact, and environmental constraints. Consequently, modelling and estimating their complete state remain difficult problems in robotic manipulation [24], [25].

For this reason, the RGB images used in this work are not interpreted as direct measure-

ments of a complete three-dimensional wire model. Instead, they are treated as appearance-based observations that contain task-relevant information about the spatial distribution of the wire harness, the relative configuration of the robot and the wire, and the changes caused by previously executed actions. A learning-based policy can extract such information directly from demonstration data without requiring an explicitly reconstructed geometric model of the entire wire.

Wire harness position and configuration.

The first function of visual observation is to represent the position and current configuration of the wire harness in the workspace. Depending on the camera viewpoint and the amount of occlusion, an RGB image can contain observable cues related to the location of the wire on the working surface, its main extension direction, local bending, and its position relative to the robot gripper. It can also indicate whether a visible section of the wire remains on the table or has been lifted during manipulation.

Vision-based manipulation of flexible USB wires has demonstrated that visual feedback can be used to guide wire grasping and subsequent manipulation operations [25]. In this type of system, the visually observed wire configuration is not necessarily converted into a complete physical model. Instead, image features describing the currently visible wire arrangement are used to determine or correct the robot motion.

A similar image-space perspective is adopted by Lee et al., who learn a visual forward model for deformable linear object manipulation directly from real-world images [26]. Their method transforms observations and actions into a canonical image representation and predicts the visual result of a manipulation action. This demonstrates that task-relevant information about a DLO configuration and its changes can be represented in image space without explicitly recovering all physical parameters of the object.

In the present learning-based system, the visual encoder is therefore expected to identify image regions and appearance patterns that are relevant to the demonstrated actions. These patterns may include the visible wire body, its local direction near a potential grasping region, the distance between the wire and the gripper, and changes in the wire appearance after lifting. These properties are not manually defined as separate geometric variables. Instead, their relationship to the demonstrated robot actions is learned from the training data.

Robot and gripper state.

Visual observations also provide information about the external relationship between the robot, the gripper, the wire harness, and the surrounding workspace. This information should be distinguished from the proprioceptive state of the robot. The proprioceptive state is measured internally by the robot and may include joint positions and the gripper position.

The visual state, in contrast, represents externally observable relationships in the camera image.

The combined policy observation can be expressed as

$$\mathbf{o}_t = (\mathbf{I}_t, \mathbf{s}_t^{\text{prop}}), \quad (2.10)$$

where $\mathbf{I}_t$ denotes one or multiple RGB observations at time step t , and $\mathbf{s}_t^{\text{prop}}$ denotes the proprioceptive robot state. These two sources of information are complementary. Joint measurements describe the kinematic configuration of the manipulator, but they do not directly determine whether the gripper has successfully enclosed the wire or whether the wire has moved as intended. Conversely, an RGB image can provide evidence about the relative position of the gripper and the wire, but it does not directly measure contact force, wire tension, or the exact physical contact state.

Relevant visual cues include whether the gripper is approaching the wire, from which side the gripper approaches it, whether the visible wire lies between the gripper fingers, and whether the wire moves together with the gripper after the grasping command. The image may also reveal a large deviation between the expected and actual grasping region. Such information complements the joint state because the same robot configuration can produce different task outcomes when the initial wire position or local deformation changes.

Nevertheless, visual state estimation is limited by occlusion. Galassi and Palli note that computer vision can be used before a wire is grasped, whereas the gripper itself can occlude the wire after grasping, motivating the use of tactile sensing for in-hand perception [27]. In the present RGB-based system, this limitation means that the policy can only use the wire sections and robot–object relationships that remain visible. It should not be assumed that the camera always observes the exact contact state inside the gripper.

Environmental changes and closed-loop observation.

The third function of visual information is to represent changes in the environment during task execution. Since a wire harness deforms under robot motion, an action can alter not only the position of the grasped region but also the visible shape and position of other wire sections. Relevant visual changes include displacement of the wire on the working surface, lifting of the grasped section, local straightening or bending, and changes in occlusion caused by the moving arm and gripper.

Gao et al. use visual feedback during the grasping and manipulation of flexible wires, illustrating that updated image information can be used to guide the execution of wire operations rather than relying exclusively on a predetermined motion [25]. Lee et al. explicitly formulate the relation between a current visual observation, an action, and the subsequent visual state using an image-based forward model [26]:

$$\hat{\mathbf{I}}_{t+1} = f_{\theta}(\mathbf{I}_t, \mathbf{a}_t), \quad (2.11)$$

where $\mathbf{I}_t$ is the current image, $\mathbf{a}_t$ is the applied manipulation action, and $\hat{\mathbf{I}}_{t+1}$ is the predicted image after the action. Although the ACT policy used in this work does not explicitly employ this forward-model formulation, the equation illustrates why successive images are informative: the updated observation contains evidence about the physical consequences of the previous robot action.

Accordingly, the manipulation task is not based only on a single image captured at the beginning of an episode. At each policy update, the latest visual observation provides the policy with an updated description of the workspace. The subsequent action prediction can therefore depend on the current wire configuration and on the effects of earlier movements. This observation-action-observation process introduces visual feedback into the execution and reduces reliance on a completely open-loop trajectory.

Overall, visual information in wire harness manipulation should be understood as a partial and task-dependent observation rather than a complete physical state measurement. RGB images provide information about the visible wire configuration, the relative relationship between the gripper and the wire, and action-induced changes in the environment. Proprioceptive measurements provide complementary information about the internal robot configuration. Their combination allows the policy to associate visual scene changes with robot actions while acknowledging limitations caused by occlusion, projection ambiguity, and the absence of direct force or tactile measurements.

2.3.3. Image Representation and Observation

Visual observations constitute a primary source of environmental information for vision-based robotic policies. At time step t , an RGB image captured by camera c can be represented as

$$I_t^c \in \mathbb{R}^{H_{\text{img}} \times W_{\text{img}} \times 3}, \quad (2.12)$$

where H_{img} and W_{img} denote the image height and width, respectively, and the last dimension corresponds to the three RGB channels. Before being passed to the visual encoder, the image is transformed by a preprocessing operator $\mathcal{P}(\cdot)$:

$$\tilde{I}_t^c = \mathcal{P}(I_t^c). \quad (2.13)$$

In this notation, $\mathcal{P}(\cdot)$ generally represents the conversion of the original image into a tensor with the input dimensions required by the model. Its exact parameters depend on the policy implementation.

The processed image is mapped into a visual representation by an encoder E_ϕ :

$$F_t^c = E_\phi(\tilde{I}_t^c). \quad (2.14)$$

Depending on the encoder architecture, F_t^c may be represented as a spatial feature map or as a sequence of visual tokens. The purpose of the encoder is therefore not merely to compress the raw pixels, but to retain information that is relevant to downstream robotic tasks. Voltron demonstrates that the representation objective affects the information preserved by the encoder: language-conditioned visual reconstruction is used to learn low-level visual patterns, whereas visually grounded language generation encourages the representation of higher-level semantics [28]. For manipulation tasks requiring accurate localization, it remains important to preserve local information such as object boundaries, contours, and relative spatial arrangements.

A single-view system uses only one camera observation at each time step:

$$O_t^{\text{vision}} = I_t^c. \quad (2.15)$$

Although this setting requires the policy to process only one image stream, a single viewpoint provides limited spatial information and can lose task-relevant regions because of occlusion by objects, the robot arm, or the end effector. SCDP identifies two central requirements for precise manipulation from a single global RGB view: preserving fine-grained visual cues and identifying task-relevant image regions. The method therefore produces a hierarchy of coarse-to-fine feature maps:

$$\mathcal{F}_t^c = \{F_{t,1}^c, F_{t,2}^c, \dots, F_{t,L}^c\}, \quad (2.16)$$

where higher-resolution features preserve fine local details, while lower-resolution features provide broader contextual information [29]. These results indicate that a single RGB view can support robotic manipulation, provided that the representation maintains an appropriate balance between local detail and global scene context.

In the camera configuration used in this work, a front camera and an overhead camera observe the same manipulation area from different directions. The front image records the lateral appearance of the robot arm, gripper, wire harness, and plug, and provides visual cues related to gripper motion and changes in local occlusion. The overhead image provides an overall two-dimensional view of the spatial distribution of the wire harness, plug, and robot arm within the workspace. Since both cameras provide RGB observations, these images are not treated as direct measurements of the complete three-dimensional configuration of the wire harness. Instead, they provide complementary two-dimensional visual information.

For a system with M cameras, the multi-view observation can be expressed as

$$\mathcal{I}_t = \{I_t^1, I_t^2, \dots, I_t^M\}. \quad (2.17)$$

Complementary viewpoints can reduce the occlusion and spatial ambiguity present in a single image. Nevertheless, providing multiple images to a model does not by itself guarantee that the model understands their spatial relationships. XVR organizes cross-view reasoning into correspondence, verification, and localization. These categories respectively address the identification of the same physical entities across views, the consistency of multi-view observations, and the spatial relationships between camera viewpoints [30].

From an architectural perspective, multi-view information can be combined at the input, feature, or decision level. Input-level, or early, fusion combines observations before high-level visual features are extracted. TVVE provides an example of geometric input-level aggregation. It converts RGB-D observations from multiple fixed cameras into point clouds, aggregates them into a global point cloud in the world coordinate system, and subsequently renders task-relevant virtual views [31]. This approach requires depth observations and camera geometry and therefore differs from the RGB-only setup used in the present work.

Feature-level fusion first encodes each camera image independently:

$$F_t^c = E_\phi^c(\tilde{I}_t^c), \quad c \in \{1, \dots, M\}, \quad (2.18)$$

and subsequently combines the resulting representations:

$$F_t^{\text{multi}} = \mathcal{G}(F_t^1, F_t^2, \dots, F_t^M), \quad (2.19)$$

where $\mathcal{G}(\cdot)$ denotes a feature-fusion mechanism. Dual-GAT employs multi-view feature fusion to improve the representations of occluded objects before performing object-level and relational-level graph reasoning [32]. XVR further shows that explicit supervision of cross-view relations can encourage a model to establish correspondences between visual elements observed from different viewpoints [30].

Decision-level, or late, fusion processes the individual views separately and combines their features or predictions only before the final policy output. Compared with input-level fusion, this design preserves a higher degree of independence between view-specific processing branches. Compared with feature-level fusion, interactions between the views occur at a later stage. The reviewed works primarily focus on geometric input aggregation, feature-level fusion, or explicit cross-view relation learning rather than on a purely decision-level fusion pipeline.

In this work, the visual observation consists of a front RGB image and an overhead RGB image. These observations are jointly provided to the ACT policy together with the current

robot joint state. The complete observation at time step t is defined as

$$o_t = (I_t^{\text{front}}, I_t^{\text{top}}, q_t), \quad (2.20)$$

where $q_t \in \mathbb{R}^{d_q}$ contains the current robot joint positions and the gripper state. The two camera observations are processed separately by the shared visual encoder:

$$\begin{aligned} F_t^{\text{front}} &= E_\phi(\mathcal{P}(I_t^{\text{front}})), \\ F_t^{\text{top}} &= E_\phi(\mathcal{P}(I_t^{\text{top}})). \end{aligned} \quad (2.21)$$

Conditioned on the two visual representations and the robot state, the ACT policy predicts an action chunk with horizon H :

$$\hat{A}_t = \pi_\theta(F_t^{\text{front}}, F_t^{\text{top}}, q_t) = [\hat{a}_t, \hat{a}_{t+1}, \dots, \hat{a}_{t+H-1}]. \quad (2.22)$$

This joint conditioning indicates that the two camera representations and the joint state contribute together to action prediction. It does not assume that the policy explicitly reconstructs the complete three-dimensional shape of the wire harness from the RGB images. The front and overhead images provide complementary scene observations, while the joint state provides information about the robot configuration that cannot be obtained reliably from external RGB images alone. Their combination allows the policy to predict subsequent actions according to both the current visual environment and the current robot state.

2.4. Transformer in Robot Learning

Transformer was originally introduced for sequence modelling and is commonly organized as an encoder–decoder architecture. Each Transformer block normally contains multi-head attention, a feed-forward network, residual connections, and layer normalization. The self-attention mechanism first maps input features into query, key, and value matrices and computes relationships among different input elements:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (2.23)$$

where Q , K , and V denote the query, key, and value matrices, respectively, and d_k is the dimension of the key vectors. The resulting attention weights determine the contribution of each input element to the updated representation. Multi-head attention projects the input into different representation spaces and allows the model to learn several types of relationships simultaneously. Since self-attention alone does not represent input order, positional encodings are added to preserve sequence positions or the spatial locations of visual tokens. Compared with convolutional networks that primarily extract local characteristics,

Transformer models can directly establish relationships between distant input elements. In contrast to recurrent networks that update hidden states sequentially, multiple input positions in a Transformer can be processed in parallel [33].

For visual tasks, an image can be divided into image patches, with each patch represented as a visual token. Self-attention then models relationships among these visual tokens. Video data additionally contain an ordered sequence of frames, allowing Transformer models to process spatial and temporal information jointly [33]. In robot learning, visual features, proprioceptive states, task instructions, and actions can also be converted into tokens or feature sequences. However, different robot policies organize these tokens in different ways. Some models use Transformer layers to process several sensory inputs from the same time step, whereas other methods explicitly predict actions over multiple future time steps.

Robot demonstrations in behaviour cloning can be represented as sequences of state–action pairs:

$$\tau_i = \{(s_1, a_1), (s_2, a_2), \dots, (s_T, a_T)\}. \quad (2.24)$$

Liu et al. state that actions in human demonstrations may depend not only on the current observed state, but also on latent planning strategies and preferences that persist across multiple time steps. To represent these temporal dependencies, action-chunking methods model the joint distribution of future actions conditioned on previous states:

$$\pi(a_{t:t+\ell-1} \mid s_{t-c+1:t}), \quad (2.25)$$

where c is the number of states in the context and ℓ is the number of predicted future actions. During deployment, the policy may execute E actions from the predicted sequence before obtaining a new observation and replanning. This value is defined as the execution horizon. Increasing the action-chunk length provides access to more previously generated actions and can improve the representation of multi-step temporal dependencies in demonstrations, including idle pauses and different latent strategies [34].

Action chunking also introduces a trade-off between temporal consistency and reactivity. A longer action horizon can preserve consistency within a predicted chunk, but the robot cannot use new observations to correct the actions during this execution period. When the environment is stochastic or the actual state differs from the expected state, a longer action horizon may reduce the ability of the policy to react to recent information. A shorter action horizon provides more frequent observation updates, but independently generated action chunks may fail to preserve the temporal dependencies in the demonstrations. The cited analysis therefore concludes that there is no universally optimal action horizon. Its selection depends on the temporal dependencies in the demonstrations, environmental stochasticity, and model-prediction errors [34].

Action Chunking with Transformers, or ACT, combines a conditional variational autoencoder

with a Transformer to predict future action chunks from multimodal inputs. In the ACT policy used by George and Farimani, a latent variable is computed from the demonstrated action sequence and robot state during training, and a Transformer is used to predict an action sequence. During inference, the policy returns an action chunk beginning at the current time step. Since chunks predicted at consecutive time steps overlap, one physical execution step can receive several candidate predictions generated at different earlier time steps. Temporal ensembling combines these overlapping predictions through a weighted average to obtain the executed action [35].

George and Farimani further observe that earlier predictions may become unsuitable when the actual environment state changes. They therefore use the standard deviation of overlapping action predictions to measure their level of agreement. When predictions are consistent, a larger contribution from previous predictions is retained. When disagreement increases, older predictions receive lower weights. If the disagreement exceeds a predefined threshold, temporal ensembling is temporarily suspended and a segment of the current action chunk is executed directly. This method shows that an action chunk can represent relationships among consecutive actions, while the fusion of different chunks must also account for whether earlier predictions remain consistent with the current state [35].

Another role of Transformer models in robot learning is to establish relationships among different sensory inputs. Kim et al. jointly input cropped visual features, gaze coordinates, and the kinematic states of the left and right manipulators into a Transformer encoder. Each scalar robot-state element is represented separately, and a one-hot positional embedding is used to distinguish different state variables. The image embedding and state representations are concatenated and processed by a three-layer Transformer encoder. The encoded features are subsequently used to predict actions for both robot arms. Self-attention dynamically changes the importance assigned to the image, gaze position, left-arm state, and right-arm state, thereby reducing distractions caused by irrelevant variables in the high-dimensional dual-arm state [36].

The input sequence used by Kim et al. consists of visual and kinematic variables from the same time step, and the policy predicts the next action. The model therefore demonstrates that Transformer layers can jointly process visual information and robot kinematic states. However, its Transformer does not directly receive a history of multiple time steps and does not predict a complete future action chunk. This work primarily supports the use of self-attention for sensory fusion and relevant-feature selection rather than long-term temporal modelling [36].

Any2Policy further applies Transformer models to the unified processing of different task instructions and environmental observations. The model accepts task instructions in the form of text, audio, goal images, and demonstration videos, while its observations can include images, videos, and point clouds. Each modality is first converted into a common token representation using pretrained encoders and learnable projection layers. Self-attention

processes tokens within the instruction or observation stream, while cross-attention establishes relationships between task instructions and environmental observations. Its cross-attention operation is represented as

$$h_i = \text{softmax} \left(\frac{Q_i K_i^T}{\sqrt{d_h}} \right) V_i, \quad (2.26)$$

where learnable queries are used to obtain representations relevant to the current instruction and observation. Each Transformer block in the policy network contains self-attention, cross-attention, and a feed-forward network, and the fused representations are used to generate continuous robot actions [37].

Images, videos, and point clouds produce relatively large numbers of visual tokens, while the computational requirements of standard self-attention increase with the token count. Any2Policy therefore applies TokenLearner to remove redundant visual tokens and uses modality tokens and positional embeddings to preserve modality identities and token order. This architecture demonstrates that Transformer models can provide a unified fusion structure for heterogeneous robot inputs. At the same time, multimodal inputs increase computational costs and may require token-compression mechanisms [37]. Any2Policy primarily generates continuous actions from current instructions and observations, and therefore mainly supports multimodal fusion rather than future action-chunk prediction.

DreamVLA encodes language instructions, visual observations, and robot proprioceptive states as tokens and uses learnable dream queries to predict a future environment representation. Its future world embedding is defined as

$$w_{t+n} = \mathcal{M}(l, o_t, s_t \mid \langle \text{dream} \rangle), \quad (2.27)$$

where l denotes the language instruction, o_t is the current visual observation, and s_t is the robot state. Separate queries represent future dynamic regions, monocular depth, and high-level semantic information:

$$\hat{p}_{t+n} = \mathcal{P}(w_{t+n}) = [\hat{f}_{t+n}, \hat{d}_{t+n}, \hat{c}_{t+n}]. \quad (2.28)$$

DreamVLA uses block-wise structured attention to restrict direct attention connections among dynamic, depth, and semantic queries. Each query type can attend to the shared language, visual, and state tokens, while direct connections to other query types are disabled to reduce interference among the prediction objectives [38].

For action generation, DreamVLA uses a separate action query to obtain an action-related representation and employs a denoising diffusion Transformer to generate an n -step action sequence:

$$\hat{a}_{t:t+n-1} = \mathcal{D}(\mathcal{M}(l, o_t, s_t, \langle \text{dream} \rangle \mid \langle \text{action} \rangle)). \quad (2.29)$$

The model therefore uses a predicted future environment representation as an intermediate

representation for action prediction and generates multiple future actions conditioned on the current language, visual observation, and robot state [38].

The cited studies demonstrate three main roles of Transformer models in robot learning. Self-attention can establish relationships between visual features and robot-state variables, cross-attention can fuse task instructions with environmental observations, and action-chunking or diffusion Transformers can predict multi-step future action sequences. The meaning of a “sequence” differs among these methods. The dual-arm imitation-learning model and Any2Policy mainly process multiple inputs under the current condition, whereas ACT, the action-chunking analysis, and DreamVLA explicitly involve future action sequences. Action chunks can represent dependencies across multiple time steps, but a longer execution horizon also reduces the opportunity to update actions using recent observations. Transformer models therefore provide a structure for continuous robot-action modelling, while the action-prediction length, executed horizon, and closed-loop update mechanism must still be selected according to the specific policy and task.

2.5. Conditional Variational Autoencoder

Human demonstrations are not fully deterministic. Even under similar visual observations and robot configurations, a human operator may perform a manipulation task using slightly different approach trajectories, motion timings, or gripper-closing instants. Consequently, the same observation may correspond to multiple valid future action sequences. A deterministic behavior-cloning policy must map these demonstrations to a single output and may therefore average between different action modes. Conditional generative models address this problem by learning a conditional distribution over possible outputs instead of predicting only one fixed result [39], [40].

For a Conditional Variational Autoencoder (CVAE), let x denote the target data and let c denote the conditioning information. The encoder approximates the conditional posterior distribution as

$$q_\phi(z \mid x, c) = \mathcal{N}(\mu_\phi, \text{diag}(\sigma_\phi^2)), \quad (2.30)$$

A latent sample is obtained using the reparameterisation trick:

$$z = \mu_\phi + \sigma_\phi \odot \xi, \quad \xi \sim \mathcal{N}(0, I), \quad (2.31)$$

The general CVAE objective combines reconstruction and latent-space regularisation:

$$\mathcal{L}_{\text{CVAE}} = \mathcal{L}_{\text{rec}} + \beta_{\text{KL}} D_{\text{KL}}(q_\phi(z \mid x, c) \parallel \mathcal{N}(0, I)), \quad (2.32)$$

The first term measures reconstruction accuracy, while the KL-divergence term regularises the approximate posterior towards the standard Gaussian prior. The ACT-specific application of this formulation is introduced in Section 3.2.

3. State of the Art

3.1. Overview of Robot Learning Policies

A robot policy determines which action should be selected from the information available to the robot. Argall et al. formalize a policy as a mapping from the observed state space $\mathcal{Z}$ to the action space $\mathcal{A}$:

$$\pi : \mathcal{Z} \rightarrow \mathcal{A}. \quad (3.1)$$

The observed state does not necessarily contain the complete physical state of the robot and its environment. Instead, it contains the information that can be obtained through the available sensors. The policy uses this observation to select a low-level motion command, an action primitive, or a higher-level behavior, depending on the formulation of the task [41]. For the robotic system considered in this thesis, the observation at time step t is defined as

$$\mathbf{o}_t = \{ \mathbf{I}_t^{\text{top}}, \mathbf{I}_t^{\text{front}}, \mathbf{s}_t \}, \quad (3.2)$$

where $\mathbf{I}_t^{\text{top}}$ denotes the RGB image captured by the overhead camera, $\mathbf{I}_t^{\text{front}}$ denotes the RGB image captured by the camera mounted near the gripper, and $\mathbf{s}_t$ denotes the current motor-state vector of the SO-101 robot. The overhead camera observes the spatial distribution of the wire harness over the workspace, whereas the front camera mounted near the gripper provides a local observation of the gripper and the grasping region.

The action is represented as

$$\mathbf{a}_t \in \mathbb{R}^6,$$

where the six components correspond to the target positions of the six SO101 motors, including the gripper motor. The learned policy predicts the desired motor targets, while their physical execution is handled by the lower-level motor-control system. The learned policy and the low-level feedback controller therefore perform different functions within the overall control pipeline.

Learning from Demonstration (LfD) derives a robot policy from examples provided by a teacher. A demonstration can be represented as a sequence of observation–action pairs,

$$d_j = \{ (\mathbf{z}_i^j, \mathbf{a}_i^j) \}_{i=0}^{T_j-1}, \quad \mathcal{D} = \{ d_j \}_{j=1}^N, \quad (3.3)$$

where d_j is one demonstration, T_j is its number of time steps, and $\mathcal{D}$ is the complete demonstration dataset. The learner uses this dataset to derive a policy that reproduces the demonstrated relationship between observations and actions [41].

Argall et al. distinguish three principal approaches to policy derivation. In the *mapping-function* approach, the demonstration data are used to directly approximate a function

$$f : \mathcal{Z} \rightarrow \mathcal{A}. \quad (3.4)$$

In the *system-model* approach, the demonstrations are used to estimate a state-transition model

$$T(s' \mid s, a), \quad (3.5)$$

and, in some formulations, a reward function $R(s)$, from which a policy is subsequently derived. In the *planning* approach, the demonstrations are used to learn action preconditions and postconditions, which are then used to construct a sequence of actions [41]. The end-to-end visuomotor policy used in this thesis is most closely related to the mapping-function category because it directly learns a mapping from visual and proprioceptive observations to robot actions.

The demonstrations in this work are collected through leader-follower teleoperation. In the terminology of Argall et al., teleoperation is a demonstration method in which the teacher controls the robot learner while the robot records the execution using its own sensors. Because the recorded observations and actions already belong to the learner's sensing and action spaces, this setup reduces the correspondence problems that arise when a human and a robot execute the task using different embodiments [41].

A policy learned from a fixed demonstration dataset remains dependent on the states and actions represented in that dataset. Policy improvement can therefore involve collecting additional demonstrations or incorporating feedback on learner executions. For example, Advice-Operator Policy Improvement generates additional training data by applying human-selected correction operators to selected parts of a robot execution. This demonstrates that continuous motion policies can be improved using targeted corrections rather than complete new demonstrations [42]. The present study, however, uses an offline setting: the dataset is recorded before training, and no online advice, reward optimization, or autonomous exploration is used during policy learning.

A single-step policy predicts only the next action:

$$\hat{\mathbf{a}}_t = \pi_\theta(\mathbf{o}_t). \quad (3.6)$$

When such a policy is used for a long manipulation task, errors made at one time step may change the subsequent observations and move the robot away from the distribution represented by the demonstrations. The ACT study identifies this accumulation of prediction errors as an important difficulty in fine-grained imitation learning [6].

Sequence-based policies instead generate several future actions together:

$$\hat{\mathbf{A}}_t = [\hat{\mathbf{a}}_t, \hat{\mathbf{a}}_{t+1}, \dots, \hat{\mathbf{a}}_{t+H-1}] = \pi_\theta(\mathbf{o}_{\leq t}), \quad (3.7)$$

where H denotes the predicted action horizon. Such a formulation allows a policy to represent temporal relationships between successive robot commands instead of treating every command as an independent prediction.

Different methods generate these action sequences in different ways. ACT predicts target joint positions for the next H time steps and thereby reduces the effective decision horizon of the task. The method was introduced to mitigate compounding errors and to represent temporally correlated behavior in human demonstrations [6]. Diffusion-based imitation-learning methods instead generate action sequences through an iterative denoising process. Movement Primitive Diffusion further combines diffusion-based imitation learning with Probabilistic Dynamic Movement Primitives, which decode the model output into smooth action sequences with specified initial position and velocity conditions [43]. MPD is not implemented in this thesis, but it illustrates an alternative approach to continuous and sequence-based robot motion generation.

Multiple cameras can provide complementary visual information about a robotic manipulation task. Seo et al. distinguish between simply using multi-view images as policy inputs and explicitly learning cross-view representations. Their Multi-View Masked World Model masks complete viewpoints and reconstructs them from the remaining viewpoints and neighboring video frames. In their experiments, the front camera provides a broad observation of the robot workspace, whereas the wrist camera provides a closer view of target objects [44].

The present work does not implement the masked autoencoder or world model proposed by Seo et al. Instead, the overhead and front-camera images are supplied directly to the visuomotor policy. Nevertheless, the two views serve a related complementary role: the overhead image describes the global workspace configuration, while the front-camera image retains local visual information as the gripper approaches, grasps, and lifts the wire harness.

The task investigated in this thesis consists of locating a randomly positioned wire harness, approaching a suitable grasping region, closing the gripper, transporting the object toward the central task region, and lifting it from the table. These stages form a temporally connected motion process rather than a collection of independent single-step actions.

ACT is selected as the primary policy because it learns directly from teleoperated demonstrations and predicts a sequence of continuous joint-position targets instead of only one

action at a time. The original ACT system maps RGB camera observations and robot joint positions to action chunks and was designed for precise, visually guided, contact-rich manipulation [6]. In this thesis, the same action-sequence formulation is instantiated for the single-arm SO101 action space and the two available camera views.

This selection does not imply that ACT is universally superior to other policy classes. Rather, ACT provides a defined imitation-learning framework that is compatible with the available leader-follower demonstrations, continuous motor actions, and multi-view visual observations. Its detailed architecture and training objective are introduced in Section 3.2, while the experimental effects of the action-chunk and optimization parameters are examined in the implementation and results chapters.

3.2. Action Chunking with Transformers

Action Chunking with Transformers, abbreviated as ACT [6], is an end-to-end imitation learning policy for robotic manipulation. It was originally proposed in the ALOHA system for learning fine-grained, contact-rich, and visually guided bimanual manipulation tasks. The central idea of ACT is to predict a sequence of future actions instead of predicting only the next single action. In this way, a long-horizon control problem is decomposed into a set of short action chunks. This design reduces the effective decision horizon and helps mitigate the compounding error problem commonly observed in standard behavior cloning.

In conventional behavior cloning, the policy is usually formulated as:

$$\pi_{\theta}(\mathbf{a}_t \mid \mathbf{o}_t) \quad (3.8)$$

where $\mathbf{o}_t$ denotes the observation at timestep t , and $\mathbf{a}_t$ denotes the action to be executed at the current timestep. This single-step prediction formulation can work for simple tasks, but it is often unstable in fine manipulation tasks. A small prediction error at one timestep may move the robot into a state that is rarely observed in the demonstration dataset. The following predictions are then made under out-of-distribution observations, causing errors to accumulate over time and eventually leading to task failure.

ACT reformulates the policy as:

$$\pi_{\theta}(\mathbf{a}_{t:t+H-1} \mid \mathbf{o}_t) \quad (3.9)$$

where $\mathbf{a}_{t:t+H-1}$ contains exactly H future actions, and H is the chunk size. Instead of outputting a single action, the policy outputs a continuous action segment. If the original task horizon is T , action chunking approximately reduces the effective decision horizon to T/H . Therefore, ACT can reduce long-horizon error propagation and better represent short-term action structures in human demonstrations. For example, in a robotic wire harness manipulation task, an action chunk may correspond to a continuous motion such as approaching the wire, closing the gripper, and lifting the wire, rather than treating each control step as

an isolated prediction.

The observation input of ACT usually consists of multi-view RGB images and robot proprioceptive states. The current observation can be written as:

$$\mathbf{o}_t = \{\mathbf{I}_t^1, \mathbf{I}_t^2, \dots, \mathbf{I}_t^M, \mathbf{s}_t\} \quad (3.10)$$

where $\mathbf{I}_t^c$ denotes the RGB image from the c -th camera at timestep t , M is the number of cameras, and $\mathbf{s}_t$ denotes the proprioceptive robot state. The action is represented as the target-position vector:

$$\mathbf{a}_t \in \mathbb{R}^{d_a} \quad (3.11)$$

where d_a is the action dimension. In the original ALOHA bimanual system, each arm has 6 joints and 1 gripper degree of freedom, resulting in $d_a = 14$ for two arms. In other robotic systems, d_a can be adjusted according to the actual robot action space. Let H denote the number of actions contained in one action chunk. The demonstrated action chunk beginning at timestep t is defined as

$$\mathbf{A}_t := \mathbf{a}_{t:t+H-1} = \begin{bmatrix} \mathbf{a}_t^\top \\ \mathbf{a}_{t+1}^\top \\ \vdots \\ \mathbf{a}_{t+H-1}^\top \end{bmatrix} \in \mathbb{R}^{H \times d_a}, \quad (3.12)$$

For the SO-101 system used in this thesis, $d_a = 6$. ACT instantiates the general CVAE formulation introduced in Section 2.5. During training, the encoder receives the demonstrated action chunk and the proprioceptive state $\mathbf{s}_t$, and estimates

$$q_\phi(z \mid \mathbf{A}_t, \mathbf{s}_t) = \mathcal{N}(\mu_\phi, \text{diag}(\sigma_\phi^2)), \quad (3.13)$$

The ACT decoder receives the complete observation and the latent variable:

$$\hat{\mathbf{A}}_t = \pi_\theta(\mathbf{o}_t, z), \quad (3.14)$$

The visual observations are encoded into visual features and fused with the robot state and latent representation by the Transformer-based policy. The decoder produces a predicted action tensor with dimensions $H \times d_a$.

The ACT training objective is

$$\mathcal{L}_{\text{ACT}} = \mathcal{L}_{\text{rec}} + \beta_{\text{KL}} \mathcal{L}_{\text{KL}}, \quad (3.15)$$

The reconstruction loss is

$$\mathcal{L}_{\text{rec}} = \left\| \hat{\mathbf{A}}_t - \mathbf{A}_t \right\|_1, \quad (3.16)$$

The latent-space regularisation term is

$$\mathcal{L}_{\text{KL}} = D_{\text{KL}}(q_{\phi}(z \mid \mathbf{A}_t, \mathbf{s}_t) \parallel \mathcal{N}(0, I)), \quad (3.17)$$

During inference, the demonstrated future action chunk is unavailable and the CVAE encoder is discarded. The latent variable is fixed to the mean of the standard Gaussian prior:

$$z = \mathbf{0},$$

The ACT training procedure can be summarized as follows: During inference, executing one action chunk every H timesteps may cause discontinuous or jerky motion, because new observations are only incorporated at chunk boundaries. To address this issue, ACT uses temporal ensembling. The policy is queried at every timestep, and each query predicts the next H actions. Therefore, the same execution timestep may receive multiple action predictions generated at different previous timesteps. ACT combines these predictions through an exponentially weighted average:

$$\mathbf{a}_t^{\text{exec}} = \frac{\sum_i w_i \hat{\mathbf{a}}_t^{(i)}}{\sum_i w_i} \quad (3.18)$$

where:

$$w_i = \exp(-\lambda_{\text{ens}} i) \quad (3.19)$$

Here, $\hat{\mathbf{a}}_t^{(i)}$ denotes the i -th candidate prediction for execution timestep t , and λ_{ens} controls the decay rate of the ensemble weights. This is different from ordinary temporal smoothing, because ACT averages multiple predictions for the same timestep, rather than averaging actions from neighboring timesteps. Therefore, it can improve motion smoothness without introducing a strong temporal bias. The inference procedure of ACT can be summarized as follows: ACT has several advantages for fine-grained robotic manipulation. First, action chunking reduces the effective horizon of the task and makes the policy less sensitive to long-term error accumulation. Second, the Transformer architecture is suitable for process-

Algorithm 1 Training Procedure of the ACT Policy**Require:** Demonstration dataset $\mathcal{D}$, chunk size H , KL weight β_{KL} **Ensure:** Trained ACT policy decoder π_θ

Initialize the CVAE encoder

$$q_\phi(z \mid \mathbf{a}_{t:t+H-1}, \mathbf{s}_t)$$

Initialize the ACT policy decoder

$$\pi_\theta(\hat{\mathbf{a}}_{t:t+H-1} \mid \mathbf{o}_t, z)$$

while the training criterion is not satisfied **do**Sample a current observation $\mathbf{o}_t$ and a future action sequence $\mathbf{a}_{t:t+H-1}$ from $\mathcal{D}$ Extract the proprioceptive state $\mathbf{s}_t$ from $\mathbf{o}_t$

Estimate the latent distribution using the CVAE encoder:

$$q_\phi(z \mid \mathbf{a}_{t:t+H-1}, \mathbf{s}_t)$$

Sample the latent variable z from

$$q_\phi(z \mid \mathbf{a}_{t:t+H-1}, \mathbf{s}_t)$$

Predict the future action sequence using the ACT decoder:

$$\hat{\mathbf{a}}_{t:t+H-1} = \pi_\theta(\mathbf{o}_t, z)$$

Compute the reconstruction loss:

$$\mathcal{L}_{\text{rec}} = \|\hat{\mathbf{a}}_{t:t+H-1} - \mathbf{a}_{t:t+H-1}\|_1$$

Compute the KL regularization term:

$$\mathcal{L}_{\text{KL}} = D_{\text{KL}}(q_\phi(z \mid \mathbf{a}_{t:t+H-1}, \mathbf{s}_t) \parallel \mathcal{N}(0, I))$$

Compute the total training loss:

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \beta_{\text{KL}} \mathcal{L}_{\text{KL}}$$

Update θ and ϕ using gradient descent or the Adam optimizer**end while****return** Trained ACT policy decoder π_θ

ing multi-view visual tokens and action-sequence tokens, allowing the model to capture both spatial information and temporal dependencies. Third, the CVAE formulation helps model the stochastic and multi-modal nature of human demonstrations, instead of forcing the model to average different valid trajectories. Fourth, temporal ensembling improves action smoothness during inference without adding extra training cost. Fifth, ACT directly predicts continuous actions, making it more suitable for manipulation tasks that require millimeter-level accuracy.

However, ACT also has limitations. It still depends strongly on the quality and coverage of the demonstration dataset. If recovery states are not included in the demonstrations, the policy may still fail after drifting away from the demonstration distribution. ACT also does not explicitly model object dynamics. Therefore, for deformable objects, transparent objects, small objects, or heavily occluded scenes, the policy performance depends heavily on the quality of visual observations. When the target object has low contrast or occupies only a small region in the image, ACT alone may not solve the perception problem. In such cases, better camera placement, object detection, keypoint estimation, or segmentation may be required.

In this research, ACT can be used to learn the robotic grasping and manipulation process

Algorithm 2 ACT Inference with Temporal Ensemble

Require: Trained ACT policy decoder π_θ , episode length T , chunk size H , temporal-ensemble decay λ_{ens}

Ensure: Smooth closed-loop robot action sequence

Initialize action buffers $B[1 : T]$ as empty lists

Set latent variable $z \leftarrow 0$

for $t = 1$ to T **do**

 Obtain the current observation o_t

 Predict a future action chunk using the ACT decoder

$\hat{\mathbf{a}}_{t:t+H-1} = \pi_\theta(\mathbf{o}_t, z = 0)$

for $j = 0$ to $H - 1$ **do**

if $t + j \leq T$ **then**

 Store the predicted action $\hat{a}_{t+j}$ in buffer $B[t + j]$

end if

end for

 Retrieve candidate predictions $\mathcal{P}_t \leftarrow B[t]$

 Compute $w_i \leftarrow \exp(-\lambda_{\text{ens}} i)$ for $i = 0, \dots, |\mathcal{P}_t| - 1$

 Normalize the weights $\tilde{w}_i \leftarrow w_i / \sum_j w_j$

 Compute $\mathbf{a}_t^{\text{exec}} \leftarrow \sum_i \tilde{w}_i \mathcal{P}_t[i]$

 Send $\mathbf{a}_t^{\text{exec}}$ to the low-level robot controller

end for

return Smooth closed-loop robot action sequence

of wire harnesses from teleoperation demonstrations. For wire harness manipulation, the robot needs to identify the wire position from visual observations, approach the target region, close the gripper, lift the wire, and adjust its pose. This process cannot be represented well by a single isolated action. Instead, it consists of a continuous sequence of short motion segments. Therefore, the action chunking mechanism of ACT is well matched to this task.

Moreover, since wire harnesses are deformable and difficult to model with precise analytical dynamics, ACT provides a practical learning-based alternative by directly learning the mapping from visual observations and robot states to future action sequences.

3.3. Robotic Foundation Models

Foundation models are generally defined as models that are pretrained at scale on broad and diverse datasets and can subsequently be adapted to a wide range of downstream tasks. This adaptation may be achieved through direct reuse, in-context conditioning, or task-specific fine-tuning [45], [46]. In robotics, three related but distinct concepts should be separated. A *foundation model used in robotics* is a pretrained model integrated into a robotic system as a reusable component for perception, reasoning, planning, or control. Such a model does not necessarily contain robot actions in its pretraining data. A *robotic*

foundation model, in contrast, incorporates robot-relevant observations, states, actions, or embodiment information and is intended to support multiple robotic tasks, environments, or platforms. A *vision-language-action model* describes a model interface in which visual observations and language instructions, often together with proprioceptive states, are mapped to robot actions.

These definitions describe different properties of a model. A vision-language-action architecture may qualify as a robotic foundation model when it is pretrained on broad and heterogeneous data and can be adapted to multiple downstream tasks. However, the vision-language-action input–output structure alone is not sufficient to establish this status. Similarly, the use of a Transformer does not automatically make a robot policy a foundation model. The decisive criteria are the scope and diversity of the pretraining data, the reuse of the pretrained model, and its ability to adapt to multiple downstream problems rather than its parameter count or network architecture alone [45], [46].

Traditional task-specific policies and robotic foundation models therefore differ primarily in their training scope and intended generalization. A task-specific policy is commonly trained using demonstrations collected for one robot and one defined task. Its observation and action spaces are fixed by the experimental system, and its generalization objective is usually limited to variations close to the training distribution. Robotic foundation models instead attempt to share representations and physical skills across tasks, objects, environments, and, in some cases, robot embodiments.

The development of robotic foundation models depends strongly on the availability and organization of training data. Relevant sources include demonstrations from multiple tasks, observations collected in different environments, data from different robot embodiments, and visual, linguistic, or video data obtained outside the target robotic platform. Internet-scale visual and linguistic pretraining can provide semantic knowledge, while robot trajectories are required to connect this knowledge to physical actions. Because different robots use different sensors, state definitions, control frequencies, and action spaces, heterogeneous datasets must be converted into compatible formats before joint training. This requires consistent representations of observations, robot states, camera inputs, and actions.

A common training paradigm is therefore to first pretrain a model on a broad mixture of data and then adapt it using a smaller amount of target-task data. Pretraining is intended to learn reusable visual, linguistic, temporal, and action representations, whereas downstream fine-tuning specializes these representations for a particular robot and task. The π_0 model follows this paradigm by combining a pretrained vision-language model with a flow-matching action expert and training the resulting policy using diverse data from multiple robot platforms. It predicts continuous action chunks conditioned on images, language instructions, and robot states [47]. SmoIVLA adopts a related vision-language-action formulation but focuses on reducing the training and inference requirements. It uses community-collected data from affordable robotic platforms and is designed for training on a single GPU and

deployment on consumer-grade GPUs or CPUs [48]. A systematic study of generalist VLA policies shows that performance depends strongly on the VLM backbone, action formulation, history integration, and training-data strategy. Continuous-action policy heads using historical observations and target-domain post-training were found to be particularly effective [49].

Such pretraining can provide several capabilities that are difficult to obtain from isolated task datasets. These include language-conditioned control, knowledge sharing between tasks, adaptation to changes in objects and environments, fine-tuning with a smaller target dataset, and the transfer of visual-semantic knowledge to robot control. Nevertheless, these capabilities do not imply unrestricted physical generalization. The physical skills of a policy remain dependent on the robot data used during training, and semantic knowledge does not automatically provide accurate geometric, contact, or dynamic reasoning.

Robotic foundation models also introduce substantial practical limitations. Real robot trajectories are expensive and time-consuming to collect, while data obtained from different embodiments may use incompatible observation and action spaces. Precise contact reasoning, force-sensitive manipulation, and fine geometric control remain challenging. Large models can also introduce inference and communication latency that conflicts with real-time closed-loop control. Furthermore, reliable safety guarantees and calibrated uncertainty estimates are still limited. Comparisons between published models are difficult because studies frequently use different robots, tasks, datasets, control interfaces, and evaluation metrics [45], [46].

The relationship between ACT, SmoVLA, and π_0 illustrates the distinction between a task-specific imitation policy and generalist vision-language-action models. All three methods predict temporally extended action sequences rather than treating every control step as an independent prediction. However, they differ in their training scope, action-generation mechanism, and computational requirements. The comparison shows as the table 3.1

Table 3.1.: Simplified comparison of ACT, SmoVLA, and π_0

Model	Training scope	Action generation	Computation
ACT	Target-task demonstrations	CVAE and Transformer action chunks	Lower
SmoVLA	Community pretraining and adaptation	VLM and flow-matching action chunks	Moderate
π_0	Large multi-task and multi-robot pretraining	VLM and flow-matching action expert	High

ACT learns a generative model of action sequences from demonstrations by combining a conditional variational autoencoder with a Transformer-based action-chunking policy [6]. It does not rely on broad vision-language or multi-robot pretraining. Consequently, ACT should be described in this thesis as a task-specific imitation-learning policy rather than a robotic foundation model. Its use of a Transformer and action chunking does not change

this classification.

This thesis investigates policy learning under limited-data, single-robot, and single-task conditions. The target system consists of one SO101 robot performing wire-harness grasping and lifting from multi-view RGB observations and robot states. The experimental objective is to study the effects of policy parameters and to measure performance under controlled real-world conditions. ACT is therefore selected because it can be trained directly on the collected demonstrations, has lower computational and deployment requirements than the two vision-language-action models, and permits controlled analysis of parameters such as chunk size, the number of executed actions, KL regularization, and learning rate.

The selection of ACT is not intended to imply that it provides the same generalization capabilities as SmolVLA or π_0 . Instead, it establishes a task-specific reference for examining what can be achieved without large-scale multimodal pretraining. Future work could investigate the adaptation of SmolVLA or π_0 to the SO101 platform and wire-harness manipulation. Such experiments could determine whether broad visual-language and robot-data pretraining improves generalization to new wire configurations, positions, or task instructions sufficiently to justify the additional data, computation, adaptation, and deployment complexity.

3.4. Benchmarks for Robot Policies

Robot policy benchmarks provide standardized tasks, observation spaces, action spaces, experimental conditions, and evaluation metrics for comparing different learning-based control methods. A meaningful benchmark should not only report whether a robot completes a task, but should also characterize the conditions under which the policy succeeds or fails. Depending on the research objective, robot policy benchmarks may evaluate task success, generalization, robustness, action prediction accuracy, execution efficiency, motion stability, or transfer across tasks and robot platforms.

Existing robot learning benchmarks can generally be divided into simulation-based benchmarks, offline imitation-learning benchmarks, and real-world robot benchmarks. Simulation environments such as RLBench provide a large collection of standardized manipulation tasks with reproducible initial conditions and multimodal observations [50]. Such benchmarks enable scalable comparison between algorithms, but they cannot fully reproduce physical effects such as joint backlash, actuator delay, calibration errors, object deformation, contact uncertainty, and grasp slippage. Multi-task benchmarks such as LIBERO focus more strongly on knowledge transfer, continual learning, and generalization across manipulation tasks [51]. Offline imitation-learning frameworks such as robomimic provide standardized datasets and evaluation procedures for comparing policies trained from demonstrations [13]. However, low offline prediction error does not necessarily imply high closed-loop task success because small prediction errors may accumulate during execution and shift the

robot into states that are not represented in the demonstration dataset.

The Action Chunking with Transformers policy is commonly evaluated through real-world task execution and task success rate [6]. Since ACT predicts a sequence of future actions instead of a single action at each time step, its performance can be affected by parameters such as the action chunk size, the number of executed actions, the learning rate, and the weight of the Kullback–Leibler regularization term. Evaluating these parameters requires both offline metrics and real-world execution results. Offline evaluation measures how closely the predicted action sequence matches the recorded demonstrations, whereas real-world evaluation determines whether the resulting closed-loop behavior completes the manipulation task reliably.

In this work, a task-specific real-world benchmark is established for evaluating ACT policies on wire-harness grasping using an SO-101 robotic arm. The proposed evaluation should not be interpreted as a general-purpose robot manipulation benchmark because it contains one robot platform, one primary object category, and one manipulation task. Instead, it is designed as a standardized evaluation protocol for comparing different ACT parameter configurations under identical hardware, perception, workspace, and task conditions.

3.4.1. Benchmark Task

The benchmark task consists of grasping a wire harness from a tabletop, transporting, and lifting it. Connector insertion is not included in the benchmark. The complete task can therefore be represented by the following sequence:

approach $\rightarrow$ align $\rightarrow$ close gripper $\rightarrow$ transport $\rightarrow$ lift.

The benchmark uses the policy observation defined in Equation (2.2) and the six-dimensional action defined in Equation (2.3). Accordingly, the policy receives the fixed top-camera image, the front-camera image, and the six-dimensional robot state containing five arm-joint positions and one gripper position. It predicts five target arm-joint positions and one target gripper position.

3.4.2. Benchmark Dimensions Used in This Study

The proposed benchmark evaluates the ACT policy along four main dimensions:

1. **Task effectiveness**, measured by the overall real-world success rate;
2. **Spatial robustness**, measured by cell-wise success rates and their distribution across the 6×3 workspace;
3. **Imitation fidelity**, measured by global action L1 error, gripper L1 error, and the mean action error within consecutive 1000-frame intervals;
4. **Execution quality**, measured by motion smoothness.

These dimensions are used to compare different combinations of ACT hyperparameters, including the action chunk size, number of executed action steps, KL_weight, and learning rate. All configurations are trained and evaluated using the same robot platform, camera arrangement, dataset format, workspace, and task definition. Consequently, differences in the results can be attributed more directly to the policy configuration rather than changes in the experimental environment.

In summary, the benchmark developed in this work is a task-specific real-world evaluation protocol rather than a general multi-task robot benchmark. Its purpose is to provide a reproducible and quantitatively measurable basis for selecting an ACT configuration for wire-harness grasping. The combination of real-world success rate, grid-based spatial evaluation, offline action prediction error, gripper error, and motion smoothness provides a more comprehensive assessment than any single metric alone.

4. Method

Figure 4.1 summarises the complete method. The task and controlled workspace define the data-collection procedure; the demonstrations are then used to train the ACT configurations. Offline replay and spatially stratified robot trials provide complementary evidence for comparing the configurations.

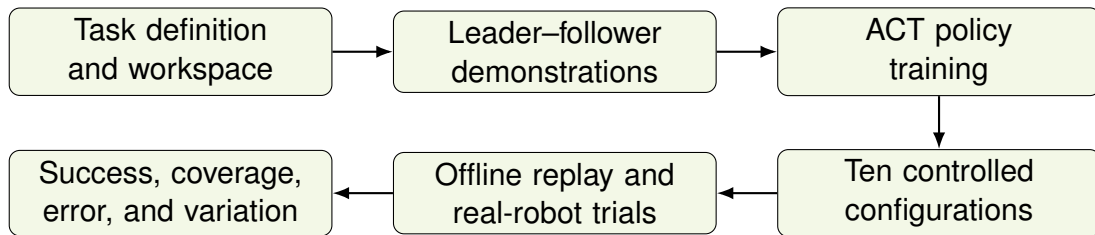


Figure 4.1.: Overview of the experimental method from task definition and demonstration collection to policy comparison.

4.1. Task Definition

The wire-harness manipulation problem is formulated as an end-to-end visuomotor imitation-learning task. A single SO-101 robot arm is required to approach, grasp, transfer, and lift one flexible wire harness based on two RGB camera observations and the current robot state.

Wire Harness Object

The manipulation object used in this study was a flexible multi-conductor wire harness, as shown in Figure 4.2. The harness consisted of a black multi-pin connector, multiple purple insulated conductors, a central section wrapped with black fabric tape, and a set of unconnected wire ends.

The overall harness length was approximately 30 cm, measured along the centreline from the connector end to the longest free wire end. Since the harness was flexible, its projected length and shape on the work surface varied with its initial placement.

The fabric-wrapped section kept the individual conductors together and provided a visually distinctive region for grasping. In contrast, the unwrapped wire ends could separate and deform independently. These characteristics made the task different from rigid-object grasping: the apparent object geometry, local orientation, and contact response could change between demonstrations and evaluation trials.



Figure 4.2.: Wire harness used as the manipulation object. The harness contains a multi-pin connector, multiple flexible conductors, a fabric-wrapped central section, and separated free wire ends.

4.1.1. Task Scope and Assumptions

The same physical wire harness is used throughout data collection and formal evaluation. A black tape segment is attached to the front half of the harness to represent a preferred grasp region. The tape resembles taped sections that may occur on real wiring harnesses and provides a visually consistent local feature for grasping.

Although the tape position is not supplied as a numerical grasp coordinate or visual annotation, it is visible in the camera observations. It should therefore be regarded as an engineered visual affordance rather than an externally specified grasp point.

The task is subject to the following assumptions:

1. Exactly one wire harness is present in each trial.
2. The harness is initially located inside the reachable workspace.
3. At least part of the harness is visible in the top-view image.
4. The harness is not severely occluded by other objects.
5. The robot base, cameras, table, background, and illumination remain fixed.
6. No grasp coordinate, bounding box, or object pose is supplied to the policy.
7. The policy only uses the defined visual and proprioceptive observations.
8. Minor and transient contact with the table is permitted.

During preliminary experiments, approximately vertical harness placements were found to be easier to grasp than approximately horizontal placements. To reduce harness orientation as an additional experimental variable, the harness is placed in an approximately vertical orientation during formal testing, while its position in the reachable workspace is varied.

The harness is allowed to extend across two or more workspace cells. Spatial evaluation is therefore based on the actual gripper landing or grasp location rather than on the geometric centre of the complete harness.

4.1.2. Observation and Action Spaces

At time step t , the method uses the policy observation defined in Equation (2.2), where I_t^{top} and I_t^{front} denote the top-view and front-view RGB images, respectively, and s_t denotes the robot proprioceptive state.

The SO-101 contains five arm joints and one gripper actuator. The proprioceptive state is therefore represented as

$$s_t = \begin{bmatrix} q_{t,1} & q_{t,2} & q_{t,3} & q_{t,4} & q_{t,5} & g_t \end{bmatrix}^T \in \mathbb{R}^6. \quad (4.1)$$

A single action is represented as

$$a_t = \begin{bmatrix} a_{t,1} & a_{t,2} & a_{t,3} & a_{t,4} & a_{t,5} & a_{t,g} \end{bmatrix}^\top \in \mathbb{R}^6. \quad (4.2)$$

The action corresponds to target positions for the five arm motors and the gripper motor.

4.1.3. Task Stages and Termination Conditions

Each trial begins with the robot returning to a predefined home pose and the gripper being reset to its initial opening. After the wire harness has been placed within the reachable workspace, the policy receives the current top-view image, front-view image, and robot proprioceptive state. Based on these observations, the robot moves the gripper towards the preferred grasp region, closes the gripper around the harness, transfers the harness away from its initial position, and finally lifts it from the table. These stages describe the intended progression of the manipulation task but are not provided to the policy as explicit stage labels. The ACT policy generates the complete behaviour directly from the visual and proprioceptive observations.

A grasp is considered valid only when the harness is securely retained by the gripper and moves together with the end effector during the subsequent transfer or lifting motion. Gripper closure alone is therefore insufficient to indicate a successful grasp. Cases in which the gripper closes without retaining the harness, pushes the harness across the table, or drags it without establishing stable retention are classified as grasp failures.

A trial is considered successful when the robot establishes a stable grasp, retains control of the harness during the required transfer motion, and lifts the grasped section visibly away from the table within the permitted execution time. Minor and temporary contact between the harness or gripper and the tabletop is allowed, provided that it does not prevent task completion or create a safety risk. A trial is terminated as a failure when the robot cannot grasp the harness, loses it during transfer or lifting, fails to complete the required motion, or produces unsafe behaviour that requires manual intervention. If none of the success or failure conditions is reached within the maximum execution time of 60 s, the trial is terminated and recorded as a timeout.

4.2. Dataset Collection Method

4.2.1. Hardware and Camera Configuration

Data were collected with the leader–follower robot and the two-camera system documented once in Section 5.1. The fixed top-view and gripper-mounted RGB streams were recorded

at 640×480 pixels and a nominal 30 FPS together with the follower state and leader-derived target action. Camera poses, robot-base pose, illumination, and background were held fixed; the intended experimental variation was the initial harness configuration.

4.2.2. Workspace Partitioning and Harness Placement

The reachable tabletop area measures approximately $60 \text{ cm} \times 35 \text{ cm}$. For data collection, it is divided into three coarse vertical regions in the top-camera image: left, centre, and right. The finer evaluation grid contains six columns, labelled A–F from left to right, and three rows, labelled 1–3 from the far side of the table towards the robot. Consequently, the left collection region contains columns A–B, the centre region contains columns C–D, and the right region contains columns E–F. Figure 4.3 shows both partitions and defines the spatial labels used throughout the thesis.

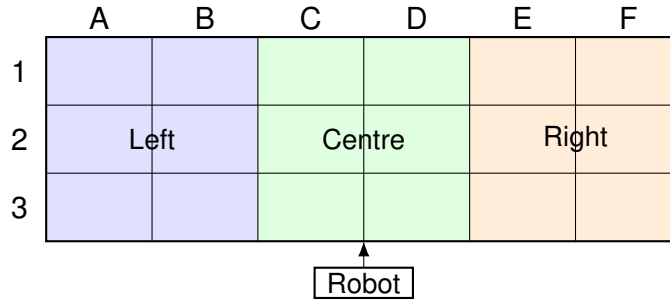


Figure 4.3.: Workspace partition used for balanced data collection and spatial evaluation. Columns A–B, C–D, and E–F form the left, centre, and right collection regions, respectively; the 6×3 grid defines the 18 evaluation cells.

Before each demonstration, the harness is placed within one of the three coarse regions. The dataset contains an equal number of demonstrations from each region, $N_{\text{left}} = N_{\text{centre}} = N_{\text{right}} = 65$.

This balanced distribution prevents one part of the workspace from dominating the demonstration dataset and provides spatial coverage for evaluating the policy in different initial harness locations. The table, background, cameras, and robot base remain fixed across all three regions.

The three regions represent an approximate spatial partition of the reachable workspace rather than three physically separated experimental environments. Within each region, the harness is repositioned between episodes to introduce variation in its position, orientation, and local shape while remaining visible and reachable by the robot.

4.2.3. Leader–Follower Demonstration Procedure

All demonstrations are collected by the same human operator using the leader–follower teleoperation configuration. At the beginning of each episode, the follower arm is placed

in a predefined home configuration. The operator then moves the leader arm to guide the follower arm through the complete manipulation sequence.

During a demonstration, the operator guides the follower arm from the home configuration towards the harness, closes the gripper around the selected harness segment, transfers the grasped harness, and lifts it from the table. The follower arm is the physical task-execution robot, whereas the leader arm provides the target motor-position commands used as expert actions.

The recording procedure starts and terminates automatically according to the configured episode-recording interval. No manual start or stop command is required during an individual demonstration. According to the stored dataset timestamps, each episode contains approximately 30 s of recorded data. The operator is therefore required to complete the demonstrated manipulation within this recording interval.

4.2.4. Robot State and Action Acquisition

The robot-state and action vectors are defined in Equations (4.1) and (4.2). Both use the channel order `shoulder_pan`, `shoulder_lift`, `elbow_flex`, `wrist_flex`, `wrist_roll`, and `gripper`.

At the hardware-interface level, the five arm-joint position channels are mapped to the range $[-100, 100]$ whereas the gripper-position channel is mapped to $[0, 100]$.

Accordingly, the recorded state satisfies $s_{t,j} \in [-100, 100]$, $j = 1, \dots, 5$, and $s_{t,6} \in [0, 100]$. For the gripper channel, $s_{t,6} = 0$ represents the fully closed configuration, whereas $s_{t,6} = 100$ represents the fully open configuration. Intermediate values represent partially open gripper configurations.

The demonstrated action corresponds to the target motor-position command generated from the leader-arm configuration and transmitted to the follower arm. The state and action therefore use the same six-channel ordering and the same hardware-level position representation.

These hardware-level values should not be confused with the dataset-statistics-based standardisation applied during ACT training.

4.2.5. Sampling Frequency and Temporal Synchronisation

During each recording iteration, the system acquires the top-view image, the front-view image, the current follower-arm state, and the corresponding demonstrated action. The recorded modalities at time step t are therefore associated through the same dataset frame index.

Because both cameras operate at 30 FPS and the robot data are recorded within the same software recording process, the images, states, and actions are nominally aligned at 30 Hz.

However, the two cameras are not synchronised using a shared external hardware trigger. The resulting alignment should therefore be described as software-level or nominal temporal synchronisation rather than strict hardware synchronisation.

The value of 30 Hz refers to the dataset sampling frequency. It does not necessarily describe the internal motor-control frequency of the SO-101 hardware interface.

4.2.6. Dataset Composition

Each recorded time step is represented as

$$d_t = (I_t^{\text{top}}, I_t^{\text{front}}, s_t, a_t, \delta_t, f_t, e_t, n_t, \kappa_t), \quad (4.3)$$

where

- I_t^{top} is the top-view RGB image;
- I_t^{front} is the front-view RGB image;
- $s_t \in \mathbb{R}^6$ is the follower-arm state;
- $a_t \in \mathbb{R}^6$ is the demonstrated action;
- δ_t is the timestamp;
- f_t is the frame index within the episode;
- e_t is the episode index;
- n_t is the global frame index; and
- κ_t is the task index.

The timestamp, frame index, episode index, global frame index, and task index preserve ordering and identify the stored record. They are metadata and are not direct policy inputs. The policy observation is therefore the visual and proprioceptive tuple defined in Equation (2.2). A demonstration episode is represented as

$$\mathcal{E}_i = \left\{ \left(o_t^{(i)}, a_t^{(i)} \right) \right\}_{t=0}^{T_i-1}, \quad (4.4)$$

where T_i denotes the number of recorded time steps in episode i . The complete demonstration dataset is defined as

$$\mathcal{D} = \{\mathcal{E}_i\}_{i=0}^{N_{\text{ep}}-1}, \quad N_{\text{ep}} = 195. \quad (4.5)$$

The final dataset contains $N_{\text{ep}} = 195$ demonstration episodes and $N_{\text{frame}} = 174,140$ recorded time steps.

At a sampling frequency of 30 Hz, the total recorded duration is approximately

$$T_{\text{total}} = \frac{N_{\text{frame}}}{f_s} = \frac{174,140}{30 \text{ s}^{-1}} \approx 5804.7 \text{ s} \quad (4.6)$$

The average number of recorded frames per episode is

$$\bar{T} = \frac{174,140}{195} \approx 893.03$$

frames, corresponding to an average stored duration of approximately $893.03/30 \approx 29.77 \text{ s}$ per episode.

4.2.7. Dataset Storage Format

The demonstrations are stored using the LeRobot dataset format and are available in the dataset repository ¹.

The repository is organised into three main directories: `data`, `meta`, `videos`.

The `data` directory stores the frame-level numerical data in Parquet files. These data include the robot state, demonstrated action, timestamp, frame index, episode index, global sample index, and task index.

The `videos` directory contains two separate video streams: `observation.images.front` and `observation.images.top`. The two streams are stored as MP4 video files with a resolution of 640×480 pixels, three image channels, and a frame rate of 30 FPS.

The `meta` directory contains the dataset description and supporting metadata. In particular, it includes the dataset information file, global dataset statistics, task definitions, and episode-level metadata. This structure allows the numerical robot data and corresponding camera frames to be associated through their episode indices, frame indices, and timestamps.

4.3. Policy Training

The collected demonstrations were used to train an Action Chunking with Transformers (ACT) policy. The overall architecture and operating principle of ACT have been introduced in the preceding theoretical chapter and are therefore not repeated here.

At each policy inference step, the model receives the front-view RGB image, the top-view RGB image, and the current robot state. Based on these observations, the policy predicts

¹<https://huggingface.co/datasets/kb127/test195>

a sequence of future robot actions rather than only a single next action.

The image preprocessing and state–action standardisation applied in this study follow the implemented LeRobot ACT pipeline [6].

All 195 recorded demonstration episodes were used for policy training. No separate training, validation, or test subset was constructed from the demonstration dataset.

A training sample at timestep t is represented as

$$\mathcal{S}_t = (I_t^{\text{front}}, I_t^{\text{top}}, s_t, \mathbf{A}_t), \quad (4.7)$$

where I_t^{front} and I_t^{top} denote the two camera observations, s_t denotes the current robot state, and $\mathbf{A}_t$ denotes the corresponding future action sequence.

4.3.1. Image Preprocessing

Each raw RGB image is initially stored as an HWC-ordered unsigned eight-bit array:

$$I_t^c \in \{0, \dots, 255\}^{480 \times 640 \times 3}, \quad c \in \{\text{top}, \text{front}\}. \quad (4.8)$$

Before being passed to the policy, the image is converted to a CHW-ordered single-precision tensor and scaled to the interval $[0, 1]$:

$$X_t^c = \frac{\text{permute}(I_t^c)}{255}, \quad (4.9)$$

where

$$X_t^c \in [0, 1]^{3 \times 480 \times 640}.$$

The image tensor is subsequently normalised using the ImageNet channel statistics:

$$\tilde{X}_{t,r}^c = \frac{X_{t,r}^c - \mu_r^{\text{ImageNet}}}{\sigma_r^{\text{ImageNet}}}, \quad (4.10)$$

where $r \in \{R, G, B\}$ denotes the colour channel.

No additional image resizing or cropping was applied in the ACT configuration used in this study. The spatial image resolution therefore remained 640×480 throughout the preprocessing pipeline.

Image augmentation was disabled. Consequently, no random cropping, colour jitter, geometric transformation, or other stochastic image augmentation was applied during training. This ensured that the visual inputs retained the same camera geometry and appearance as the recorded demonstrations.

4.3.2. State and Action Standardisation

The state and action representations remain those defined in Equations (4.1) and (4.2); the following transformations are applied component-wise to their six channels.

ACT standardises the robot state and action independently using statistics computed from the complete demonstration dataset. For state dimension j , the standardised value is

$$\tilde{s}_{t,j} = \frac{s_{t,j} - \mu_j^s}{\sigma_j^s + \varepsilon}, \quad (4.11)$$

where μ_j^s and σ_j^s denote the dataset mean and standard deviation of the corresponding state dimension.

Similarly, each action dimension is standardised as

$$\tilde{a}_{t,j} = \frac{a_{t,j} - \mu_j^a}{\sigma_j^a + \varepsilon}, \quad (4.12)$$

where μ_j^a and σ_j^a denote the corresponding action statistics, and ε is a small constant used to avoid numerical instability when the standard deviation is close to zero.

Before a predicted action is sent to the robot, the standardisation is reversed:

$$a_{t,j} = \tilde{a}_{t,j} (\sigma_j^a + \varepsilon) + \mu_j^a. \quad (4.13)$$

The complete processing pipeline therefore contains two distinct transformations:

1. hardware-interface range mapping for the SO-101 motor positions; and
2. dataset-statistics-based z-score standardisation for ACT training and inference.

The first transformation defines the common hardware representation of the joint and gripper positions, whereas the second transformation adjusts the numerical distribution of each state and action dimension for model training.

4.3.3. Action-Chunk Construction

Instead of predicting only the next robot action, ACT predicts a sequence of future actions. For a sampled timestep t , the target action chunk is defined as

$$\mathbf{A}_t = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}] \in \mathbb{R}^{H \times 6}. \quad (4.14)$$

The corresponding predicted action chunk is

$$\hat{\mathbf{A}}_t = [\hat{\mathbf{a}}_{t|t}, \hat{\mathbf{a}}_{t+1|t}, \dots, \hat{\mathbf{a}}_{t+H-1|t}] \in \mathbb{R}^{H \times 6}. \quad (4.15)$$

Here, H is determined by the parameter `chunk_size`. If fewer than H recorded actions remain between timestep t and the end of the episode, the action sequence is padded to the required fixed length. Padded action positions are excluded from the valid-action reconstruction loss by the padding mask implemented in the ACT training pipeline.

The parameter $H = \text{chunk_size}$ defines the predicted chunk length. The execution horizon $E = \text{n_action_steps}$ defines how many of those actions are executed before the next policy query.

The baseline policy therefore predicts an action chunk containing 100 future actions and provides all 100 actions to the execution queue.

4.3.4. Training Objective and Baseline Configuration

The ACT policy was trained using its standard action-reconstruction and variational regularisation objective. The reconstruction component penalises the difference between the predicted action chunk and the demonstrated action chunk, while the KL-divergence component regularises the latent representation. The mathematical definition of the ACT training objective and the KL-divergence term has already been introduced in the theoretical chapter.

Accordingly, the baseline ACT parameter configuration used in this study was

$$H = \text{chunk_size} = 100, \quad E = \text{n_action_steps} = 100, \quad \beta_{\text{KL}} = 10.$$

These values served as the reference configuration for the subsequent parameter-tuning experiments. During each tuning experiment, only the parameter under investigation was changed, while the remaining baseline parameters were kept constant whenever applicable.

4.3.5. Training Duration and Model Output

Each policy was trained for a fixed total of $N_{\text{train}} = 600\,000$ optimisation steps. Training was terminated automatically when the 600 000-step limit was reached.

Intermediate model checkpoints were not retained. Therefore, checkpoint-based model selection and early stopping were not applied. The model parameters obtained at the end of the 600 000th optimisation step were saved as the final trained policy and used in the subsequent physical robot evaluation.

Using a fixed number of optimisation steps ensured that the compared parameter configurations were trained under the same training-duration criterion.

4.4. Testing Protocol

The real-world testing protocol operationalises the task-specific benchmark. It specifies how each physical trial is allocated, initialised, executed, terminated, and counted.

Each ACT policy configuration is evaluated using the same robot platform, camera arrangement, workspace division, task definition, initialisation procedure, execution conditions, and success criteria. This controlled protocol is intended to ensure that differences in the experimental results can be attributed primarily to differences between the evaluated policy configurations.

4.4.1. Spatial Trial Allocation

The 18 cells defined in Figure 4.3 are used as the spatial test strata. Each policy is tested twice in every cell, resulting in $N_{\text{test}} = 18 \times 2 = 36$ valid physical trials per configuration.

For every trial, the corresponding spatial-cell identifier is recorded together with the trial outcome. The same spatial allocation procedure is applied to all evaluated ACT parameter configurations.

4.4.2. Controlled Experimental Conditions

The physical setup is the one documented in Section 5.1 and is not redescribed here. The same robot, harness, grid, cameras, nominal lighting, background, safety limits, and preprocessing are used for every configuration. Policy parameters remain fixed during autonomous execution; no online update, corrective demonstration, or manual modification of generated actions is permitted.

4.4.3. Trial Initialisation

Before every test trial, the experimental system is reset according to the same initialisation procedure:

1. the robot is returned to the predefined home pose;
2. the gripper is reset to the same initial opening;
3. the spatial test cell assigned to the trial is identified;
4. the wire harness is placed within the selected spatial cell;
5. the main direction of the harness is placed in an approximately vertical orientation relative to the image plane of the top-view camera;
6. at least part of the harness is confirmed to be visible in the top-view camera;

7. the camera streams and robot-state observations are checked for availability;
8. the previous policy action queue and all temporary execution states are cleared; and
9. the trial timer is started when the first policy-generated action is transmitted to the follower robot.

The placement procedure is kept consistent across the evaluated policy configurations. Once autonomous execution has started, neither the robot nor the harness is manually repositioned. Manual intervention is permitted only after the trial has terminated or when an immediate safety intervention is required.

4.4.4. Policy Execution

The online control loop is configured with a nominal target frequency of $f_{\text{control}} = 30$ Hz. The corresponding target control period is

$$\Delta t_{\text{control}} = \frac{1}{f_{\text{control}}} \approx 33.3 \text{ ms.} \quad (4.16)$$

Although the notation I_t^{front} is retained for consistency with the benchmark definition, the corresponding camera is physically mounted near the gripper and moves with the robot end effector.

Based on the current observation, the ACT policy produces the H -step action chunk defined in Equation (4.15).

At each control cycle, one target-position action is transmitted to the follower robot. The predicted actions are consumed sequentially according to the configured `n_action_steps` value. After the required number of actions has been executed, the policy receives an updated observation and generates a new action chunk.

The values of the action chunk size and `n_action_steps` depend on the evaluated ACT parameter configuration. Except for the parameter being evaluated, the testing procedure and physical experimental conditions remain unchanged.

The value of 30 Hz represents the configured target control frequency. It is not assumed that every control cycle is completed in exactly 33.3 ms. If timing measurements are available, the achieved control frequency and policy inference latency are reported separately in the experimental results.

4.4.5. Trial Outcome Assessment

The same success criterion is applied to all trials and all evaluated policy configurations. A trial is considered successful when the robot:

1. approaches the wire harness;
2. establishes a stable grasp;
3. retains the harness during the required transfer motion;
4. lifts the grasped harness segment from the tabletop; and
5. completes the task within the maximum permitted execution time.

A grasp is considered stable only when the harness follows the gripper without immediately slipping from the grasp. Closing the gripper without retaining the harness is not classified as a successful grasp. Likewise, merely pushing or dragging the harness across the tabletop without lifting it does not satisfy the task-success criterion.

The trial outcome is determined using the previously defined visual and task-based assessment procedure. The assessment is based on the observed robot behaviour and the camera recordings. When the outcome cannot be determined unambiguously during execution, the recorded camera observations are reviewed before the final trial label is assigned.

Each valid trial is assigned one of the following main task labels:

$$y_i \in \{\text{success}, \text{failure}\}. \quad (4.17)$$

In addition to the binary task outcome, the spatial test cell, execution duration, and primary termination reason are recorded for each trial.

4.4.6. Qualitative Failure Analysis Framework

To support the qualitative interpretation of policy behaviour, a predefined failure taxonomy was established for reviewing representative unsuccessful executions. The taxonomy distinguishes the following failure behaviours:

1. **Approach failure:** the robot does not reach an appropriate grasping position relative to the wire harness;
2. **Alignment failure:** the gripper reaches the wire harness but is not sufficiently aligned to establish a valid grasp;
3. **Grasp failure:** the gripper closes without securely retaining the wire harness;
4. **Retention failure:** the wire harness is initially grasped but subsequently slips from the gripper;
5. **Transfer failure:** the wire harness remains retained, but the required transport motion is not completed;

6. **Lift failure:** the policy does not lift the grasped wire-harness segment from the table-top;
7. **Timeout:** the task is not completed within the predefined maximum execution time; and
8. **Safety-related failure:** the policy behaviour requires a manual safety intervention.

This taxonomy is used as an interpretive framework for discussing observable failure behaviours in selected trials. However, a complete and consistent trial-level annotation of failure causes was not collected for all policy executions. Therefore, the individual failure categories are not reported as quantitative metrics, and no category-wise failure frequencies or percentages are calculated.

Where multiple failure behaviours are visible within the same execution, the earliest observable behaviour that prevents successful task completion may be used to describe the representative failure case. Such descriptions are qualitative and should not be interpreted as a complete classification of all unsuccessful trials.

4.4.7. Termination and Safety Conditions

Each trial has a maximum execution time of

$$T_{\max} = 60 \text{ s.}$$

The timer starts when the first policy-generated action is transmitted to the follower robot. The timer stops when task success, task failure, timeout, safety intervention, or an invalid-trial condition is recorded.

The timeout condition is defined as

$$\chi_{\text{timeout}} = \mathbb{I}[t - t_0 \geq T_{\max}], \quad (4.18)$$

where t_0 denotes the time at which the first policy-generated action is transmitted.

Minor and transient contact between the gripper, wire harness, and tabletop is permitted because contact is an expected part of the manipulation task. However, a trial is terminated for safety when the policy causes any of the following conditions:

1. a hard collision;
2. sustained compression of the robot or gripper against the tabletop;
3. motor stalling;
4. motion outside the predefined safe workspace;

5. collision with fixed experimental equipment; or
6. any condition requiring immediate manual intervention.

A safety intervention caused by policy behaviour is counted as a task failure. Such a trial remains part of the 36 valid trials used to calculate the real-world task-success rate.

4.4.8. Invalid Trials and Repetition Procedure

A trial is classified as invalid only when it is interrupted by a policy-independent technical fault. Examples include:

1. failure of a camera stream;
2. loss of communication with the robot;
3. failure to transmit or receive robot-state data;
4. unexpected shutdown of the control software; or
5. another hardware or software fault that is not caused by the policy behaviour.

Invalid trials are excluded from the calculation of the task-success rate. They are repeated in the same spatial test cell until the predefined number of valid trials has been obtained. For each policy configuration, testing therefore continues until

$$N_{\text{valid}} = 36.$$

The valid-trial count satisfies

$$N_{\text{valid}} = N_{\text{success}} + N_{\text{failure}}. \quad (4.19)$$

Because two valid trials are required in each of the 18 spatial regions, an invalid trial in one cell is repeated in that same cell rather than being replaced by an additional trial in another region. This preserves the balanced spatial distribution of the evaluation.

Although invalid trials are excluded from the task-success calculation, their number and technical causes are documented separately to maintain transparency regarding the reliability of the experimental system.

For each test trial, the available evaluation records include:

- the evaluated policy configuration;
- the trial identifier;

- the assigned spatial-cell identifier;
- the available images from the top-view and gripper-mounted cameras;
- the available robot and gripper states;
- the policy-generated target actions;
- the trial start and termination times;
- the total execution duration;
- the binary task outcome;
- whether a safety intervention occurred; and
- whether the trial was valid or invalid.

These records provide the basis for the task-success, spatial-robustness, offline action-prediction, and motion-smoothness evaluations. They also permit a qualitative inspection of selected unsuccessful executions.

4.4.9. Evaluation Metrics

For policy configuration i , the physical task-success rate is

$$R_i = \frac{N_{\text{success},i}}{N_{\text{valid},i}} \times 100\%, \quad (4.20)$$

where $N_{\text{success},i}$ is the number of successful valid trials and $N_{\text{valid},i} = 36$ is the valid-trial count. The spatial coverage is

$$\Gamma_i = \frac{N_{\text{cell},i}^{\text{success}}}{18}, \quad (4.21)$$

where $N_{\text{cell},i}^{\text{success}}$ counts cells with at least one successful trial.

Offline errors are calculated after inverse standardisation in the recorded SO-101 position representation. For T evaluated frames and $d = 6$ action components, the global Action L1 error is

$$E_{\text{action}} = \frac{1}{Td} \sum_{t=1}^T \sum_{j=1}^d |\hat{a}_{t,j} - a_{t,j}|, \quad (4.22)$$

where $a_{t,j}$ and $\hat{a}_{t,j}$ are the recorded and predicted values of component j at time step t . Gripper L1 uses only $j = 6$. A reported “mean $\pm$ standard deviation” denotes the mean and frame-level standard deviation of the per-frame six-component L1 error. The 1000-frame curves average that per-frame error within consecutive windows and do not represent training loss.

4.4.10. Motion Smoothness

The temporal consistency of the predicted action sequence is evaluated using a first-order action smoothness metric. For a predicted action sequence $\{\hat{\mathbf{a}}_1, \dots, \hat{\mathbf{a}}_T\}$, the mean change between consecutive actions is defined as

$$S_{\text{action}} = \frac{1}{T-1} \sum_{t=2}^T \|\hat{\mathbf{a}}_t - \hat{\mathbf{a}}_{t-1}\|_1. \quad (4.23)$$

A larger value indicates greater changes between consecutive action commands and may be associated with less continuous or more oscillatory motion. Abrupt action changes can reduce grasp stability, particularly when manipulating a deformable object such as a wire harness.

Nevertheless, smoothness must be interpreted together with task success. An excessively smooth policy may react too slowly, produce insufficient correction movements, or fail to close the gripper at the appropriate moment. Therefore, a lower smoothness value does not necessarily indicate a better manipulation policy [52].

For a fair comparison, all configurations are evaluated using the same control frequency, initial robot pose, maximum trial duration, and termination criteria.

4.5. ACT Parameter Tuning Strategy

A controlled parameter study was conducted to investigate the effects of the ACT action horizon, KL regularisation weight, and learning rate. The baseline configuration used a chunk size of 100, 100 executed action steps, and a KL weight of 10.

For each comparison, only the parameter under investigation was changed, while the dataset, image preprocessing, model architecture, batch size, training duration, and remaining optimisation settings were kept unchanged. All policies were trained for 600 000 optimisation steps. No separate training–validation split or intermediate checkpoint selection procedure was applied. Therefore, the policy obtained at the final training step was used for subsequent offline and real-robot evaluation.

4.5.1. `chunk_size` / `n_action_steps`

In the ACT policy [6], `chunk_size` and `n_action_steps` are two key parameters that determine the length of the predicted action sequence and the frequency of closed-loop replanning. The corresponding H -step policy and target sequence are defined in Equations (3.9) and (4.14). Therefore, `chunk_size` defines how many future action steps the model predicts in a single forward pass. It can be interpreted as the temporal length of an action chunk or an action primitive. During training, the model uses the next `chunk_size` actions

as the supervised target and learns a mapping from the current visual observation and robot state to a continuous sequence of target joint positions. During inference, the model also outputs an action sequence of length `chunk_size`, which is then used for execution. `n_action_steps` defines how many actions from the predicted chunk are actually executed before the policy is queried again. For example, when `chunk_size = 100` and `n_action_steps = 100`, the model predicts 100 future actions and executes the entire sequence before receiving a new observation and performing another inference step. When `chunk_size = 100` and `n_action_steps = 50`, the model still predicts 100 actions, but only the first 50 actions are executed; the remaining actions are discarded, and a new action chunk is predicted from the updated observation. Thus, `chunk_size` mainly controls the temporal length of the learned and generated action sequence, whereas `n_action_steps` controls the replanning frequency during rollout. In the implementation, `n_action_steps` must be smaller than or equal to `chunk_size`, since the number of executed actions cannot exceed the number of predicted actions.

The role of these two parameters can be understood as a trade-off between action coherence and closed-loop reactivity. A larger `chunk_size` enables the policy to learn longer and more coherent action patterns, such as approaching the target, closing the gripper, and lifting the object as one continuous manipulation primitive. Predicting action chunks reduces the effective horizon of the task and helps mitigate compounding errors in imitation learning. It can also model non-Markovian structures in human demonstrations, such as pauses, preparation phases, or temporally correlated behavior that cannot be easily represented by a single-step policy. However, if the chunk length is too large, the policy must predict a long future trajectory from the current observation alone, which can reduce reactivity when the environment changes during execution.

For the wire harness grasping task, tuning these two parameters is necessary because the manipulated object is deformable. During gripper contact, closing, dragging, and lifting, the shape, position, and orientation of the wire harness may change rapidly. The default setting, `chunk_size = 100` and `n_action_steps = 100`, provides smooth action execution and reduces the number of policy queries. However, it also means that the robot executes a relatively long action sequence before incorporating new visual feedback. If the grasp point shifts, the wire is pushed away, the gripper does not fully close, or the wire deforms after lifting, the robot may continue executing actions generated from an outdated observation. This can amplify the initial error and lead to failure. Reducing `n_action_steps` increases the frequency of closed-loop replanning, allowing the robot to update its actions earlier based on the new camera images and robot state.

Nevertheless, smaller values are not always better. A smaller `chunk_size` or `n_action_steps` makes the policy more reactive, but it may also weaken the temporal coherence that ACT gains from action chunking. If the action chunk is too short, the model may only learn local movements rather than a complete grasping primitive. If the execution horizon is too

short, the policy may replan too frequently, leading to discontinuous motion, hesitation, or even regression toward common safe poses in the training data. Therefore, these parameters should be selected according to the task dynamics, control frequency, and real-robot rollout behavior. For wire harness manipulation, a relatively large `chunk_size` can preserve the structure of the grasping motion, while a moderately smaller `n_action_steps` can improve closed-loop correction without completely destroying action continuity. The final choice should be validated through real-world evaluation using metrics such as grasp success rate, motion smoothness, wire slippage, execution time, and task completion stability.

4.5.2. KL Weight

The theoretical ACT objective and its KL regularisation term are defined in Equations (3.15)–(3.17). In the LeRobot implementation used in this study, the configuration parameter `kl_weight` directly specifies the coefficient β_{KL} in Equation (3.15).

The baseline configuration used `kl_weight` = 10. Additional values were evaluated while the dataset, model architecture, preprocessing pipeline, optimiser settings, and training duration were kept unchanged. This isolates the effect of the KL weight on offline action reconstruction and real-world policy execution.

4.5.3. Learning Rate

The learning rate, denoted as η , is an important optimization hyperparameter in the training process of the ACT policy. It controls the step size of parameter updates during gradient-based optimization. ACT formulates imitation learning as a supervised sequence prediction problem, where the policy predicts a future action chunk from the current observation. The observation consists of multi-view RGB images and robot joint states, while the output is a sequence of target joint positions for the next H timesteps. In the original ACT training procedure, the policy is trained as a conditional variational autoencoder (CVAE), and the encoder and decoder parameters are updated using the Adam optimizer with a loss composed of a reconstruction term and a KL regularization term. Therefore, the learning rate directly affects the update speed of the visual encoder, Transformer encoder and decoder, CVAE latent module, and final action regression head.

In ACT, the learning rate does not change the policy input-output structure, the robot control frequency, or the action chunk size. Instead, it determines how quickly and stably the model learns the visual-motor mapping from demonstration data. In the original ACT hyperparameter setting, the learning rate is set to 1×10^{-5} , together with a batch size of 8, a chunk size of 100, $\beta_{\text{KL}} = 10$, and dropout of 0.1. In the LeRobot implementation of ACT, the training configuration also includes `optimizer_lr` and `optimizer_lr_backbone`, which control the learning rate of the general policy network and the visual backbone respectively.

This indicates that learning rate tuning can affect both the action prediction module and the image feature extraction module.

If the learning rate is too low, each parameter update becomes very small. This usually makes training more stable, but also slows down convergence. Under a limited training budget, an excessively low learning rate may cause underfitting, where both training and validation losses decrease slowly and the learned policy only captures coarse or averaged action patterns. In the wire harness manipulation task, this may result in weak arm motions, insufficient grasping behavior, unclear gripper closing timing, or policies that approach the wire but fail to reliably complete grasping and lifting.

If the learning rate is too high, the parameter updates may become too aggressive, resulting in oscillating losses, unstable validation performance, gradient explosion, or policy collapse. Since ACT predicts an entire action sequence rather than a single-step action, an overly large learning rate can damage the temporal consistency inside each action chunk. During real robot execution, this may appear as sudden arm movements, unstable gripper behavior, overshooting, or excessive correction after contact with the wire harness. The ACT paper emphasizes that small action errors in fine manipulation can accumulate over time and push the robot into states outside the training distribution, making training stability especially important for this type of task.

Therefore, learning rate tuning aims to balance convergence speed and policy stability. For contact-rich and visually sensitive tasks such as wire harness grasping, the default ACT learning rate of 1×10^{-5} can be used as the baseline. A practical tuning range is 1×10^{-5} , and 3×10^{-5} . If the training loss decreases too slowly and the rollout behavior is weak or incomplete, the learning rate can be increased moderately. If the loss curve oscillates strongly, or if the rollout shows jerky motion, overshooting, or unstable gripper timing, the learning rate should be reduced. For the visual backbone, a smaller `optimizer_lr_backbone` may also be used to prevent over-updating pretrained visual features on a small task-specific dataset. The final learning rate should be selected based not only on training loss, but also on validation loss, grasping success rate, trajectory smoothness, and task completion time.

5. Implementation

5.1. Hardware Setup

The implementation of the proposed system involves two separate computing platforms. An NVIDIA Jetson AGX Orin Developer Kit was used for robot communication, demonstration collection, and real-world policy evaluation, whereas the ACT policy was trained on a high-performance GPU workstation. Separating the deployment platform from the training platform allowed the computationally intensive optimisation process to be performed on a dedicated GPU while retaining an embedded computing device for real-time robot operation.

5.1.1. Robot Manipulation Platform

The robotic platform consists of one SO-101 leader arm and one SO-101 follower arm. During demonstration collection, the operator manually moves the leader arm, and its joint positions are transmitted to the follower arm. The follower arm reproduces the demonstrated motion and directly interacts with the wire harness.

Each arm contains five actuated arm joints and one gripper joint. Consequently, the robot state and action use the six-dimensional representations defined in Equations (4.1) and (4.2).

The follower arm is equipped with a parallel gripper used to approach, grasp, transfer, and lift the wire harness. Before every demonstration episode and evaluation trial, the follower arm is returned to a predefined home configuration, and the gripper is reset to a consistent initial opening.

5.1.2. Camera System

Two RGB cameras provide the visual observations used for demonstration collection and policy execution. A top-view camera observes the complete workspace and captures the global position, orientation, and shape of the wire harness. A front-view camera is mounted close to the manipulation area and provides a detailed view of the gripper and the local grasping interaction.

Both cameras acquire RGB images at a resolution of 640×480 pixels and operate at approximately 30 Hz.

The camera poses, viewing directions, background, table position, and lighting conditions were kept unchanged throughout data collection and real-world evaluation.

5.1.3. Experimental Workbench

The complete experimental workbench is shown in Figure 5.1. The SO101 leader and follower arms were installed along the right side of the work surface. An aluminium-profile frame surrounded the manipulation area and provided fixed mounting positions for the visual sensing and illumination equipment.

The manipulation surface consisted of a flat white board located within the reachable workspace of the follower arm. Orange reference lines were marked on the board to divide the workspace into spatial regions.

The photograph documents the physical placement of the components. Camera roles and acquisition settings are specified once in Section 5.1.2, while the grid definition and trial use are specified in Figure 4.3 and Chapter 4.

Power, communication, and camera cables were routed around the aluminium frame and outside the principal manipulation region. This arrangement reduced the probability that external cables would interfere with the robot motion or appear as task-relevant objects within the central workspace.

5.1.4. Deployment Computer

Robot communication, camera acquisition, dataset recording, and real-world policy inference were performed on an NVIDIA Jetson AGX Orin Developer Kit. The platform uses the ARM64 architecture and integrates a 12-core ARM Cortex-A78AE processor with an NVIDIA Orin GPU.

The principal deployment hardware is summarised in Table 5.1.

Table 5.1.: Hardware configuration of the data-collection and deployment platform.

Component	Configuration
Platform	NVIDIA Jetson AGX Orin Developer Kit
Architecture	ARM64 / AArch64
CPU	12-core ARM Cortex-A78AE
GPU	Integrated NVIDIA Orin GPU
Operating system	Ubuntu 22.04.5 LTS
Kernel	Linux 5.15.148-tegra
NVIDIA L4T release	R36.4.7

5.1.5. Training Workstation

The ACT policy was trained on a separate high-performance workstation equipped with an NVIDIA RTX A6000 GPU. The GPU provides approximately 49 GB of graphics memory, which is sufficient for processing two high-resolution image streams and training the transformer-based ACT architecture.

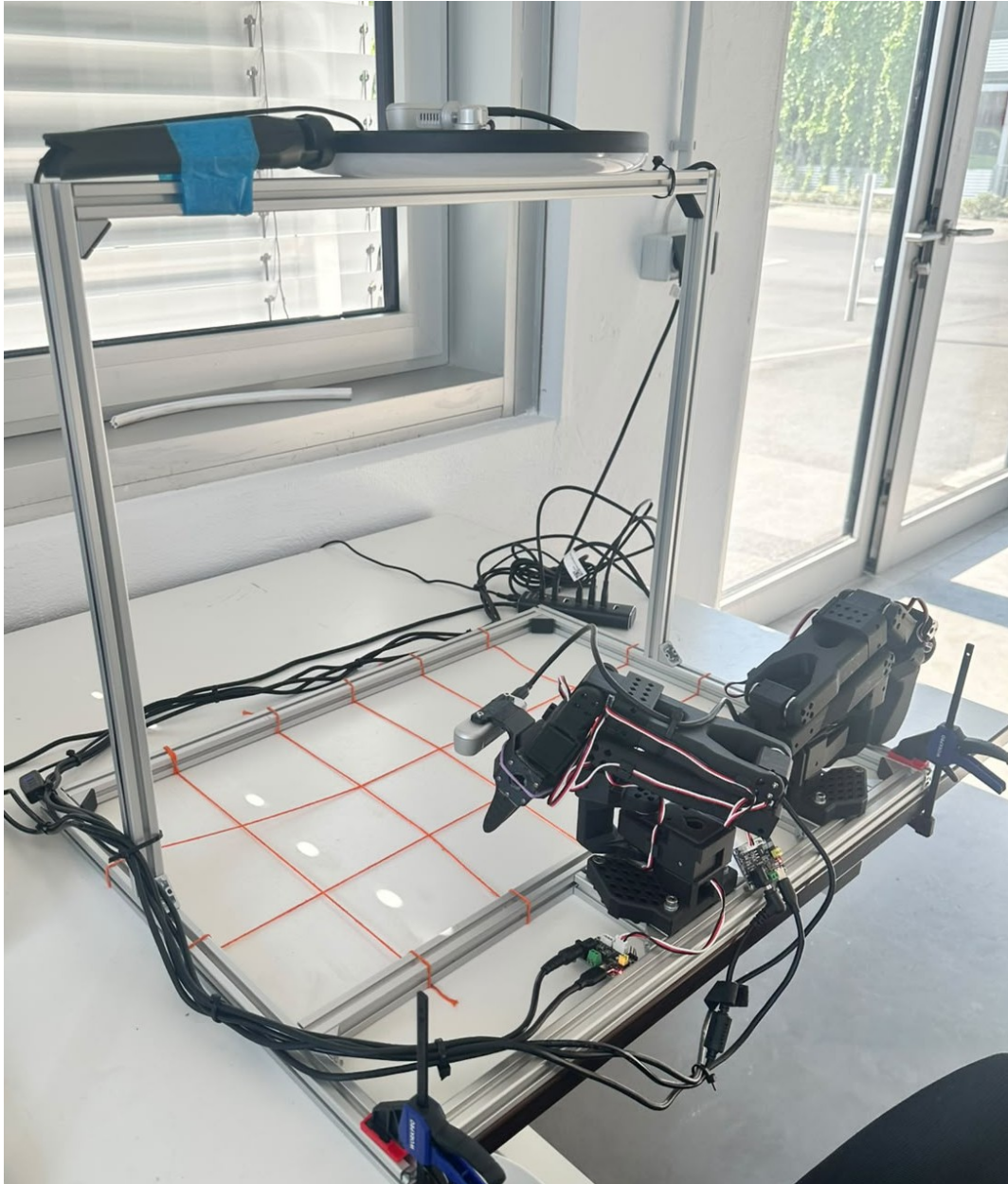


Figure 5.1.: Overview of the experimental workbench, including the SO101 leader–follower system, aluminium mounting frame, fixed camera and illumination arrangement, and the spatially divided manipulation surface.

The workstation contains an AMD Ryzen Threadripper PRO 5975WX processor with 32 physical cores and 64 logical CPU threads. Its main hardware configuration is presented in Table 5.2.

Table 5.2.: Hardware configuration of the ACT training workstation.

Component	Configuration
GPU	NVIDIA RTX A6000
GPU memory	49,140 MiB
CPU	AMD Ryzen Threadripper PRO 5975WX
Physical CPU cores	32
Logical CPU threads	64
Threads per core	2
System architecture	x86_64
Operating system	Ubuntu 22.04.5 LTS
Linux kernel	6.8.0-124-generic

5.2. Software Setup

The software implementation is based on the LeRobot framework and PyTorch. LeRobot provides the interfaces for robot communication, teleoperation, multimodal data recording, dataset management, policy training, and real-world policy deployment. PyTorch is used to implement and optimise the neural-network models, while the Hugging Face libraries provide dataset and model repository interfaces.

Two different software environments were used in this work. The NVIDIA Jetson AGX Orin environment was used for robot communication, demonstration collection, and real-world policy execution. The ACT policy was trained on a separate GPU workstation. The two environments are therefore described separately.

5.2.1. LeRobot

LeRobot [15] is an open-source framework for end-to-end robot learning. It is designed to integrate real-world robot control, teleoperation-based data collection, standardised multi-modal datasets, policy training, and policy deployment within a unified software stack.

Rather than proposing a single robot-learning algorithm, LeRobot addresses the fragmentation of the robot-learning ecosystem by providing consistent hardware interfaces, the `LeRobotDataset` format, reusable PyTorch implementations of multiple learning policies, and an asynchronous inference stack. It supports low-cost robotic platforms such as the SO-100 and SO-101, allowing users to read leader-arm states for teleoperation, command follower arms, record visual and proprioceptive observations, and deploy trained policies directly on physical robots.

In this thesis, LeRobot serves as the principal experimental framework. It is used to collect demonstrations with the SO-101 leader–follower system, record the top-view and front-view image observations together with robot states and actions, organise the demonstrations in the LeRobot dataset format, train imitation-learning policies, and execute the learned wire-harness grasping policy on the follower arm.

Although several policy implementations, including ACT, π_0 , and SmolVLA, are available within the LeRobot ecosystem, ACT is the policy used for the main training, parameter-tuning, and real-world evaluation experiments in this thesis. The other policies are therefore part of the available software framework rather than the primary experimental method.

At timestep t , LeRobot processes the policy observation defined in Equation (2.2), where

$$I_t^{\text{top}}, I_t^{\text{front}} \in \mathbb{R}^{3 \times 480 \times 640} \quad (5.1)$$

denote the top-view and front-view RGB images. The state and action are the six-dimensional vectors in Equations (4.1) and (4.2).

LeRobot additionally provides configuration interfaces for defining input and output features, dataset repositories, normalisation statistics, image preprocessing, policy parameters, training devices, and model storage locations. The detailed dataset representation is described in Section 5.3, while the ACT configuration is presented in Section 5.4.

5.2.2. Deployment Software Environment

The NVIDIA Jetson AGX Orin used for demonstration collection and real-world policy execution runs Ubuntu 22.04.5 LTS with the NVIDIA Linux-for-Tegra kernel. The system uses the AArch64 architecture, and its Python environments are managed using Miniforge and Conda.

The deployment environment contains Python 3.12.12 and the CUDA 12.6 toolkit. The CUDA compiler reports version 12.6.68. This environment is used for leader–follower robot communication, camera acquisition, episode recording, local policy loading, and real-world inference.

Table 5.3.: Software environment of the Jetson deployment platform.

Software component	Version or configuration
Operating system	Ubuntu 22.04.5 LTS
System architecture	AArch64
Linux kernel	5.15.148-tegra
Python	3.12.12
Environment manager	Miniforge / Conda
CUDA toolkit	12.6
CUDA compiler	NVCC 12.6.68
NVIDIA-SMI	540.4.0

The exact LeRobot and PyTorch package versions installed on the Jetson system were not contained in the recorded system information. They should therefore only be added to Table 5.3 after verification on the deployment computer.

5.2.3. Training Software Environment

The ACT policy was trained on a separate x86_64 workstation running Ubuntu 22.04.5 LTS. The training environment uses Python 3.12.12, LeRobot 0.5.1, and PyTorch 2.10.0.

The NVIDIA driver reports CUDA 13.0 as the maximum driver-supported CUDA version. However, the installed PyTorch package is 2.10.0+cu128, meaning that the CUDA runtime bundled with and used by PyTorch is version 12.8. These two values refer to different software layers and are therefore not contradictory. The recorded training environment is summarised in Table 5.4.

Table 5.4.: Software environment used for ACT training.

Software component	Version
Operating system	Ubuntu 22.04.5 LTS
System architecture	x86_64
Linux kernel	6.8.0-124-generic
Python	3.12.12
LeRobot	0.5.1
PyTorch	2.10.0+cu128
PyTorch CUDA runtime	12.8
NVIDIA driver	580.126.16
Driver-supported CUDA version	13.0
Torchvision	0.25.0
Transformers	5.3.0
Accelerate	1.13.0
Hugging Face Hub	1.7.1
NumPy	2.2.6

5.2.4. PyTorch and CUDA Acceleration

The ACT model is implemented and trained using PyTorch. PyTorch provides the tensor operations, automatic differentiation, neural-network modules, and optimisation tools required for ACT training. Computationally intensive operations, including convolutional visual-feature extraction, transformer attention, forward propagation, backpropagation, and optimiser updates, are executed on the NVIDIA RTX A6000 GPU.

The training installation uses the CUDA-enabled PyTorch build 2.10.0+cu128. Therefore, CUDA 12.8 is the runtime associated with PyTorch training, whereas CUDA 13.0 reported by `nvidia-smi` indicates the compatibility level of the installed NVIDIA driver.

Mixed-precision training was not enabled in the final training configuration. Consequently, model optimisation was performed without automatic mixed precision, even though the GPU and PyTorch environment support CUDA acceleration.

5.2.5. Supporting Libraries

Torchvision 0.25.0 provides image-related neural-network components and image processing utilities. The Transformers library version 5.3.0 supports transformer-related modules, while Accelerate 1.13.0 provides device and execution utilities.

The Hugging Face Hub library version 1.7.1 is used to access dataset and policy repositories. NumPy 2.2.6 is used for numerical operations and array processing. Camera acquisition and image handling are performed through the camera interfaces integrated into LeRobot and OpenCV-compatible image representations.

5.2.6. Transfer from Training to Deployment

After training, the ACT policy is transferred from the GPU workstation to the Jetson AGX Orin. During real-world evaluation, the policy is loaded from a local filesystem path instead of being downloaded repeatedly from an online repository.

At each control step, the deployment software acquires the two RGB images and the current follower-arm state. The observations are processed using the same feature definitions and normalisation statistics used during training. The ACT policy then predicts a future action sequence, and the selected actions are sent to the follower arm.

Robot observations, commanded actions, timestamps, and evaluation recordings are stored locally. This offline execution procedure prevents network availability from affecting policy evaluation and ensures that all trials use the same trained policy.

5.3. Data Structure

This section describes how the collected demonstrations are represented in the file system and converted into the tensor structures required by the ACT policy.

5.3.1. Dataset Overview

The dataset used in this work is published in the Hugging Face repository `kb127/test195` and follows the LeRobotDataset v3.0 format. It contains $N_{\text{ep}} = 195$ recorded demonstration episodes and $N_{\text{frame}} = 174\,140$ individual frame records. The nominal recording frequency stored in the dataset metadata is $f_s = 30$ Hz.

The complete dataset follows the representation in Equation (4.5), where $\mathcal{E}_i$ denotes the i -th episode. If episode i contains T_i frames, the total number of frames satisfies

$$\sum_{i=0}^{N_{\text{ep}}-1} T_i = 174\,140. \quad (5.2)$$

The average number of recorded frames per episode is $\bar{T} = \frac{174\,140}{195} \approx 893.03$. At a sampling frequency of 30 Hz, this corresponds to an average recorded sequence length of approximately $\frac{893.03}{30} \approx 29.77$ s.

The actual frame count reported in the metadata is used instead of the theoretical frame count derived from the nominal recording duration.

All demonstrations were collected by the same operator. All 195 episodes were used for ACT training. No separate training, validation, or test subset was created for offline model development. Correspondingly, the repository metadata defines only one split: `train = [0, 195)`. The real-world policy evaluations described later in this thesis were conducted as separate robot rollouts and were not stored as an offline test split in the demonstration dataset.

5.3.2. Episode Structure

Each episode is represented as a temporally ordered sequence of synchronized frame records. A single record follows Equation (4.3), where

- I_t^{front} is the front-view RGB observation;
- I_t^{top} is the top-view RGB observation;
- s_t is the measured follower-arm state;
- a_t is the recorded robot action;
- δ_t is the timestamp within the episode;
- f_t is the frame index within the episode;
- e_t is the episode index;
- n_t is the global frame index in the complete dataset; and
- κ_t is the task index.

The corresponding LeRobot feature keys are

```
observation.images.front
observation.images.top
observation.state
action
timestamp
frame_index
episode_index
index
task_index
```

The natural-language task description is not duplicated in every frame record. Instead, each frame contains a `task_index`. During dataset loading, this index is used to retrieve the corresponding task string from `meta/tasks.parquet`. The LeRobot dataset loader then adds this string to the returned sample under the key `task`.

Dataset-level information and episode-level metadata are stored separately from the per-frame data. Consequently, metadata such as the dataset version, feature definitions, episode lengths, file offsets, and normalization statistics are not treated as direct policy inputs.

5.3.3. Image Data Structure

The dataset contains two camera features: $c \in \{\text{front}, \text{top}\}$.

Their exact LeRobot feature names are `observation.images.front` and `observation.images.top`. Before video encoding, a camera observation follows the raw RGB representation in Equation (4.8). The logical image representation therefore has the following properties:

- image height: 480 pixels;
- image width: 640 pixels;
- number of channels: 3;
- channel order: RGB;
- tensor order before conversion: HWC; and
- original pixel datatype: unsigned eight-bit integer.

The image payload is not stored directly inside the Parquet frame table. Instead, the two camera streams are encoded as separate MP4 video files. According to the dataset metadata, the videos use the AV1 codec with the `yuv420p` pixel format and a nominal frame rate of 30 fps.

During data loading, the required video frame is located from the frame timestamp and the episode-specific video offset metadata. The decoded frame is converted into a PyTorch tensor according to

$$I_{t,\text{raw}}^c \in \mathbb{R}^{480 \times 640 \times 3} \longrightarrow I_t^c \in \mathbb{R}^{3 \times 480 \times 640}. \quad (5.3)$$

The resulting tensor uses the CHW convention and the `float32` datatype. Its pixel values are scaled to

$$I_t^c \in [0, 1]^{3 \times 480 \times 640}. \quad (5.4)$$

After batching, each camera tensor has the shape

$$\mathbf{I}^c \in \mathbb{R}^{B \times 3 \times 480 \times 640}, \quad (5.5)$$

where B denotes the training batch size. In the ACT implementation, the two camera tensors are handled as a list of image features:

$$[\mathbf{I}^{\text{front}}, \mathbf{I}^{\text{top}}]. \quad (5.6)$$

They are therefore not required to be stored as a single explicit tensor of shape $B \times 2 \times 3 \times 480 \times 640$.

5.3.4. Robot State and Action Structure

The robot state and action are the six-dimensional floating-point vectors defined in Equations (4.1) and (4.2).

Their component ordering is

$$\mathbf{s}_t = \begin{bmatrix} s_t^{\text{shoulder_pan}} \\ s_t^{\text{shoulder_lift}} \\ s_t^{\text{elbow_flex}} \\ s_t^{\text{wrist_flex}} \\ s_t^{\text{wrist_roll}} \\ s_t^{\text{gripper}} \end{bmatrix}, \quad \mathbf{a}_t = \begin{bmatrix} a_t^{\text{shoulder_pan}} \\ a_t^{\text{shoulder_lift}} \\ a_t^{\text{elbow_flex}} \\ a_t^{\text{wrist_flex}} \\ a_t^{\text{wrist_roll}} \\ a_t^{\text{gripper}} \end{bmatrix}. \quad (5.7)$$

The corresponding feature names stored in the dataset are

```
shoulder_pan.pos
shoulder_lift.pos
elbow_flex.pos
wrist_flex.pos
```

```
wrist_roll.pos
gripper.pos
```

The state vector s_t , stored under `observation.state`, represents the measured positions of the follower-arm motors at the current control step. These values are obtained from the follower motor feedback register `Present_Position`.

The action vector a_t , stored under `action`, originates from the measured positions of the leader arm. During teleoperation, these values are mapped to the follower arm and sent to its motor controllers as `Goal_Position` commands. The recorded actions therefore represent absolute target positions rather than position increments or velocity commands. This target-position formulation is consistent with the original ACT formulation, in which the policy predicts future target joint positions [6].

Only position features are included in the dataset used in this work. The state and action vectors do not contain measured velocity, acceleration, motor current, force, torque, or temperature information.

5.3.5. Dataset File Structure

The dataset uses the LeRobotDataset v3.0 directory structure. Its relevant files are organised as follows:

```
test195/
|-- data/
|   '-- chunk-000/
|       '-- file-000.parquet
|
|-- videos/
|   |-- observation.images.front/
|   |   '-- chunk-000/
|   |       |-- file-000.mp4
|   |       |-- file-001.mp4
|   |       '-- ...
|   |
|   '-- observation.images.top/
|       '-- chunk-000/
|           |-- file-000.mp4
|           |-- file-001.mp4
|           '-- ...
|
'-- meta/
```

```
|-- info.json
|-- stats.json
|-- tasks.parquet
'-- episodes/
    '-- chunk-000/
        '-- file-000.parquet
```

The functions of the individual files are described below.

Frame data. The file `data/chunk-000/file-000.parquet` contains the low-dimensional frame data. This includes the robot state, action, timestamp, frame index, episode index, global index, and task index. The encoded image payload is not stored in this Parquet file.

Video data. The encoded front-view and top-view camera streams are stored in `videos/observation.images.front` and `videos/observation.images.top`, respectively. Multiple episodes may share the same MP4 shard. Episode boundaries and video offsets are recovered from the episode metadata.

Dataset information. The file `meta/info.json` defines the dataset version, robot type, frame rate, dataset splits, path templates, total numbers of episodes, frames, and tasks, and the names, shapes, and datatypes of all features.

Dataset statistics. The file `meta/stats.json` stores statistical values for the dataset features, including the mean, standard deviation, minimum, maximum, selected quantiles, and sample count. The statistics of the state and action features are used for numerical normalization.

Episode metadata. The file under `meta/episodes` stores information associated with each episode, including its length, task assignment, global frame range, Parquet location, video-file location, and the offsets required to retrieve the correct video segment.

Task metadata. The file `meta/tasks.parquet` stores the mapping between numerical task indices and natural-language task descriptions. Since the present dataset contains one task, all frame records use the same task index.

Unlike earlier LeRobot dataset versions, the v3.0 format does not require one Parquet and one video file for every episode. Instead, multiple episodes are stored in larger data and video shards, while their logical separation is preserved through episode metadata.

5.3.6. Data Preprocessing Structure

Image preprocessing

The image-processing pipeline applied before ACT inference can be summarised as

$$I_{\text{video}} \longrightarrow I_{\text{CHW}} \longrightarrow I_{[0,1]} \longrightarrow \tilde{I}. \quad (5.8)$$

After video decoding, the image is converted from HWC to CHW format as shown in Equation (5.3). The decoded integer values are converted to `float32` and divided by 255:

$$I_{t,c,h,w} = \frac{I_{t,c,h,w}^{\text{uint8}}}{255}. \quad (5.9)$$

The ACT dataset configuration uses ImageNet channel statistics for visual normalization according to Equation (4.10). The fixed channel statistics are

$$\boldsymbol{\mu}^{\text{ImageNet}} = \begin{bmatrix} 0.485 & 0.456 & 0.406 \end{bmatrix},$$

and

$$\boldsymbol{\sigma}^{\text{ImageNet}} = \begin{bmatrix} 0.229 & 0.224 & 0.225 \end{bmatrix}.$$

The original 480×640 spatial resolution is retained. No image resizing or cropping is applied. Image augmentation is disabled, so no random colour, affine, sharpness, or geometric transformation is introduced during training.

State and action preprocessing

The robot state and action are normalized independently using the statistics computed from their respective dataset features. For $\mathbf{x} \in \{\mathbf{s}_t, \mathbf{a}_t\}$, mean–standard-deviation normalization is defined as

$$\tilde{\mathbf{x}} = \frac{\mathbf{x} - \boldsymbol{\mu}_x}{\boldsymbol{\sigma}_x + \varepsilon}. \quad (5.10)$$

The state input uses $(\boldsymbol{\mu}_s, \boldsymbol{\sigma}_s)$ and the action target uses $(\boldsymbol{\mu}_a, \boldsymbol{\sigma}_a)$ in Equation (5.10).

The state and action statistics are calculated separately and stored under the `observation.state` and `action` entries of `stats.json`. The same statistics are loaded by the policy preprocessor during training and online inference.

The ACT network predicts actions in the normalized action space. Before an action is transmitted to the follower arm, the policy postprocessor performs the inverse transformation:

$$\hat{\mathbf{a}}_t = \tilde{\mathbf{a}}_t \odot (\boldsymbol{\sigma}_a + \varepsilon) + \boldsymbol{\mu}_a, \quad (5.11)$$

where $\odot$ denotes element-wise multiplication. The resulting six-dimensional vector is then interpreted as an absolute target-position command.

5.3.7. Action Chunk Data Structure

ACT does not use a single action as its supervised target. Instead, for an observation at time step t , the training target is a sequence of future actions, using the target-chunk definition in Equation (4.14). Here $H = \text{chunk_size}$ denotes the number of actions in one predicted chunk. Each row of $\mathbf{A}_t$ is a six-dimensional absolute target-position vector.

For a dataset recorded at 30 Hz, the action sequence corresponds to the temporal offsets

$$\Delta\mathcal{T} = \left[0, \frac{1}{30}, \frac{2}{30}, \dots, \frac{H-1}{30}\right]. \quad (5.12)$$

The LeRobot ACT configuration creates this sequence using action-delta indices

$$[0, 1, \dots, H-1].$$

Near the end of an episode, fewer than H future actions may remain. To maintain a fixed tensor shape, query indices outside the episode boundary are clipped to the final valid frame. Consequently, the final valid action is repeated in the padded positions. A Boolean padding mask is generated simultaneously:

$$\mathbf{m}_t = \left[m_t, m_{t+1}, \dots, m_{t+H-1}\right] \in \{\text{False}, \text{True}\}^H, \quad (5.13)$$

and is stored under the key `action_is_pad`. A value of `True` indicates that the corresponding action position lies outside the valid episode range.

During ACT training, the padding mask is applied to the element-wise reconstruction error. For action dimension $d_a = 6$, the masked L1 term can be expressed as

$$\mathcal{L}_{\text{L1}} = \frac{1}{BHd_a} \sum_{b=1}^B \sum_{h=1}^H \sum_{j=1}^{d_a} (1 - m_{b,h}) |\hat{a}_{b,h,j} - a_{b,h,j}|. \quad (5.14)$$

The padded positions therefore contribute zero reconstruction error and are not treated as valid action targets. The same mask is also supplied to the CVAE Transformer encoder so that padded action tokens are ignored by its attention mechanism.

For a batch size B , the principal ACT input and target shapes in this work are

```

observation.images.front :  $B \times 3 \times 480 \times 640$ ,
observation.images.top :  $B \times 3 \times 480 \times 640$ ,
observation.state :  $B \times 6$ ,
action :  $B \times H \times 6$ ,
action_is_pad :  $B \times H$ ,
predicted action :  $B \times H \times 6$ .

```

For the baseline value $H = 100$, the action target and model output have shape $B \times 100 \times 6$, while the padding mask has shape $B \times 100$.

5.4. ACT Implementation

The theoretical principles of Action Chunking with Transformers (ACT) and the conditional variational autoencoder have already been introduced in the preceding chapters. Therefore, this section focuses on the concrete implementation used in this work, including the policy interface, visual feature extraction, model configuration, action-chunk execution, data normalisation, and online inference procedure. The implementation is based on the ACT policy provided by LeRobot and adapted to the two-camera, single-arm SO-101 setup used in this study [6].

5.4.1. ACT Input Interface

At time step t , the policy receives the observation defined in Equation (2.2). The robot state and action use the six-dimensional representations in Equations (4.1) and (4.2). In the LeRobot interface, the two camera observations remain separate dictionary entries and are collected as a list; no explicit $B \times 2 \times 3 \times 480 \times 640$ tensor is constructed. The complete batch shapes are listed in Section (5.3.7). Only the current observation is used because

$$n_obs_steps = 1.$$

Consequently, no history of previous images or robot states is supplied to the policy.

5.4.2. Visual Feature Extraction

A ResNet-18 network is used as the visual backbone. Its parameters are initialised using `ResNet18_Weights.IMAGENET1K_V1`. The final classification layer is not used; instead, the feature map produced by the fourth residual stage, `layer4`, is extracted. Frozen batch-normalisation layers are used inside the backbone.

Both camera views are processed sequentially by the same backbone instance. The top-view and front-view cameras therefore share all ResNet parameters rather than using two independent visual encoders. For an input resolution of 480×640 pixels and the standard ResNet-18 stride configuration, each camera produces a feature map of approximately

$$F_t^c \in \mathbb{R}^{B \times 512 \times 15 \times 20}. \quad (5.15)$$

A 1×1 convolution projects the backbone output to the transformer hidden dimension $d_{\text{model}} = 512$, which is included in the complete architecture configuration below. A two-dimensional sinusoidal positional encoding is generated for each feature map to preserve spatial information. The feature map is then flattened along its spatial dimensions:

$$15 \times 20 = 300$$

visual tokens per camera. With two cameras, the transformer encoder receives a total of 600 visual tokens.

5.4.3. Transformer and CVAE Implementation

The implemented ACT policy uses a transformer encoder, a transformer decoder, and an additional transformer encoder for the variational latent representation. The architecture used in this work is configured as follows:

$$\begin{aligned} d_{\text{model}} &= 512, \\ d_{\text{feedforward}} &= 3200, \\ N_{\text{heads}} &= 8, \\ N_{\text{encoder}} &= 4, \\ N_{\text{decoder}} &= 1, \\ N_{\text{VAE encoder}} &= 4, \\ d_{\text{latent}} &= 32, \\ p_{\text{dropout}} &= 0.1. \end{aligned}$$

ReLU is used as the feed-forward activation, and the implementation applies post-normalisation because `pre_norm` is disabled.

During training, the variational encoder receives a learned [CLS] token, the embedded current robot state, and the embedded target action chunk. For a chunk length of $H = 100$, the variational encoder input contains

$$1 + 1 + H = 102$$

tokens. The output associated with the [CLS] token is linearly projected to the mean and log-variance of the 32-dimensional latent distribution. A latent sample is then obtained using the reparameterisation operation.

During inference, the target action sequence is unavailable and the variational encoder is not evaluated. Instead, the latent input is set to

$$z = \mathbf{0} \in \mathbb{R}^{32}. \quad (5.16)$$

The policy transformer encoder then receives the latent token, the robot-state token, and the visual tokens from the two cameras. In the present configuration, its input sequence therefore contains approximately

$$1 + 1 + 2(15 \times 20) = 602$$

tokens, each with a dimensionality of 512.

The transformer decoder is supplied with H learned positional embeddings, one for each predicted action step. Its output is projected by a linear action head from 512 dimensions to the six-dimensional SO-101 action space.

5.4.4. Action Chunk Prediction and Execution

For each policy invocation, the model predicts an action chunk

$$\hat{\mathcal{A}}_t = \text{stack}_{b=1}^B \left(\hat{\mathbf{A}}_t^{(b)} \right) \in \mathbb{R}^{B \times H \times 6}, \quad (5.17)$$

where each per-sample chunk $\hat{\mathbf{A}}_t^{(b)}$ has the structure defined in Equation (4.15).

The baseline values are given in Section (4.3.4). The chunk size defines the number of actions predicted by the network, whereas the execution horizon defines how many predictions are retained after each policy invocation. The implementation enforces

$$E = \text{n_action_steps} \leq H = \text{chunk_size}.$$

Temporal ensembling is disabled in the baseline configuration:

$$\text{temporal_ensemble_coeff} = \text{None}.$$

The predicted actions are therefore not averaged with predictions from overlapping chunks. Instead, the first 100 predicted actions are inserted into a first-in, first-out action queue. One action is removed from the queue and returned during each control cycle. The policy is queried again only when the queue becomes empty.

At the control frequency of 30 Hz, the baseline queue corresponds to an execution interval

of

$$T_{\text{queue}} = \frac{E}{f_{\text{control}}} = \frac{100}{30 \text{ s}^{-1}} \approx 3.33 \text{ s}. \quad (5.18)$$

Thus, the baseline policy incorporates a new camera observation and robot state once every 100 executed actions. Newly predicted actions do not overlap with the previous chunk and are not appended while an earlier chunk is still being consumed.

5.4.5. ACT Loss Implementation

The mathematical formulation of the ACT objective has already been presented earlier. In the LeRobot implementation, the reconstruction component is the masked element-wise L1 loss in Equation (5.14), where $1 - m_{b,h}$ is the valid-position mask. Padded action positions near the end of an episode therefore contribute zero reconstruction error. In the total loss, this term is denoted by $\mathcal{L}_{\text{action}}$.

When the variational objective is enabled, the KL-divergence term is computed from the predicted latent mean μ and log-variance $\log \sigma^2$:

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \text{mean} \left[\sum_{j=1}^{d_{\text{latent}}} (1 + \log \sigma_j^2 - \mu_j^2 - \exp(\log \sigma_j^2)) \right]. \quad (5.19)$$

The total loss is

$$\mathcal{L} = \mathcal{L}_{\text{action}} + \beta_{\text{KL}} \mathcal{L}_{\text{KL}}, \quad (5.20)$$

The baseline uses $\beta_{\text{KL}} = \text{kl_weight} = 10$, as specified in Equation (4.3.4); the variational objective is therefore enabled.

5.4.6. Pre-processing and Policy Output

The ACT pre-processing pipeline first adds a batch dimension to each observation, transfers the tensors to the configured computing device, and normalises the visual observations, robot state, and action targets using statistics stored with the dataset.

The state and action tensors use the dataset-statistics normalisation in Equation (5.10). The RGB inputs use the fixed ImageNet channel statistics in Equation (4.10), which are compatible with the ImageNet-pretrained ResNet-18 backbone.

The model predicts actions in the normalised action space. The post-processing pipeline applies the inverse transformation in Equation (5.11) and transfers the resulting action tensor to the CPU.

In this study, the resulting six-dimensional vector represents absolute target positions for the five arm joints and the gripper. It is subsequently passed to the follower-arm control interface. The ACT model and the provided pre/post-processing code do not themselves apply joint clipping, velocity limiting, or per-step action-change limiting. Any such constraints

must therefore be implemented and documented at the robot-interface or deployment level rather than being attributed to the ACT network.

5.4.7. Online Inference and Runtime Handling

Online policy execution can be summarised as follows:

1. acquire the current top-view image, front-view image, and follower-arm state;
2. add the batch dimension, transfer the data to the inference device, and apply dataset-based normalisation;
3. invoke the ACT model when the action queue is empty;
4. place the first `n_action_steps` predictions into the action queue;
5. remove the next action from the queue;
6. unnormalise the action and transfer it to the CPU;
7. send the six-dimensional target-position command to the follower arm.

The `select_action` and `predict_action_chunk` functions are executed without gradient tracking, and the policy is explicitly placed in evaluation mode. Whenever an episode or evaluation trial is reset, the policy reset operation clears the action queue. This prevents unused actions from a previous trial from being executed in the next trial.

The provided ACT policy files do not contain explicit handling for missing camera frames, robot communication failures, non-finite output actions, manual emergency stops, or automatic return to the home position.

5.5. Training Configuration

This section documents the dataset, baseline configuration, training procedure, and hyperparameter configurations used in the experiments. The functions and theoretical effects of the ACT parameters were introduced in the preceding chapters and are therefore not repeated here. Experimental results and comparisons between the trained policies are presented in Chapter 6.

A total of 10 ACT configurations were trained. Unless explicitly stated otherwise, all parameters of an experimental configuration were identical to those of the baseline configuration.

5.5.1. Training Dataset Configuration

All configurations were trained using the same dataset containing 195 demonstration episodes. Each episode had a duration of approximately 30s. The policy received observations from a front-view RGB camera and a fixed top-view RGB camera. Each image used the $3 \times 480 \times 640$ tensor shape defined in Equation (5.1).

The robot state and action were both represented as six-dimensional vectors. The state contained the positions of five arm joints and the gripper, while the action represented the corresponding target positions.

All 195 episodes were used for training. No independent validation or test subset was created. Therefore, errors calculated from the available dataset are referred to as *offline action prediction errors* rather than test errors. Such measurements describe action reconstruction on the recorded dataset but do not independently evaluate generalisation to unseen data.

No image augmentation, resizing, or cropping was applied. Images used fixed ImageNet channel statistics, whereas robot states and actions used statistics computed from their respective dataset features.

5.5.2. Baseline Training Configuration

The baseline configuration was obtained from the saved LeRobot ACT policy configuration. It used one observation step, predicted action chunks of length 100, and executed 100 predicted actions before querying the policy again.

Table 5.5 lists the baseline parameters used in the implementation. The theoretical meanings of these parameters have already been discussed in Sections 5.4 and 4.5; the table is included only to document the implemented configuration.

The ResNet-18 backbone was initialised using the `ResNet18_Weights.IMAGENET1K_V1` pretrained weights. The backbone parameters formed a separate optimiser parameter group, although both the policy and backbone learning rates were set to 1×10^{-5} in the baseline configuration.

The ACT optimisation preset used AdamW with a gradient-clipping norm of 10 and no learning-rate scheduler. Automatic mixed precision was disabled by setting `use_amp=false`.

5.5.3. Training Procedure

For each training run, the LeRobot dataset and its associated normalisation statistics were loaded. Mini-batches containing the front-view image, top-view image, current robot state, and corresponding future action chunk were then sampled from the dataset.

For a configuration with chunk size H , the target action chunk is the $H \times 6$ tensor defined in Equation (4.14).

Table 5.5.: Baseline ACT configuration used in this study.

Parameter	Baseline value
Policy type	ACT
Number of observation steps	1
Number of RGB cameras	2
Image shape per camera	$3 \times 480 \times 640$
State dimension	6
Action dimension	6
Chunk size	100
Number of action steps	100
Vision backbone	ResNet-18
Backbone initialisation	ImageNet1K_V1
Model dimension	512
Feed-forward dimension	3200
Feed-forward activation	ReLU
Number of attention heads	8
Transformer encoder layers	4
Transformer decoder layers	1
Pre-normalisation	Disabled
VAE	Enabled
VAE encoder layers	4
Latent dimension	32
Dropout	0.1
KL weight	10
Reconstruction loss	L1 loss
Optimiser	AdamW
Policy learning rate	1×10^{-5}
Backbone learning rate	1×10^{-5}
Weight decay	1×10^{-4}
Gradient-clipping norm	10
Learning-rate scheduler	None
Mixed-precision training	Disabled
Temporal ensembling	Disabled
Parameter-efficient fine-tuning	Disabled
Visual normalisation	Fixed ImageNet mean–standard deviation
State normalisation	Dataset mean–standard deviation
Action normalisation	Dataset mean–standard deviation
Training steps	600,000
Validation split	None
Test split	None

The two RGB observations and the current robot state were passed through the ACT policy. During training, the VAE encoder additionally received the target action chunk. The model then predicted the tensor in Equation (5.17).

The total training objective is defined in Equation (5.20), where β_{KL} is determined by the KL weight of the corresponding configuration.

After the forward pass, the total loss was backpropagated and the model parameters were updated using AdamW. Each configuration was trained for 600,000 optimisation steps.

No validation phase was performed, and validation loss was not used for early stopping or checkpoint selection. Training stopped after the predefined number of steps. No intermediate checkpoint was retained for the subsequent evaluation; the model obtained at step 600,000 was treated as the final model of the corresponding configuration.

5.5.4. Hyperparameter Experiment Configuration

The hyperparameter experiments were organised as a staged empirical search rather than an exhaustive grid search. The baseline configuration was first used as a reference for examining the effects of `chunk_size`, `n_action_steps`, `kl_weight`, and the policy learning rate. Controlled pairwise comparisons were used wherever possible so that the effect of one parameter could be examined while the remaining parameters were held constant.

Based on the results of the individual parameter comparisons, selected parameter values were subsequently combined in an additional configuration. The purpose of this combined experiment was to examine whether parameter changes that appeared beneficial individually remained effective when applied together. The final objective was to identify a suitable parameter configuration among the evaluated candidates for deployment on the physical robot. The experiment was not intended to establish a globally optimal ACT hyperparameter configuration.

The experiment matrix contained 10 configurations. In the original experiment records, pairs such as “50/100” were written in the order `n_action_steps/chunk_size`. For clarity, Table 5.6 lists the chunk size before the number of action steps. The configuration identifiers use C for `chunk_size` and A for `n_action_steps`.

All parameters not explicitly identified as modified were kept identical to the baseline configuration. All runs used the same dataset, preprocessing pipeline, normalisation method, model components, batch size, optimiser type, and number of optimisation steps. Except for the policy learning rate in the LR3e-5 configuration, the remaining optimiser settings were also unchanged.

The `chunk_size` parameter determines the length of the future action sequence predicted by ACT and used as the training target. For a chunk size H , the policy predicts the sequence in Equation (5.17). Changing `chunk_size` therefore changes the prediction horizon and the output sequence length of the policy during training.

Table 5.6.: ACT configurations used in the hyperparameter experiments.

Configuration	Chunk size	Action steps	KL weight	Policy LR	Backbone LR	Modified parameter(s)
Baseline	100	100	10	1×10^{-5}	1×10^{-5}	None
C100-A50	100	50	10	1×10^{-5}	1×10^{-5}	Action steps
C100-A80	100	80	10	1×10^{-5}	1×10^{-5}	Action steps
C50-A50	50	50	10	1×10^{-5}	1×10^{-5}	Chunk size and action steps
C150-A100	150	100	10	1×10^{-5}	1×10^{-5}	Chunk size
KL5	100	100	5	1×10^{-5}	1×10^{-5}	KL weight
KL15	100	100	15	1×10^{-5}	1×10^{-5}	KL weight
KL18	100	100	18	1×10^{-5}	1×10^{-5}	KL weight
LR3e-5	100	100	10	3×10^{-5}	1×10^{-5}	Policy learning rate
C150-A100-KL15	150	100	15	1×10^{-5}	1×10^{-5}	Chunk size and KL weight

In contrast, `n_action_steps` determines how many actions from the predicted chunk are executed before the policy processes a new observation and generates another action chunk. For a predicted chunk of length H and an execution horizon E , the controller executes

$$(\hat{\mathbf{a}}_t, \hat{\mathbf{a}}_{t+1}, \dots, \hat{\mathbf{a}}_{t+E-1}), \quad E \leq H. \quad (5.21)$$

The remaining predicted actions are not executed after the policy is queried again. Consequently, the Baseline, C100-A50, and C100-A80 configurations share the same training prediction horizon of 100 steps. They differ only in the number of predicted actions executed before the next observation is processed. These configurations therefore evaluate different execution horizons and policy re-query frequencies rather than different training target lengths.

Although these configurations were trained in separate runs, differences between their training losses should not be attributed directly to `n_action_steps`. This parameter controls policy execution but does not change the action chunk used as the training target when `chunk_size` remains fixed. Differences in their training curves may instead arise from stochastic effects in model initialisation, minibatch sampling, and optimisation. The primary comparison between these configurations must therefore be based on their behaviour during real-robot execution.

The C50-A50 and C100-A50 configurations use the same execution length of 50 steps. Comparing them allows the effect of increasing `chunk_size` from 50 to 100 to be examined while keeping `n_action_steps` fixed. Similarly, the Baseline and C150-A100 configurations both execute 100 actions before re-querying the policy and therefore allow chunk sizes of 100 and 150 to be compared under the same execution length.

The KL5, KL15, and KL18 configurations differ from the Baseline only in the KL weight. Together with the Baseline KL weight of 10, these configurations provide a direct comparison of

$$\beta_{\text{KL}} \in \{5, 10, 15, 18\}.$$

The KL weight controls the relative contribution of the latent-space regularisation term in the ACT objective. This comparison is intended to determine whether weaker or stronger

latent regularisation produces more accurate and reliable robot behaviour for the collected human demonstrations.

The LR3e-5 configuration changes only the policy learning rate from 1×10^{-5} to 3×10^{-5} . The backbone learning rate remains fixed at 1×10^{-5} . This configuration therefore examines whether faster optimisation of the policy-specific layers improves convergence without simultaneously changing the update rate of the visual backbone.

The C150-A100-KL15 configuration combines a chunk size of 150 with a KL weight of 15. It was introduced after the individual parameter comparisons to investigate whether the larger prediction horizon and the increased KL weight remained beneficial when applied together. Comparing C150-A100-KL15 with C150-A100 isolates the effect of changing the KL weight from 10 to 15 under a chunk size of 150. Conversely, comparing C150-A100-KL15 with KL15 examines the effect of increasing the chunk size from 100 to 150 while keeping the KL weight fixed at 15. These comparisons provide evidence about whether promising parameter values identified separately can be combined into a more suitable deployment configuration.

All configurations were trained for 600,000 optimisation steps. No separate training, validation, and test split was used, and no validation set was available for model selection. Consequently, the final configuration was not selected according to validation loss.

Training loss and offline action prediction error were used as diagnostic measures of optimisation and action reconstruction quality. However, because the offline error was calculated using data from the same collected dataset rather than an independent test set, it does not constitute an independent estimate of policy generalisation. In addition, a lower offline prediction error does not necessarily imply better closed-loop behaviour, because small action errors may accumulate during physical execution and move the robot outside the demonstration distribution.

The final comparison of the configurations is presented in Chapter 6. Real-world task success rate is used as the primary selection criterion because it directly measures whether the policy completes the wire-harness grasping task. Motion smoothness and offline action prediction error are used as supplementary metrics to distinguish configurations with similar success rates and to analyse the causes of successful or unsuccessful execution.

Accordingly, the selected parameter set represents the best-performing configuration among the configurations evaluated in this study. It should not be interpreted as a globally optimal ACT configuration, because the experiments covered only a limited set of parameter values and combinations.

5.5.5. Training Runtime and Model Storage

Each configuration was trained as a separate run and was assigned an individual output directory. Only the final model obtained after 600,000 optimisation steps was retained for evaluation.

6. Results

This chapter presents the quantitative evaluation of the ACT policies trained with different hyperparameter configurations. The evaluation contains two complementary components. First, the trained policies are evaluated offline by comparing their predicted actions with the recorded demonstration actions. Second, each policy is deployed on the physical robot and evaluated in 36 real-world trials distributed across 18 predefined workspace regions.

Complete training histories containing the total loss, action L1 training loss, and KL loss were not retained. Consequently, this chapter does not report training-loss convergence and does not reconstruct or approximate missing training curves. The available action L1 curves are results of offline policy evaluation and must not be interpreted as training-loss curves.

The main objective is to determine how the selected hyperparameters influence real-robot task performance and whether offline action-prediction metrics are consistent with closed-loop task success.

6.1. Evaluation Overview

Ten ACT configurations were evaluated. The original experiment names record the number of executed action steps before the chunk size. In this chapter, the notation C denotes `chunk_size`, while A denotes `n_action_steps`. For example, the original configuration 100_150 is denoted as $C150-A100$.

The trial allocation, success criterion, and metric definitions are given once in Sections 4.4.1, 4.4.5, and 4.4.9. This chapter reports the resulting measurements without repeating the experimental setup.

6.2. Offline Policy Evaluation Results

Since complete training-loss histories were unavailable, the quantitative offline analysis is based on the retained policy replay results. Each policy was evaluated using 174,140 recorded samples. The recorded observations were provided to the policy, and the predicted actions were compared with the corresponding recorded actions.

The retained offline results contain the Action L1 error, Gripper L1 error, and first-order predicted-action variation defined in Section 4.4.9.

The mean ground-truth smoothness was identical across the evaluated configurations because all policies were compared against the same recorded dataset:

$$S_{GT} = 0.196151$$

Table 6.1 reports the available offline metrics. A dash indicates that no corresponding replay output was retained.

Table 6.1.: Offline policy replay metrics.

Configuration	Action L1	Gripper L1	Predicted smoothness
<i>C100–A50–KL10</i>	–	–	–
<i>C50–A50–KL10</i>	–	–	–
<i>C100–A100–KL5</i>	10.864 ± 7.675	5.228	0.5809
Baseline: <i>C100–A100–KL10</i>	7.467 ± 5.919	3.679	0.3834
<i>C100–A80–KL10</i>	10.219 ± 10.920	4.404	0.3340
<i>C100–A100–KL10–LR3 × 10^{−5}</i>	3.655 ± 3.285	2.096	0.3405
<i>C100–A100–KL15</i>	6.150 ± 6.194	3.800	0.3587
<i>C100–A100–KL18</i>	7.925 ± 6.858	3.767	0.6065
<i>C150–A100–KL10</i>	8.932 ± 5.423	4.622	0.3898
<i>C150–A100–KL15</i>	11.089 ± 5.982	5.314	0.6038

The learning-rate configuration produced the smallest mean Action L1 error, with a value of 3.655, and also produced the smallest Gripper L1 error. The *C150–A100–KL15* configuration produced the largest retained mean Action L1 error, 11.089. The relationship between these offline rankings and physical task success is interpreted in Chapter 7. The remaining offline-evaluation curves are shown in Appendix A.2.

Figure 6.1 presents representative 1000-frame Action L1 error curves. These curves show the variation of offline prediction error across different parts of the recorded dataset. They are policy replay results rather than training-loss curves.

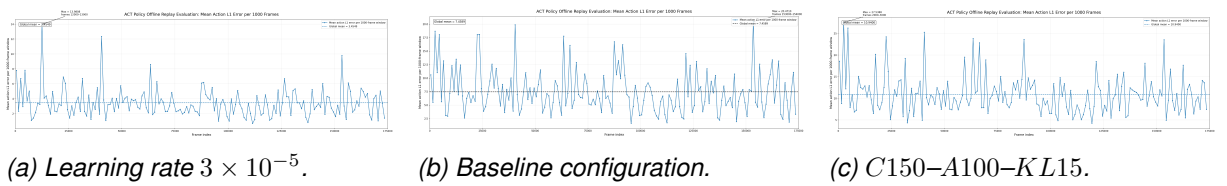


Figure 6.1.: Representative offline Action L1 error curves averaged over consecutive 1000-frame windows.

The local peaks in these curves indicate that some parts of the recorded dataset were more difficult to reproduce than others. However, the current results do not identify which task phase or workspace position corresponds to each peak. Therefore, the peaks are treated as indicators of non-uniform prediction difficulty rather than evidence of a specific failure mechanism.

6.3. Real-Robot Task Success

Table 6.2 presents the complete real-robot success results for the ten evaluated configurations.

Table 6.2.: Real-robot success rates for the evaluated ACT configurations.

Configuration	Trials	Successes	Success rate
<i>C</i> 100– <i>A</i> 50– <i>KL</i> 10	36	0	0.0%
<i>C</i> 50– <i>A</i> 50– <i>KL</i> 10	36	0	0.0%
<i>C</i> 100– <i>A</i> 100– <i>KL</i> 5	36	10	27.8%
Baseline: <i>C</i> 100– <i>A</i> 100– <i>KL</i> 10	36	8	22.2%
<i>C</i> 100– <i>A</i> 80– <i>KL</i> 10	36	4	11.1%
<i>C</i> 100– <i>A</i> 100– <i>KL</i> 10– $\text{LR}3 \times 10^{-5}$	36	2	5.6%
<i>C</i> 100– <i>A</i> 100– <i>KL</i> 15	36	14	38.9%
<i>C</i> 100– <i>A</i> 100– <i>KL</i> 18	36	6	16.7%
<i>C</i> 150– <i>A</i> 100– <i>KL</i> 10	36	13	36.1%
<i>C</i> 150– <i>A</i> 100– <i>KL</i> 15	36	21	58.3%

The best-performing configuration was *C*150–*A*100–*KL*15, which completed 21 of the 36 trials. Compared with the baseline result of eight successes, this corresponds to an increase of 13 successful trials and an absolute success-rate improvement of

$$58.3\% - 22.2\% = 36.1\%$$

percentage points.

The two configurations using only 50 executed action steps failed all 36 trials. Although the available results do not identify the exact failure stage, they demonstrate that an execution horizon of 50 action steps was not sufficient for reliable completion of the tested grasping sequence.

Changing the number of executed actions from 100 to 80 also reduced performance. The *C*100–*A*80 configuration achieved four successes, compared with eight for the baseline. Together with the zero-success 50-step results, this indicates that reducing the executed action horizon had a consistently negative effect in the tested configurations.

The influence of the KL weight was non-monotonic. With *C*100–*A*100, increasing the KL weight from 10 to 15 increased the number of successes from eight to 14. Increasing the KL weight further to 18 reduced the number of successes to six. Therefore, the largest tested KL weight did not produce the best result.

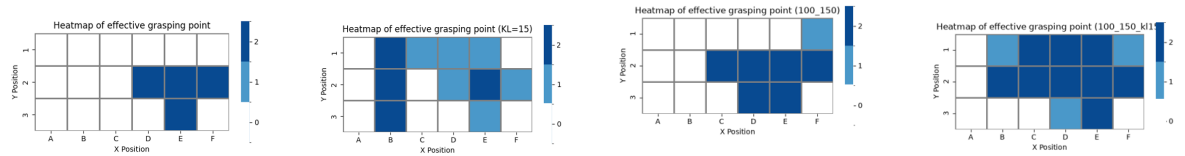
Increasing the chunk size from 100 to 150 improved the result when the executed action horizon remained 100. At *KL* = 10, the number of successes increased from eight to 13. Combining the larger chunk size with *KL* = 15 increased the number of successes further to 21.

These results suggest that the larger chunk size and the intermediate KL weight were complementary for the current task. However, each configuration was represented by one trained policy, and the results do not separate the effect of the hyperparameter from training stochasticity.

6.4. Spatial Distribution of Successful Grasps

Overall success rate alone does not indicate whether a policy generalises across the complete workspace. A policy may repeatedly succeed in a small number of favourable regions while failing in the remaining regions. Therefore, the successful trials were also analysed according to their spatial distribution.

Each heatmap contains 18 cells. A value of zero indicates that both trials in the corresponding region failed, a value of one indicates one successful trial, and a value of two indicates that both trials succeeded. All figures all show in Appendix A.1.



(a) Baseline.

(b) $C100-A100-KL15$.

(c) $C150-A100-KL10$.

(d) $C150-A100-KL15$.

Figure 6.2.: Spatial distribution of successful grasps for representative configurations. Each region was tested twice.

Table 6.3.: Spatial coverage of the available real-robot evaluations.

Configuration	Regions with success	Regions with 2/2 successes	Total successes
$C100-A100-KL5$	7/18	3	10
Baseline	4/18	4	8
$C100-A80-KL10$	2/18	2	4
$LR3 \times 10^{-5}$	2/18	0	2
$C100-A100-KL15$	10/18	4	14
$C100-A100-KL18$	4/18	2	6
$C150-A100-KL10$	7/18	6	13
$C150-A100-KL15$	12/18	9	21

The baseline achieved all eight successes in only four regions, showing a strong concentration of successful grasps in a limited part of the workspace. Increasing the KL weight to 15 expanded the successful coverage to ten regions.

The $C150-A100-KL10$ configuration succeeded in seven regions. Six of these regions achieved two successes, indicating high repeatability in a relatively limited spatial area.

The best configuration, *C150–A100–KL15*, achieved at least one success in 12 of the 18 regions. Nine regions achieved two successes, and three additional regions achieved one success. Its improvement therefore consisted of both greater repeatability and broader spatial coverage.

Nevertheless, the best configuration still failed in six complete regions. The spatial pattern shows that the policy did not achieve uniform reliability over the complete workspace.

6.5. Failure Case Analysis

The best configuration failed in 15 of the 36 trials, while the baseline failed in 28 trials. The two configurations using 50 executed action steps failed all 36 trials.

The available records contain success labels and spatial success heatmaps but do not contain a complete trial-level classification of failure causes. Therefore, the failures cannot be assigned quantitatively to categories such as missed grasp, harness slippage, visual occlusion, insufficient gripper closure, or an unsuitable action duration.

The spatial results nevertheless reveal a position-dependent limitation. Several configurations succeeded only in a small part of the workspace. The baseline, for example, achieved repeated success in four regions but failed in the remaining 14 regions. The structured distribution of these failures suggests that performance depended on the initial harness position and the corresponding visual observation and approach trajectory.

The zero-success results of the two 50-step configurations provide evidence of an execution-horizon limitation. However, because the failure phase was not recorded, the results do not determine whether these policies failed during approach, gripper closure, or lifting.

The following categories are therefore suitable as a future annotation scheme, but no numerical frequency is claimed in the present results:

- the approach trajectory missed the harness;
- the gripper closed at an incorrect location;
- the harness was grasped but slipped during lifting;
- the gripper did not close sufficiently;
- the harness or gripper was visually occluded;

These categories are included to define the interpretation of representative failure images. They are not used as quantitative results because the corresponding trial-level labels were unavailable.

7. Discussion

This chapter interprets the experimental results presented in the previous chapter, discusses the strengths and limitations of the proposed system, and places the findings in the broader context of robot learning. Particular attention is given to the relationship between offline action-prediction metrics and closed-loop physical performance, the influence of the ACT configuration, and the spatial variation observed during the real-world evaluation.

7.1. Interpretation of Results

The experimental results show that ACT can learn a coordinated wire-harness grasping and lifting behaviour from real robot demonstrations under the controlled conditions used in this study. Successful executions contained the required sequence of approaching the harness, aligning the gripper, closing the gripper, retaining the harness, and lifting it from the work surface. This indicates that the policy learned a temporally coordinated manipulation behaviour without requiring an explicit analytical model of harness deformation or a separately programmed trajectory for every initial position.

The comparison of the ACT configurations indicates that the temporal parameters affected both offline prediction accuracy and closed-loop execution. The parameter `chunk_size` determines the future action horizon represented by one policy prediction, whereas `n_action_steps` determines how many predicted actions are executed before a new observation is used. Executing more actions from one prediction can support temporally continuous motion, but it also reduces the frequency at which the policy reacts to updated visual observations. Executing fewer actions provides more frequent closed-loop correction, but can introduce transitions between successive action chunks. Action chunking therefore creates a practical trade-off between motion continuity and responsiveness. Related work has likewise shown that inference delay and misalignment between the current observation and the executed action chunk can affect task completion and motion smoothness [53].

Within the evaluated configurations, the policy with `chunk_size = 150`, `n_action_steps = 100`, and a KL weight of 15 provided the best observed balance between offline action-prediction accuracy and physical execution performance. This result is specific to the dataset, robot, control frequency, and task considered in this study and should not be interpreted as a generally optimal ACT configuration.

The KL weight introduces a second trade-off. In ACT, the KL term regularises the latent distribution, whereas the reconstruction term encourages accurate reproduction of the demonstrated actions. A very weak KL contribution can allow the latent representation to fit details of the recorded demonstrations without providing sufficient regularisation. A very strong

KL contribution can restrict the information represented by the latent variable and can lead to overly averaged action predictions. The final training loss alone is therefore not sufficient for selecting a policy; its components and its physical closed-loop behaviour must be considered together.

The experiments further demonstrate that offline action-prediction error is not a direct estimate of real-world task success. Because all 195 episodes were used for training, the reported offline errors primarily measure reconstruction within the recorded demonstration distribution. During autonomous execution, even a small action error changes the next robot state and camera observation. The policy can consequently enter states that are poorly represented in the demonstrations, where later prediction errors may accumulate. Previous imitation-learning research has similarly shown that decreasing validation loss does not necessarily correspond to increasing closed-loop task success [14]. The Action L1, gripper L1 and smoothness measures should therefore be interpreted as complementary diagnostic metrics rather than substitutes for physical task evaluation.

The spatial evaluation showed that performance was not uniformly distributed over the workspace. Successful trials occurred more frequently in the right-hand region. One possible explanation is operator-induced spatial bias: regions that were easier to reach during leader-arm teleoperation may contain smoother and more consistent demonstrations. Camera visibility, table occlusion, robot kinematics, and restricted operator motion may also have contributed to the difference between workspace regions. Because these factors were not varied and measured independently, they should be treated as plausible explanations rather than experimentally verified causes.

Finally, the evaluation illustrates why successful video examples alone are insufficient for assessing a learned robot policy. The 36 physical trials across 18 workspace regions reveal spatial variability that would not be visible from a small number of selected demonstrations. Offline prediction error and motion smoothness provide additional information about trajectory reproduction and execution behaviour. The unsuccessful executions also reveal individual failure behaviours. However, because the available records do not contain a complete and consistent trial-level assignment of the predefined failure categories, category-specific failure frequencies cannot be reported reliably.

7.2. Strengths of the Proposed System

A central strength of the proposed system is that it covers the complete experimental workflow from real-world demonstration collection to autonomous physical execution. The SO-101 leader-follower setup supports direct teleoperation, while LeRobot provides consistent interfaces for data recording, dataset organisation, policy training, and deployment. This reduces the gap between algorithm development and evaluation on real hardware.

The dataset contains synchronised visual and proprioceptive information. The fixed top

camera observes the global workspace, the front camera records local gripper–harness interactions, and the robot state describes the current arm and gripper configuration. This multimodal structure allows the ACT policy to condition its action predictions on both the visual scene and the physical robot state. The standardised LeRobot dataset format also makes the collected data reusable for future comparisons with other policy architectures.

ACT is well matched to a task that consists of several temporally related motion phases. Instead of predicting each motor command independently, the policy predicts a sequence of future actions. This allows the learned representation to capture relationships between approach, alignment, grasping, transport, and lifting. The configuration experiments additionally provide practical evidence that the prediction horizon and executed horizon are important deployment parameters rather than merely implementation details.

Another strength is that the evaluation extends beyond qualitative video demonstrations. The policy was evaluated through repeated physical trials distributed across 18 workspace regions. Task success, spatial success distribution, offline action-prediction error, and motion smoothness describe different aspects of the learned behaviour. Representative unsuccessful executions were qualitatively inspected without claiming unsupported category-level failure frequencies. This combination provides a more informative assessment than reporting only selected successful executions.

Although the ACT policy used in this study is not a robotic foundation model, the system contains several elements that are relevant to larger robot-learning pipelines: multimodal real-world data collection, a standardised dataset, reusable policy interfaces, and end-to-end deployment on physical hardware. Robotic foundation models aim to achieve broader transfer across tasks and environments through large-scale pre-training, but data availability, reliable generalisation, safety, and real-time deployment remain important challenges [54]. The present platform therefore provides a small-scale and reproducible test case for investigating task-specific policy learning and for later comparisons with more general policies.

From a practical perspective, the system demonstrates that demonstration-based learning is feasible for a constrained wire-harness manipulation task. It provides a basis for further work on automated cable handling, flexible-object manipulation, and wire-harness assembly using accessible robot hardware and open-source software.

7.3. Limitations

The first limitation concerns the scale and composition of the dataset. The policy was trained using 195 demonstration episodes collected by a single operator. Although the dataset covers different locations within the predefined workspace, it cannot represent all possible harness shapes, approach directions, contact states, and failure conditions. Furthermore, all episodes were used for training, without an independent validation or offline

test set. The reported offline action-prediction error therefore primarily measures reconstruction within the recorded data distribution rather than generalisation to unseen demonstrations.

The real-world evaluation also has a limited statistical scale. The 36 trials distributed over 18 workspace regions provide spatial coverage, but only two repetitions are available for each region. Individual failures can therefore have a large influence on the apparent regional success rate. More repeated trials would be required to estimate performance variability and rare failure events reliably.

The data-collection process may contain operator-induced spatial bias. Regions that were more comfortable for the operator to reach may contain smoother and more consistent trajectories, whereas less convenient regions may contain greater variation in motion and timing. Because trajectory quality was not measured separately for each region, the contributions of robot kinematics, camera visibility, policy behaviour, and demonstration quality cannot be isolated.

A further limitation is the difference between the controlled experimental environment and an industrial workspace. The background, lighting, camera poses, robot base, and worktable arrangement remained largely unchanged during data collection and testing. An industrial scene may contain tools, fixtures, boxes, connectors, neighbouring components, workers, and other machinery. These elements can introduce clutter, reflections, shadows, dynamic occlusion, and background textures that were not represented in the training dataset.

The current policy may consequently rely partly on visual correlations specific to the experimental workbench. Changes in background colour, illumination, camera position, or workspace layout may cause a visual domain shift and reduce policy performance. The experiments demonstrate generalisation mainly across positions within the predefined workspace rather than across different visual environments.

The two camera views were designed to provide complementary global and local observations. However, because no single-camera ablation experiment was conducted, the individual contribution of each camera to task performance cannot be quantified. The current results therefore support the feasibility of the combined observation design, but do not demonstrate that two cameras outperform a corresponding single-camera configuration.

The experiments considered only one wire harness under controlled initial conditions. Industrial production may involve harnesses with different lengths, diameters, colours, stiffness values, branching structures, surface friction, connector geometries, and mass distributions. These variations affect both visual appearance and physical behaviour. The current results therefore do not demonstrate generalisation to previously unseen harness types.

Multiple-harness scenes introduce an additional level of complexity. Harnesses may overlap, cross, or become entangled, making it necessary to distinguish individual instances

and determine which visible segments belong to the same object. The robot must also select the correct target and avoid unintentionally lifting or entangling neighbouring cables. The current policy does not explicitly perform target identification, instance segmentation, or topology estimation, and its performance in such scenes remains unknown.

The policy is based on offline imitation learning and depends strongly on the coverage and quality of the demonstrations. It does not actively explore new states or explicitly estimate whether the current observation lies outside the training distribution. When an approach or grasp begins to fail, the policy has no dedicated mechanism for detecting the failure, stopping the current action, selecting a new grasp point, or generating a recovery trajectory. The system also lacks explicit high-level task planning, geometric grasp planning, collision-aware motion planning, and safety verification. Constraints such as obstacle avoidance, joint limits, grasp feasibility, and safe recovery are represented only indirectly through the demonstrations. A practical industrial system would likely require a hierarchical architecture in which the learned policy is combined with target selection, task planning, motion planning, safety monitoring, and failure recovery.

The sensing and hardware configuration impose further limitations. RGB images and joint positions provide no direct measurement of depth, contact force, cable tension, or grasp stability. Harness slippage and insufficient gripper contact must therefore be inferred indirectly from visual information. Depth, force–torque, tactile, or gripper-force sensing could provide more reliable information for grasp verification and force-sensitive assembly.

In addition, the repeatability and mechanical behaviour of the low-cost robot and gripper affect the final result. Even when the predicted action is appropriate, positioning errors, gripper calibration, limited closing force, or changing friction conditions may cause an unsuccessful grasp. The observed performance therefore represents the combined behaviour of the learned policy, the perception system, the robot mechanics, and the experimental setup.

Finally, the current task includes grasping and lifting a single wire harness, but does not cover the complete industrial assembly process. Connector identification, harness topology reconstruction, untangling, routing, insertion, clipping, force-controlled assembly, and final quality verification remain outside the scope of the present system. Long-term operation, production cycle time, safety certification, hardware wear, and industrial reliability were also not evaluated.

The proposed system should therefore be regarded as a proof of concept for vision-based, demonstration-driven wire-harness grasping. It demonstrates the feasibility of learning the studied behaviour from real robot data while also identifying the data, perception, planning, hardware, and evaluation requirements that must be addressed before deployment in a complex industrial environment.

8. Summary and Outlook

This chapter summarises the work completed in this thesis, presents the main experimental findings, and outlines directions for extending the current wire-harness manipulation system.

8.1. Summary

This thesis investigated whether an end-to-end imitation-learning policy could learn a wire-harness manipulation task directly from demonstrations collected on a physical robot. A low-cost SO-101 leader–follower system was established for teleoperation, data collection, policy deployment, and real-world evaluation. The experimental setup included a fixed top camera for observing the global workspace and a front camera mounted near the gripper for recording local interactions between the gripper and the wire harness.

The LeRobot framework was used to connect the hardware interfaces, record synchronised multimodal demonstrations, organise the resulting dataset, train the robot policy, and deploy the trained model on the follower arm. Each policy observation consisted of the top-camera image, the front-camera image, and the current six-dimensional robot state. The policy output consisted of a six-dimensional robot action representing five arm joints and the gripper.

A dataset containing 195 real robot demonstration episodes was collected by a single operator. The demonstrations covered different positions within the predefined workspace and contained the complete sequence from the initial robot configuration to approaching, grasping, and lifting the wire harness. The dataset was used to train an Action Chunking with Transformers policy. Several training configurations were compared by varying parameters including `chunk_size`, `n_action_steps`, the KL weight, and the learning rate.

The trained policies were first examined using offline action-prediction metrics. A physical evaluation was then conducted through 36 real-world trials distributed across 18 workspace regions. The evaluation combined task success, spatial success distribution, offline action-prediction error, motion smoothness, and qualitative inspection of representative unsuccessful executions.

Overall, the thesis established an end-to-end experimental pipeline covering hardware and software integration, multimodal data collection, ACT policy training, hyperparameter comparison, real-world deployment, and quantitative physical evaluation.

8.2. Main Findings

ACT learned a coordinated wire-harness grasping and lifting behaviour from the collected demonstrations under the controlled experimental conditions. The successful executions show that the studied behaviour can be learned without explicitly modelling harness deformation or manually programming a separate trajectory for every initial position.

The temporal ACT configuration influenced both offline action-prediction accuracy and closed-loop execution behaviour. Among the evaluated configurations, a KL weight of 15, `chunk_size = 150` and `n_action_steps = 100` provided the best observed balance between offline and physical performance.

Offline action-prediction error alone was insufficient for identifying the best-performing real-world policy. Offline metrics must therefore be complemented by closed-loop physical evaluation.

Policy performance was not uniformly distributed across the workspace. Successful trials occurred more frequently in the right-hand region, showing that spatial coverage is an important part of real-world policy evaluation.

Finally, the results confirm that video demonstrations alone are insufficient for assessing reliability. Task success, spatial performance, offline prediction error, and motion smoothness reveal complementary aspects of policy behaviour, while qualitative inspection of unsuccessful executions can identify individual failure behaviours without implying unsupported failure-frequency statistics.

8.3. Future Work

The first direction for future work is to increase the scale, diversity, and balance of the demonstration dataset. Additional demonstrations should cover a wider range of wire-harness positions, orientations, deformations, and grasping approaches. Demonstration quantity and quality should be balanced across the left, centre, and right regions of the workspace to reduce operator-induced spatial bias. Collecting demonstrations from multiple operators would also reduce dependence on a single teleoperation style.

Future datasets should include harnesses with different lengths, diameters, colours, stiffness values, branching structures, connector types, and initial configurations. Complex backgrounds, different lighting conditions, partial occlusions, camera displacement, and industrial objects should be introduced to evaluate visual generalisation. Separate training, validation, and test subsets should be created, and multiple training runs with different random seeds should be conducted. More physical trials per workspace region would provide a more reliable estimate of policy variability and rare failures.

A particularly important extension is the transition from a single harness to multi-harness scenes. Multiple harnesses may overlap, cross, or become entangled. A future system

would need to identify the target harness, distinguish individual harness instances, estimate which visible segments belong to the same harness, and select a grasp point that does not unintentionally move neighbouring cables. Instance segmentation, centreline extraction, keypoint detection, or explicit topology estimation could therefore be combined with the learned manipulation policy.

The existing YOLO-based plug detector provides a direct starting point for extending the system beyond grasping and lifting. It should be integrated into the robot pipeline so that the target connector can be identified before manipulation begins. The detector can provide the connector class and two-dimensional image location, but an additional pose-estimation method is required to determine the connector position and orientation needed for robotic handling. Liu and Huang proposed connector pose-estimation strategies adapted to different structural properties and demonstrated generalisation across multiple connector types [55]. Such pose information could be combined with motion planning or compliant control for precise connector alignment and insertion.

A future hierarchical pipeline could consist of plug detection, target selection, harness grasping, connector pose estimation, motion planning, insertion, and success verification. In this architecture, the YOLO model would provide semantic target information, while ACT would generate temporally coordinated actions for grasping and locally manipulating the flexible harness. A geometric planner integrated through MoveIt or OMPL could generate collision-aware motion towards the socket. Visual servoing or force-based control could then support final alignment and insertion.

The integrated system should account for detector confidence, partial occlusion, and incorrect classification. The robot should not begin an operation when the connector type or target location is uncertain. It could instead acquire another observation, adjust the viewpoint, request human confirmation, or stop safely. After integration, the evaluation should include not only detector precision, recall, and mean average precision, but also target-selection success, grasp success, insertion success, complete task success, execution time, and the number of failures caused by perception errors.

Another direction is to add depth, force, torque, or tactile sensing. These modalities could provide information that is not directly available from RGB images, including grasp contact, cable tension, connector alignment, and harness slippage. Online grasp verification, failure detection, automatic re-grasping, and recovery policies would improve robustness when the robot deviates from the expected trajectory.

Future experiments should also compare ACT with alternative robot policies, including Diffusion Policy, SmoIVLA, π_0 , and π_0 -FAST. A fair comparison should use the same dataset, observation modalities, training budget, robot platform, and physical testing protocol. In addition to task success, the comparison should examine inference latency, execution time, motion smoothness, computational requirements, robustness to unseen conditions, and recovery behaviour.

Robotic foundation models represent a longer-term research direction. Large-scale pre-trained policies could be fine-tuned using the collected LeRobot dataset and compared with ACT trained specifically for the current task. Such experiments could investigate whether pre-training improves transfer to unseen harness types, new visual environments, or related manipulation tasks. Their computational cost, real-time performance, hardware requirements, and dependence on large-scale pre-training data must also be considered.

Future work could additionally investigate alternative representations of continuous actions. ActionMap replaces direct action regression with probability distributions over voxelised action spaces and has demonstrated improved performance and data efficiency on the LIBERO benchmark and real-world Franka manipulation tasks [56]. Although it is not directly comparable with the joint-space ACT policy used in this study, it indicates that action representation is another relevant design dimension alongside chunk size, execution horizon, and KL weight.

Finally, the spatial evaluation protocol could be extended into a more standardised real-world benchmark for wire-harness manipulation. The benchmark could contain controlled and cluttered backgrounds, single- and multi-harness scenes, different connector types, partial occlusion, and increasingly complex tasks such as grasping, routing, alignment, and insertion. Larger-scale testing should additionally evaluate long-term reliability, cycle time, hardware wear, failure recovery, and safety.

The most direct continuation of this thesis is the integration of the existing YOLO connector detector with the ACT manipulation policy, connector pose estimation, collision-aware motion planning, and closed-loop insertion control. This would extend the current system from isolated wire-harness grasping towards a more complete perception–planning–control pipeline for automated wire-harness assembly.

Bibliography

- [1] J. Trommnau, J. Kühnle, J. Siegert, R. Inderka, and T. Bauernhansl, "Overview of the state of the art in the production process of automotive wire harnesses, current research and future trends," *Procedia CIRP*, vol. 81, pp. 387–392, 2019.
- [2] S. Olbrich and J. Lackinger, "Manufacturing processes of automotive high-voltage wire harnesses: State of the art, current challenges and fields of action to reach a higher level of automation," *Procedia CIRP*, vol. 107, pp. 653–660, 2022.
- [3] X. Jiang, K.-m. Koo, K. Kikuchi, A. Konno, and M. Uchiyama, "Robotized assembly of a wire harness in a car production line," *Advanced Robotics*, vol. 25, no. 3-4, pp. 473–489, 2011.
- [4] O. Salunkhe et al., "Review of current status and future directions for collaborative and semi-automated automotive wire harnesses assembly," *Procedia CIRP*, vol. 120, pp. 696–701, 2023.
- [5] H. Wang et al., "Overview of computer vision techniques in robotized wire harness assembly: Current state and future opportunities," *Procedia CIRP*, vol. 120, pp. 1071–1076, 2023, 56th CIRP International Conference on Manufacturing Systems 2023, ISSN: 2212-8271. DOI: <https://doi.org/10.1016/j.procir.2023.09.127>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2212827123008594>.
- [6] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," *arXiv preprint arXiv:2304.13705*, 2023.
- [7] B. Singh, N. Sellappan, and P. Kumaradhas, "Evolution of industrial robots and their applications," *International Journal of emerging technology and advanced engineering*, vol. 3, no. 5, pp. 763–768, 2013.
- [8] A. Caporali and G. Palli, "Robotic manipulation of deformable linear objects via multi-view model-based visual tracking," *IEEE/ASME Transactions on Mechatronics*, 2025.
- [9] M. Zürn et al., "Robotic wiring harness bin picking solution using a deep-learning-based spline prediction and a multi-stereo camera setup," in *Advances in Automotive Production Technology – Digital Product Development and Manufacturing*, D. Holder, F. Wulle, and J. Lind, Eds., Cham: Springer Nature Switzerland, 2025, pp. 319–334, ISBN: 978-3-031-88831-1.
- [10] S. Espana et al., "Grip points generation on motorcycle wire harnesses main branch based on computer vision and clustering," in *2024 10th International Conference on Automation, Robotics and Applications (ICARA)*, IEEE, 2024, pp. 117–122.

- [11] X. Zhang, H.-C. Lin, Y. Zhao, and M. Tomizuka, "Harnessing with twisting: Single-arm deformable linear object manipulation for industrial harnessing task," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2024, pp. 4069–4075.
- [12] K. Kamiya et al., "Learning from human hand demonstration for wire harness grasping," in *2024 IEEE International Conference on Advanced Intelligent Mechatronics (AIM)*, 2024, pp. 1645–1650. DOI: 10.1109/AIM55361.2024.10637203.
- [13] A. Mandlekar et al., "What matters in learning from offline human demonstrations for robot manipulation," *arXiv preprint arXiv:2108.03298*, 2021.
- [14] M. Khansari et al., "Practical imitation learning in the real world via task consistency loss," *arXiv preprint arXiv:2202.01862*, 2022.
- [15] R. Cadene et al., "Lerobot: An open-source library for end-to-end robot learning," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.22818>.
- [16] B. Fang, S. Jia, D. Guo, M. Xu, S. Wen, and F. Sun, "Survey of imitation learning for robotic manipulation: B. fang et al.," *International Journal of Intelligent Robotics and Applications*, vol. 3, no. 4, pp. 362–369, 2019.
- [17] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, "Imitation learning: A survey of learning methods," *ACM Computing Surveys (CSUR)*, vol. 50, no. 2, pp. 1–35, 2017.
- [18] C. G. Atkeson and S. Schaal, "Robot learning from demonstration," in *ICML*, vol. 97, 1997, pp. 12–20.
- [19] M. Seo et al., "Deep imitation learning for humanoid loco-manipulation through human teleoperation," in *2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids)*, IEEE, 2023, pp. 1–8.
- [20] D. Dwibedi, J. Tompson, C. Lynch, and P. Sermanet, "Learning actionable representations from visual observations," in *2018 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, IEEE, 2018, pp. 1577–1584.
- [21] S. Bahl, R. Mendonca, L. Chen, U. Jain, and D. Pathak, "Affordances from human videos as a versatile representation for robotics," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 13 778–13 790.
- [22] A. Mohtasib, G. Neumann, and H. Cuayáhuitl, "A study on dense and sparse (visual) rewards in robot policy learning," in *Annual Conference Towards Autonomous Robotic Systems*, Springer, 2021, pp. 3–13.
- [23] J. Zhang et al., "Extract: Efficient policy learning by extracting transferable robot skills from offline data," *arXiv preprint arXiv:2406.17768*, 2024.

- [24] H. Zhou, S. Li, Q. Lu, and J. Qian, "A practical solution to deformable linear object manipulation: A case study on cable harness connection," in *2020 5th International Conference on Advanced Robotics and Mechatronics (ICARM)*, 2020, pp. 329–333. DOI: 10.1109/ICARM49381.2020.9195380.
- [25] Y. Gao, Z. Chen, Y.-H. Liu, M. Fang, and J. Lin, "Vision-based grasping and manipulation of flexible usb wires," in *2019 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 2019, pp. 1083–1088. DOI: 10.1109/ROBIO49542.2019.8961848.
- [26] R. Lee, M. Hamaya, T. Murooka, Y. Ijiri, and P. Corke, "Sample-efficient learning of deformable linear object manipulation in the real world through self-supervision," *IEEE Robotics and Automation Letters*, vol. 7, no. 1, pp. 573–580, 2022. DOI: 10.1109/LRA.2021.3130377.
- [27] K. Galassi and G. Palli, "Robotic wires manipulation for switchgear cabling and wiring harness manufacturing," in *2021 4th IEEE International Conference on Industrial Cyber-Physical Systems (ICPS)*, 2021, pp. 531–536. DOI: 10.1109/ICPS49255.2021.9468128.
- [28] S. Karamcheti et al., "Language-driven representation learning for robotics," *arXiv preprint arXiv:2302.12766*, 2023.
- [29] S. Kim, K. Kim, D. Ko, Y. J. Heo, and M. J. Kim, "Spatially conditioned diffusion policy: Learning precise and robust manipulation with a single rgb camera," *arXiv preprint arXiv:2606.14535*, 2026.
- [30] S. Jeong et al., "Learning multi-view spatial reasoning from cross-view relations," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 2570–2581.
- [31] Y. Bai et al., "Learning to see and act: Task-aware virtual view exploration for robotic manipulation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 13 386–13 396.
- [32] M. Ding, Y. Liu, Y. Shi, X. Lan, and N. Zheng, "Dual graph attention networks for multi-view visual manipulation relationship detection and robotic grasping," *IEEE Transactions on Automation Science and Engineering*, 2025.
- [33] K. Han et al., "A survey on visual transformer," *arXiv preprint arXiv:2012.12556*, 2020.
- [34] Y. Liu, J. Hamid, A. Xie, Y. Lee, M. Du, and C. Finn, "Bidirectional decoding: Improving action chunking via guided test-time sampling," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 4594–4627.
- [35] A. George and A. B. Farimani, "One act play: Single demonstration behavior cloning with action chunking transformers," *arXiv preprint arXiv:2309.10175*, 2023.

- [36] H. Kim, Y. Ohmura, and Y. Kuniyoshi, "Transformer-based deep imitation learning for dual-arm robot manipulation," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2021, pp. 8965–8972.
- [37] Y. Zhu, Z. Ou, F. Feng, and J. Tang, "Any2policy: Learning visuomotor policy with any-modality," *Advances in Neural Information Processing Systems*, vol. 37, pp. 133 518–133 540, 2024.
- [38] W. Zhang et al., "Dreamvla: A vision-language-action model dreamed with comprehensive world knowledge," *Advances in Neural Information Processing Systems*, vol. 38, pp. 24 195–24 228, 2026.
- [39] B. Ivanovic, E. Schmerling, K. Leung, and M. Pavone, "Generative modeling of multi-modal multi-human behavior," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2018, pp. 3088–3095.
- [40] B. Ivanovic, K. Leung, E. Schmerling, and M. Pavone, "Multimodal deep generative models for trajectory prediction: A conditional variational autoencoder approach," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 295–302, 2020.
- [41] B. D. Argall, S. Chernova, M. Veloso, and B. Browning, "A survey of robot learning from demonstration," *Robotics and autonomous systems*, vol. 57, no. 5, pp. 469–483, 2009.
- [42] B. D. Argall, B. Browning, and M. Veloso, "Learning robot motion control with demonstration and advice-operators," in *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*, IEEE, 2008, pp. 399–404.
- [43] P. M. Scheikl et al., "Movement primitive diffusion: Learning gentle robotic manipulation of deformable objects," *IEEE Robotics and Automation Letters*, vol. 9, no. 6, pp. 5338–5345, 2024.
- [44] Y. Seo, J. Kim, S. James, K. Lee, J. Shin, and P. Abbeel, "Multi-view masked world models for visual robotic manipulation," in *International Conference on Machine Learning*, PMLR, 2023, pp. 30 613–30 632.
- [45] R. Firoozi et al., "Foundation models in robotics: Applications, challenges, and the future," *The International Journal of Robotics Research*, vol. 44, no. 5, pp. 701–739, 2025.
- [46] K. Kawaharazuka, T. Matsushima, A. Gambardella, J. Guo, C. Paxton, and A. Zeng, "Real-world robot applications of foundation models: A review," *Advanced Robotics*, vol. 38, no. 18, pp. 1232–1254, 2024.
- [47] K. Black et al., "*pi_0*: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.
- [48] M. Shukor et al., "Smolvla: A vision-language-action model for affordable and efficient robotics," *arXiv preprint arXiv:2506.01844*, 2025.

- [49] X. Li et al., “Towards generalist robot policies: What matters in building vision-language-action models,” *arXiv preprint arXiv:2412.14058*, vol. 3, no. 7, p. 8, 2024.
- [50] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, “Rlbench: The robot learning benchmark & learning environment,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.
- [51] B. Liu et al., “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [52] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, “Chomp: Gradient optimization techniques for efficient motion planning,” in *2009 IEEE international conference on robotics and automation*, IEEE, 2009, pp. 489–494.
- [53] H. Wang, G. Zhang, Y. Yan, Y. Shang, R. Kompella, and G. Liu, “Real-time robot execution with masked action chunking,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 120 161–120 179.
- [54] Z. Xu et al., “A survey on robotics with foundation models: Toward embodied ai,” *arXiv preprint arXiv:2402.02385*, 2024.
- [55] S. Liu and Y. Huang, “Pose estimation for connector insertion in robotic peg-in-hole assembly,” in *2025 IEEE International Conference on Robotics and Biomimetics (RO-BIO)*, 2025, pp. 1390–1395. DOI: 10.1109/ROBIO66223.2025.11377292.
- [56] P. Yang, H. Ci, Y. Chen, Q. Lv, H. Cai, and M. Z. Shou, “Actionmap: Robot policy learning via voxel action heatmap,” *arXiv preprint arXiv:2606.06904*, 2026.

A. Supplementary Materials

A.1. Complete Spatial Evaluation Results

The main Results chapter compares four representative heat maps in Figure 6.2. For completeness, the remaining configuration-specific spatial results are retained here and discussed as supporting evidence. Figures A.1, A.6, A.7, and A.8 show the complete retained set. Each cell reports the number of successful trials out of two and uses the common grid defined in Figure 4.3.

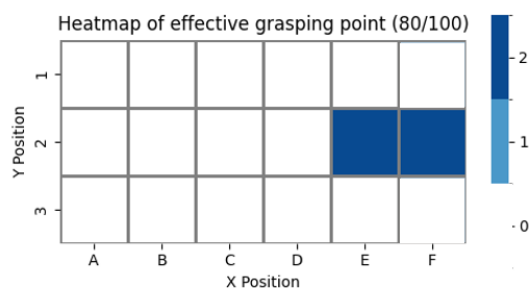


Figure A.1.: C100–A80 policy successful-grasp heat map.

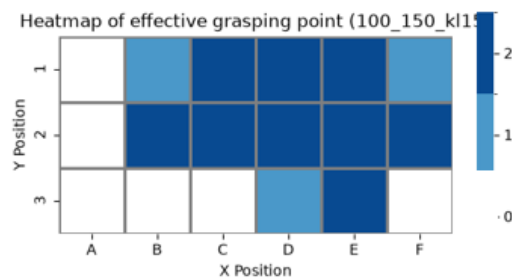


Figure A.2.: C150–A100–KL15 policy successful-grasp heat map.

A.2. Complete Offline Evaluation Results

Figures A.9, A.10, A.11, A.12 and A.13 provide the retained 1000-frame Action L1 curves that supplement the representative comparison in Figure 6.1. The curves are offline replay results and are interpreted using the metric definition in Section 4.4.9; they are not training-loss histories.

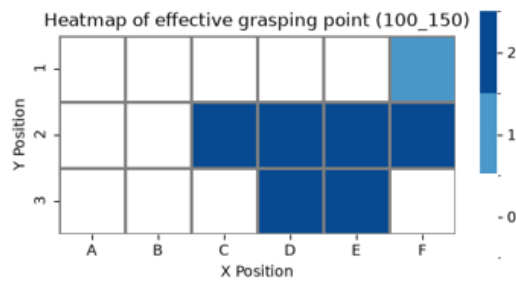


Figure A.3.: C150–A100 policy successful-grasp heat map.

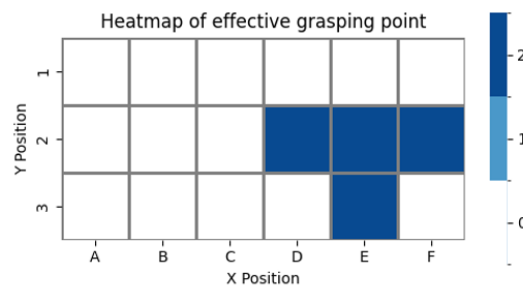


Figure A.4.: Baseline policy successful-grasp heat map.

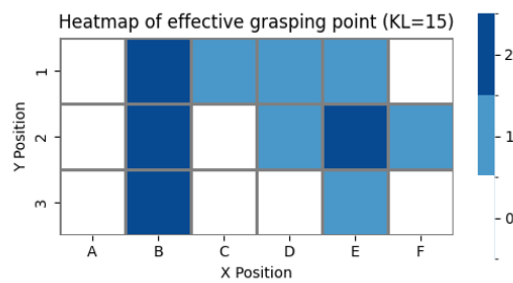


Figure A.5.: KL15 policy successful-grasp heat map.

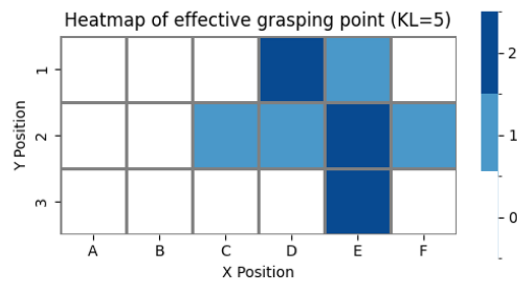


Figure A.6.: KL5 policy successful-grasp heat map.

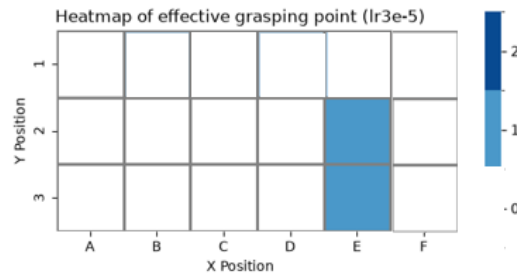


Figure A.7.: $LR3 \times 10^{-5}$ policy successful-grasp heat map.

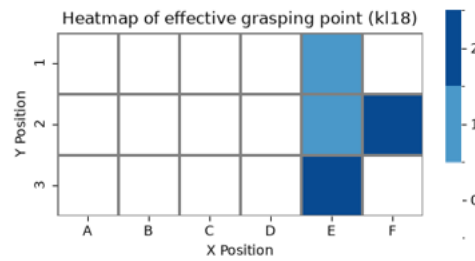


Figure A.8.: $KL18$ policy successful-grasp heat map.

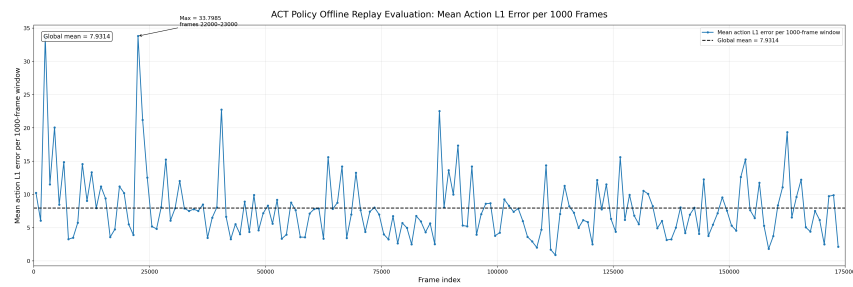


Figure A.9.: $KL18$ policy offline evaluation.

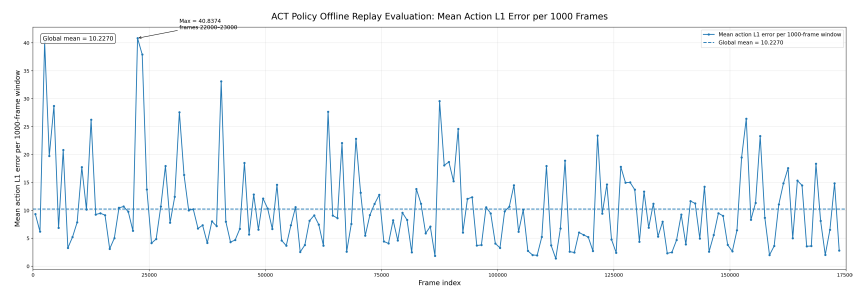


Figure A.10.: $C100-A80$ policy offline evaluation.

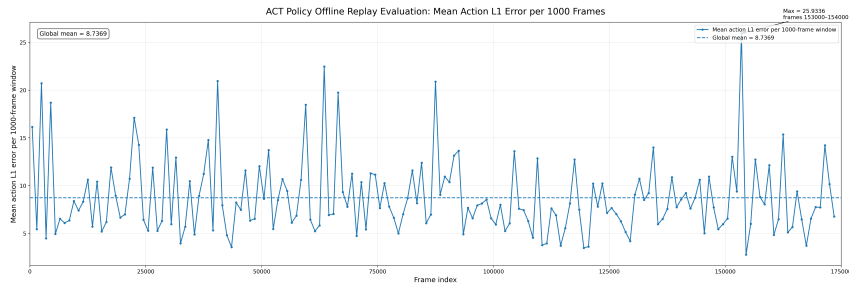


Figure A.11.: *C150–A100 policy offline evaluation.*

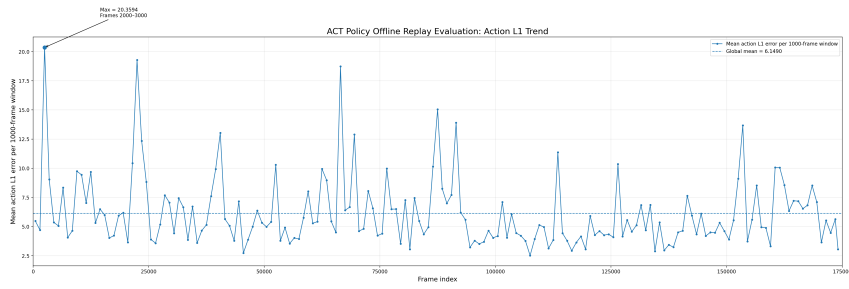


Figure A.12.: *C100–A100–KL15 policy offline evaluation.*

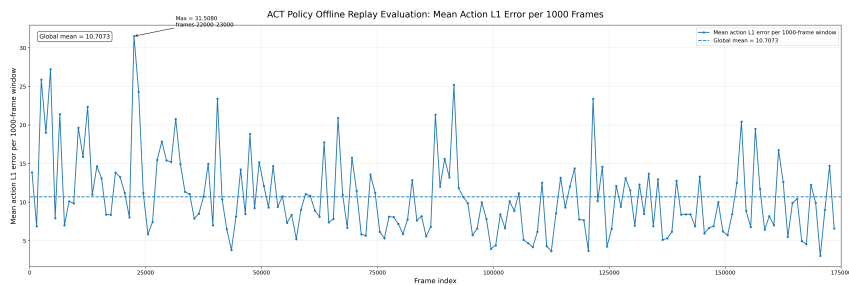


Figure A.13.: *KL5 policy offline evaluation.*

B. Reproducibility and Code Availability

The Python source code used for policy evaluation, and heat-map generation is maintained outside the thesis. The complete version, including environment information and execution instructions, should be cited through the permanent repository link below:

Code repository: <https://github.com/kb127XY/thesis.git>

demonstration dataset is available separately at <https://huggingface.co/datasets/kb127/test195>.

The repository should record the tested LeRobot version, policy checkpoint paths, command-line arguments, and the trial-log files from which the reported tables and heat maps were generated. Raw Python listings are intentionally not duplicated in this thesis because the repository is the version-controlled and executable source.

B.1. Offline Evaluation Procedure

Algorithm 3 gives only the implementation detail needed to interpret the reported offline metrics. Evaluation is performed episode by episode so that action differences are never calculated across an episode boundary. Predictions are inverse-standardised before comparison with the recorded actions.

Algorithm 3 Episode-aware offline policy replay.

Require: Recorded dataset $\mathcal{D}$ and trained policy ACT

```

1: for each episode  $\mathcal{E}_i$  in  $\mathcal{D}$  do
2:   Reset policy and action-queue state
3:   for each recorded observation  $o_t$  in  $\mathcal{E}_i$  do
4:     Predict an action and apply inverse standardisation
5:     Compare  $\hat{a}_t$  with the recorded action  $a_t$ 
6:     Store per-frame errors and predicted actions
7:   end for
8:   Compute temporal differences within  $\mathcal{E}_i$ 
9: end for
10: Aggregate Action L1, Gripper L1, windowed error, and variation
```

B.2. Spatial Heat-Map Generation

For each policy, the validated trial log is converted into a 3×6 matrix using the cell convention in Figure 4.3. Each matrix entry is an integer in $\{0, 1, 2\}$ equal to the number of

successful trials in that cell. A zero therefore denotes two failed trials and must not be encoded as a missing value. The resulting figures are reported once: representative heat maps appear in Figure 6.2, and the remaining retained maps are referenced and interpreted in Appendix A.1.